IEEE SA
STANDARDS
ASSOCIATION

IEEE Standard Testability Method for Embedded Core-based Integrated Circuits

STANDARDS

IEEE Computer Society

Developed by the
Test Technology Standards Committee

IEEE Std 1500™-2022
(Revision of IEEE Std 1500-2005)

IEEE

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

IEEE Std 1500™-2022
(Revision of IEEE Std 1500-2005)

IEEE Standard Testability Method for Embedded Core-based Integrated Circuits

Developed by the

Test Technology Standards Committee
of the
IEEE Computer Society

Approved 16 June 2022

IEEE SA Standards Board

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Abstract: A mechanism for the test of core designs within a system on chip (SoC) is defined. This mechanism is a hardware architecture and the core test language (CTL) is leveraged to facilitate communication between core designers and core integrators.

Keywords: core test, embedded core test, IEEE 1500, IP test, test reuse

The Institute of Electrical and Electronics Engineers, Inc.
3 Park Avenue, New York, NY 10016-5997, USA

Copyright © 2022 by The Institute of Electrical and Electronics Engineers, Inc.
All rights reserved. Published 12 October 2022. Printed in the United States of America.

IEEE is a registered trademark in the U.S. Patent & Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated.

PDF: ISBN 978-1-5044-8866-2 STD25531
Print: ISBN 978-1-5044-8867-9 STDPD25531

IEEE prohibits discrimination, harassment, and bullying.
For more information, visit https://www.ieee.org/about/corporate/governance/p9-26.html.
No part of this publication may be reproduced in any form, in an electronic retrieval system or otherwise, without the prior written permission of the publisher.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Important Notices and Disclaimers Concerning IEEE Standards Documents

IEEE Standards documents are made available for use subject to important notices and legal disclaimers. These notices and disclaimers, or a reference to this page (https://standards.ieee.org/ipr/disclaimers.html), appear in all standards and may be found under the heading "Important Notices and Disclaimers Concerning IEEE Standards Documents."

Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE Standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

IEEE makes no warranties or representations concerning its standards, and expressly disclaims all warranties, express or implied, concerning this standard, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. In addition, IEEE does not warrant or represent that the use of the material contained in its standards is free from patent infringement. IEEE standards documents are supplied "AS IS" and "WITH ALL FAULTS."

Use of an IEEE standard is wholly voluntary. The existence of an IEEE Standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

In publishing and making its standards available, IEEE is not suggesting or rendering professional or other services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any other person or entity to another. Any person utilizing any IEEE Standards document, should rely upon his or her own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Translations

The IEEE consensus development process involves the review of documents in English only. In the event that an IEEE standard is translated, only the English version published by IEEE is the approved IEEE standard.

Official statements

A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations Manual shall not be considered or inferred to be the official position of IEEE or any of its committees and shall not be considered to be, nor be relied upon as, a formal position of IEEE. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall make it clear that the presenter's views should be considered the personal views of that individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working Group. Statements made by volunteers may not represent the formal position of their employer(s) or affiliation(s).

Comments on standards

Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of membership affiliation with IEEE or IEEE SA. However, **IEEE does not provide interpretations, consulting information, or advice pertaining to IEEE Standards documents**.

Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is important that any responses to comments and questions also receive the concurrence of a balance of interests. For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board of Governors are not able to provide an instant response to comments, or questions except in those cases where the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation requests. Any person who would like to participate in evaluating comments or in revisions to an IEEE standard is welcome to join the relevant IEEE working group. You can indicate interest in a working group using the Interests tab in the Manage Profile & Interests area of the IEEE SA myProject system.[1] An IEEE Account is needed to access the application.

Comments on standards should be submitted using the Contact Us form.[2]

Laws and regulations

Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory requirements. Implementers of the standard are responsible for observing or referring to the applicable regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in compliance with applicable laws, and these documents may not be construed as doing so.

[1] Available at: https://development.standards.ieee.org/myproject-web/public/view.html#landing.
[2] Available at: https://standards.ieee.org/content/ieee-standards/en/about/contact/index.html.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Data privacy

Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and data ownership in the context of assessing and using the standards in compliance with applicable laws and regulations.

Copyrights

IEEE draft and approved standards are copyrighted by IEEE under US and international copyright laws. They are made available by IEEE and are adopted for a wide variety of both public and private uses. These include both use, by reference, in laws and regulations, and use in private self-regulation, standardization, and the promotion of engineering practices and methods. By making these documents available for use and adoption by public authorities and private users, neither IEEE nor its licensors waive any rights in copyright to the documents.

Photocopies

Subject to payment of the appropriate licensing fees, IEEE will grant users a limited, non-exclusive license to photocopy portions of any individual standard for company or organizational internal use or individual, non-commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400; https://www.copyright.com/. Permission to photocopy portions of any individual standard for educational classroom use can also be obtained through the Copyright Clearance Center.

Updating of IEEE Standards documents

Users of IEEE Standards documents should be aware that these documents may be superseded at any time by the issuance of new editions or may be amended from time to time through the issuance of amendments, corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the document together with any amendments, corrigenda, or errata then in effect.

Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE standard.

In order to determine whether a given document is the current edition and whether it has been amended through the issuance of amendments, corrigenda, or errata, visit IEEE Xplore or contact IEEE.[3] For more information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

Errata

Errata, if any, for all IEEE standards can be accessed on the IEEE SA Website.[4] Search for standard number and year of approval to access the web page of the published standard. Errata links are located under the Additional Resources Details section. Errata are also available in IEEE Xplore. Users are encouraged to periodically check for errata.

[3] Available at: https://ieeexplore.ieee.org/browse/standards/collection/ieee.
[4] Available at: https://standards.ieee.org/standard/index.html.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Patents

IEEE Standards are developed in compliance with the IEEE SA Patent Policy.[5]

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE SA Website at https://standards.ieee.org/about/sasb/patcom/patents.html. Letters of Assurance may indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair discrimination to applicants desiring to obtain such licenses.

Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

IMPORTANT NOTICE

IEEE Standards do not guarantee or ensure safety, security, health, or environmental protection, or ensure against interference with or from other devices or networks. IEEE Standards development activities consider research and information presented to the standards development group in developing any safety recommendations. Other information about safety practices, changes in technology or technology implementation, or impact by peripheral systems also may be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, and interference protection practices and all applicable laws and regulations.

[5] Available at: https://standards.ieee.org/about/sasb/patcom/materials.html.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Participants

At the time this IEEE standard was completed, the Embedded Core Test Working Group had the following membership:

Mike Ricchetti, *Chair*
Grady Giles, *Vice Chair*
Paul Reuter, *Editor*
Ben Niewenhuis, *Secretary*

Saman Adham
Roberto Benavides
Tapan Chakraborty
Adam Cron
Heiko Ehrenberg
Saurabh Gupta
Alan Hales
Ian Harrison
Adam Ley
Teresa McLaurin
Sankaran Menon
Thomas Rinderknecht
Tom Waayers
Jae Wu

Previous members of the Embedded Core Test Working Group are:

Karim Arabi
Luis Basto
Sudipta Bhawmik
Wu-Tung Cheng
Vivek Chickermane
Mike Collins
Francisco da Silva
Bulent Dervisoghu
Jason Doege
Rohit Kapur
Brion Keller
Eric Jan Marinissen
Mike Mateja
Fidel Muradali
Paul Soong
Lee Whetsel
Yervant Zorian

The following members of the individual Standards Association balloting group voted on this standard. Balloters may have voted for approval, disapproval, or abstention.

Abhishek Alladi
Roberto Benavides
Bill Brown
Tapan Chakraborty
Jean-Francois Cote
Adam Cron
Heiko Ehrenberg
Grady Giles
Saurabh Gupta
Ian Harrison
Peter Harrod
Kazumi Hatayama
Werner Hoelzl
Osamu Karatsu
Michael Laisne
Jim Lewis
Adam Ley
Ting Li
Ian McIntosh
Teresa McLaurin
Benoit Nadeau-Dostie
Ben Niewenhuis
Priyadarsan Patra
Paul Reuter
Mike Ricchetti
Thomas Rinderknecht
Stephen Schwarm
Roger Sowada
Walter Struppler
John Vergis
Tom Waayers
Oren Yuen

When the IEEE SA Standards Board approved this standard on 30 June 2022, it had the following membership:

David J. Law, *Chair*
Ted Burse, *Vice Chair*
Gary Hoffman, *Past Chair*
Konstantinos Karachalios, *Secretary*

Edward A. Addy
Ramy Ahmed Fathy
J. Travis Griffith
Guido R. Hiertz
Yousef Kimiagar
Joseph L. Koepfinger*
Thomas Koshy
John D. Kulick
Johnny Daozhuang Lin
Kevin Lu
Daleep C. Mohla
Andrew Myles
Damir Novosel
Annette D. Reilly
Robby Robson
Jon Walter Rosdahl
Mark Siira
Dorothy V. Stanley
Lei Wang
F. Keith Waters
Karl Weber
Sha Wei
Philip B. Winston
Daidi Zhong

*Member Emeritus

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Introduction

This introduction is not part of IEEE Std 1500-2022, IEEE Standard Testability Method for Embedded Core-based Integrated Circuits.

IEEE Std 1500 is a scalable standard architecture for enabling test reuse and integration of embedded cores. It has serial and parallel test access mechanisms (TAMs) and a rich set of instructions suitable for testing cores, system on chip (SoC) interconnects, and circuitry. In addition, IEEE Std 1500 defines features that enable core isolation and protection. IEEE Std 1500 can reduce test costs and improve test quality by facilitating improvements in design automation, design-for-test (DFT) access, and DFT methodology.

Core test language (CTL) is the standard language for describing IEEE 1500 wrappers and hardware. The test patterns associated with cores may also be provided by CTL, or other IEEE Test Technology Standards Committee (TTSC) (or successor) sponsored standard test data languages. CTL is defined in IEEE Std 1450.6™.[6]

Objective of the IEEE 1500 effort

The Embedded Core Test Working Group was approved with the charter to develop a standard test access method for embedded cores in integrated circuits (ICs). That method is independent of the underlying functionality of the IC or its individual embedded cores. The access method supports the necessary testability requirements for detection and diagnosis of such ICs, while allowing for ease of interoperability of cores that originate from different sources. This method is usable for all classes of digital cores including hierarchical ones (15.2 discusses hierarchical core-wrapper configurations).

History and achievements of this standard

Development of IEEE Std 1500 began in 1997 with the original Working Group approval. During this period, the Embedded Core Test Working Group considered all aspects of core-based test and ensured that is would support a broad spectrum of users. It provided flexibility with both serial and parallel TAMs, definition of a core wrapper, and a set of both required and optional instructions. The standard was first published in 2005, and then reaffirmed in 2011.

Since the initial publication, and subsequent reaffirmation, IEEE Std 1500 has become widely adopted throughout the industry, with full support from IP designers and core providers, SoC designers, and electronic design automation (EDA) tool providers. IEEE Std 1500 fulfilled its scope and purpose of standardizing the reuse of nonmergeable cores, successful integration of cores into an SoC, and, through design automation, facilitating the transfer of test information and test data between core providers and core users. As the number and complexity of embedded cores used within an SoC grew, and technology advanced, the need for major revisions to IEEE Std 1500 became necessary. In 2019 a new working group was approved to revise the standard.

Changes introduced by this revision

While this is a major revision of the standard, the scope and purpose of the standard have not changed since it was last reaffirmed in 2011. The revisions in this version of the standard are for the most part optional. They are intended to provide clarifications or relaxations to the previous version's rules, and to optionally extend the capabilities of this standard to provide test access for cores and SoCs that have grown in size, number, and complexity. It is believed that these revisions were needed to support improved industry adoption and compliance. It is expected that cores that are compliant with the previous version of the standard remain compliant with this version.

[6] Information on references can be found in Clause 2.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

The major changes, listed in the order in which they appear in this standard, are as follows:

In Clause 7:

— Permissions were added for data registers to be segmented, and for the scan path selected by the wrapper instruction register (WIR) to be reconfigured as a result of a data register Update operation. In 7.2 the rules that required data registers (DRs) and the selected scan path to be fixed length were removed, and new permissions were added to allow data register segments and the data register scan path to change length. A new Figure 2 was added to 7.2.2 showing DR segments accessed by inline segment insertion bits (SIBs).

In Clause 8:

— The optional Transfer terminal was removed from the wrapper serial port (WSP) terminals, and the Transfer operation was removed throughout the body of the draft. Transfer was not used by core providers and was not supported by EDA tools. It also is not compatible with the IEEE 1149.1™ test access port (TAP) controller. It was originally intended to support at-speed or transition delay fault testing, but other methods were adopted in the industry. As such, Transfer was removed from the standard. It is no longer listed as a WSP terminal and was removed from Figure 12 of the WSP.

— There is a new permission for the wrapper serial output (WSO) terminal to change on the rising edge of wrapper clock (WRCK) and the WSP may now include an optional wrapper serial output rising (WSOR) terminal. The WSOR terminal changes on the rising edge of WRCK and may be included when the mandatory WSO changes on the falling edge of WRCK. Refer to Clause 14 for WSP timing specifications with the new permissions.

In Clause 12:

— Rule 12.2.1(b) was updated to include the state of SIBs, in the case where the wrapper boundary registry (WBR) is segmented using SIBs.

— In 12.4, clarifications were made to the apply event and user-defined events.

— In 12.7, the cell naming conventions were updated for removal of transfer. The naming convention now specifies the capture position into the shift path of the WBR cell.

— In 12.8, Transfer was removed from the WBR cell examples, and some of the examples were updated with new WBR cells that do not require Transfer.

— There is an updated delay test example in 12.9. This was updated based on the removal of Transfer, using conventional delay test methods.

In Clause 14:

— The rules and permissions in 14.2 are updated to support the WSO and WSOR revisions.

— New timing diagrams for the wrapper serial output falling and rising (WSO[R]) revisions were added in 14.3. Figure 43 was added to show the permission for WSO rising edge and Figure 44 was added to show the permission for WSOR.

— A new 14.5 was added to show an example of a methodology to allow for higher shift frequencies when the CaptureWR, ShiftWR, and UpdateWR signals are timed as multi-cycle paths.

In Clause 15:

— There are new examples for hierarchical subcores, shown in Figure 51 through Figure 54.

— There is a new rule 15.2.2(a) requiring concurrent CaptureWR, ShiftWR, and UpdateWR operation of cores, when selected together in a scan path.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

In Clause 16:

— Clause 16 was updated to clarify plug-and-play with respect to the WSO(R) revisions.

In Clause 17:

— Relaxed CTL compliance requirements (CTL required for hardware description, patterns may use other IEEE TTSC standards).

— Rules 17.2.1(a) and 17.2.1(b) were updated to specify only hardware-related test information be provided in CTL and test data and protocols specified in an IEEE TTSC (or successor) sponsored standard test data language.

— Rules 17.2.1(d) and 17.2.1(e) were updated to require specifying the maximum length of a scan chain, and the order of cells in a scan chain, when SIBs are present in the chain.

— A new permission 17.2.1(h) allows the use of other test data languages for test modes, test protocols, and test pattern data.

— In 17.4.1, rules are updated for use of other test data languages.

In Clause 20:

— Updated old CTL examples to comply with released IEEE Std 1450.6 (CTL). Added a new example showing minimal CTL usage.

In Annex A:

— The WBR bubble diagram definitions were updated in order to remove Transfer.

— Added new definitions for SIB bubble diagrams, and added a SIB example in Figure A.4.

In Annex B:

— Updates to the bubble diagram examples due to the removal of transfer.

In Annex C:

— Annex C was rewritten to include the relationship of IEEE Std 1500 to IEEE Std 1687™.

— The old example of interfacing an IEEE 1500 core to an IEEE 1149.1 TAP controller was replaced with a new example. The old example accessed the IEEE 1500 WIR together with the IR of the TAP controller.

— The new example integrates multiple cores within an SoC using IEEE Std 1149.1, IEEE Std 1500, and IEEE Std 1687. It describes a hierarchical serial access network that is accessed using the TAP controller and two TAMs. This new example acceses the IEEE 1500 WIRs using IEEE Std 1687 and the DR states of the TAP controller.

Two examples of hierarchically accessing the TAMs and cores from the IEEE 1149.1 TAP controller are given.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Contents

1. Overview 14
1.1 Background 14
1.2 Scope 15
1.3 Purpose 15
1.4 Word usage 15
1.5 Need 16
1.6 Access protocols 16

2. Normative references 16

3. Definitions, acronyms, and abbreviations 16
3.1 General 16
3.2 Definitions 17
3.3 Acronyms and abbreviations 22

4. Structure of this standard 24
4.1 Introduction 24
4.2 Specifications 24
4.3 Descriptions 25
4.4 Conventions 25

5. Introduction and motivations for two compliance levels 25

6. Overview of the IEEE 1500 scalable hardware architecture 26
6.1 Introduction 26
6.2 Wrapper serial port (WSP) 26
6.3 Wrapper parallel port (WPP) 27
6.4 Wrapper instruction register (WIR) 27
6.5 Wrapper bypass register (WBY) 28
6.6 Wrapper boundary register (WBR) 28

7. WIR instructions 28
7.1 Introduction 28
7.2 Response of the wrapper circuitry to instructions and data registers 29
7.3 Wrapper instruction rules and naming convention 31
7.4 WS_BYPASS instruction 33
7.5 WS_EXTEST instruction 34
7.6 WP_EXTEST instruction 36
7.7 Wx_EXTEST instruction 38
7.8 WS_SAFE instruction 39
7.9 WS_PRELOAD instruction 41
7.10 WP_PRELOAD instruction 43
7.11 WS_CLAMP instruction 44
7.12 WS_INTEST_RING instruction 46
7.13 WS_INTEST_SCAN instruction 48
7.14 Wx_INTEST instruction 50

8. Wrapper serial port (WSP) 52
8.1 Introduction 52
8.2 WSP terminals 52

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

9. Wrapper parallel port (WPP) 54
9.1 WPP terminals 54

10. Wrapper instruction register (WIR) 54
10.1 Introduction 54
10.2 WIR configuration and DR selection 55
10.3 WIR design 56
10.4 WIR operation 59

11. Wrapper bypass register (WBY) 61
11.1 WBY register configuration and selection 61
11.2 WBY design 62
11.3 WBY operation 63

12. Wrapper boundary register (WBR) 64
12.1 Introduction 64
12.2 WBR structure and operation 67
12.3 WBR cell structure and operation 68
12.4 WBR operation events 69
12.5 WBR operation modes 70
12.6 Parallel access to the WBR 72
12.7 WBR cell naming 75
12.8 WBR cell examples 77
12.9 IEEE 1500 WBR example 80

13. Wrapper states 90
13.1 Wrapper Disabled and Wrapper Enabled states 90

14. WSP timing diagrams 92
14.1 Introduction 92
14.2 Specifications 92
14.3 Description 93
14.4 Synchronous reset timing 99
14.5 Timing for fast pipelined shift 100

15. WSP configurations for IEEE 1500 system chips 102
15.1 Introduction 102
15.2 Connecting multiple WSPs 102

16. Plug-and-play (PnP) 107
16.1 Introduction 107
16.2 Background and definition 107
16.3 PnP aspects of standard instructions 109
16.4 PnP limitations on protocols 109
16.5 Non-PnP in IEEE Std 1500 109

17. Compliance definitions common to wrapped and unwrapped cores 110
17.1 Introduction 110
17.2 General rules 110
17.3 Per terminal rules 112
17.4 Test pattern information rules 114

18. Compliance definitions specific to unwrapped cores 117
18.1 General rules 117
18.2 Per terminal rules 118

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

18.3 Additional test information rules 119

19. Compliance definitions specific to wrapped cores 119
19.1 General rules 119
19.2 Per terminal rules 120
19.3 Wrapper protocol information rules 121

20. IEEE 1500 application 121
20.1 Introduction 121
20.2 CTL (IEEE 1450.6) overview 121
20.3 IEEE 1500 examples 123

Annex A (informative) Bubble diagram definition 152

Annex B (informative) Wrapper boundary register (WBR) cell examples 154

Annex C (informative) Relationship of IEEE Std 1500 to IEEE Std 1149.1 and IEEE Std 1687 162
C.1 Introduction 162
C.2 Example of integrating IEEE 1500 cores within an SoC 162

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

IEEE Standard Testability Method for Embedded Core-based Integrated Circuits

1. Overview

1.1 Background

IEEE Std 1500™ defines a scalable architecture for independent, modular test development and test application for embedded design blocks and also enables test of the external logic surrounding these cores. Modular testing is typically a requirement for embedded nonlogic blocks, such as memories, and for embedded pre-designed nonmergeable intellectual property (IP) cores. In addition, the IEEE 1500 architecture can also be used to partition large design blocks into smaller blocks of more manageable size and to facilitate test reuse for blocks that are reused from one system-on-chip (SoC) design to the next.

The IEEE 1500 architecture comprises hardware requirements, through the definition of a standardized core wrapper, and uses test-specific languages to communicate information between core providers and core users. The IEEE 1450.6™ core test language (CTL) is required for hardware-related test information pertinent to core integration. Test data and test protocols pertinent to core integration shall be provided in either CTL or another IEEE TTSC (or successor) sponsored standard test data language. Although IEEE Std 1500 limited itself to test aspects internal to nonmergeable cores, careful consideration was given to the interoperability of such cores, resulting in plug-and-play (PnP) requirement definitions. SoC-specific issues such as those related to the design of test access mechanisms (TAMs) are excluded from this standard and assumed to be addressed by the SoC designer.

IEEE Std 1500 specifically focuses on defining test requirements for unidirectional non-tristate digital terminals, as these represent a minimum and mandatory set of requirements upon which the more complex bidirectional terminals are based. It is, therefore, implied that support for bidirectional or tristatable terminals is provided only to the extent that the individual unidirectional terminals, i.e., the bidirectional or tristatable terminal, are available for IEEE 1500 wrapper insertion. In addition, the hardware architecture defined in this standard anticipates a synchronous wrapper design methodology.

While IEEE Std 1500 does not discuss chip-level design, the architecture defined in this standard does not prevent interfacing with other standards, namely standards based on IEEE Std 1149.1™ and IEEE Std 1687™. An example of this interface is provided in Annex C for the reader's benefit.

All rules described in this standard apply to the case where the IEEE 1500 wrapper is enabled (the wrapper logic actively participates in the test of the core) except rules specific to the Wrapper Disabled state of the IEEE 1500 wrapper. In Wrapper Disabled state, the IEEE 1500 wrapper is disabled, allowing functional

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

operation of the wrapped core. IEEE 1450.6 constructs were added to this standard, where appropriate, to further guide the reader. The reader is directed to refer to these CTL constructs documented in IEEE Std 1450.6. Additional discussion that complements the body of this standard are presented in annex clauses:

— Annex A contains the legend for IEEE 1500 wrapper cells.

— Annex B shows examples of IEEE 1500 wrapper cells.

Annex C describes the relationship between IEEE Std 1500, IEEE Std 1687, and IEEE Std 1149.1, and also provides an example SoC integration of several IEEE 1500 cores using these standards.

1.2 Scope

IEEE Std 1500 is a standard design-for-testability method for integrated circuits (ICs) containing embedded nonmergeable cores. This method is independent of the underlying functionality of the IC or its individual embedded cores. The method supports the necessary requirements for the test of such ICs, while allowing for ease of interoperability of cores that might have originated from different sources. This method is usable for all classes of digital cores, including hierarchical cores.

1.3 Purpose

The aim of IEEE Std 1500 is to provide a consistent scalable solution to the test reuse challenges specific to the reuse of nonmergeable cores, while preserving the IP aspects that are often associated with these cores. This objective is achieved through provision of a core-centric methodology that enables successful integration of cores into SoCs.

1.4 Word usage

The word *shall* indicates mandatory requirements strictly to be followed in order to conform to the standard and from which no deviation is permitted (*shall* equals *is required to*).[7, 8]

The word *should* indicates that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required (*should* equals *is recommended that*).

The word *may* is used to indicate a course of action permissible within the limits of the standard (*may* equals *is permitted to*).

The word *can* is used for statements of possibility and capability, whether material, physical, or causal (*can* equals *is able to*).

[7] The use of the word *must* is deprecated and cannot be used when stating mandatory requirements; *must* is used only to describe unavoidable situations.
[8] The use of *will* is deprecated and cannot be used when stating mandatory requirements; *will* is only used in statements of fact.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

1.5 Need

IEEE Std 1500 provides a bridge between core providers and core users and also facilitates the automation of test data transfer and reuse between these two entities via the use of the IEEE 1450.6 CTL. This automation relies on information requirements (the information model) placed on the core provider to help ensure that the core can be successfully integrated by the core user. The result is shorter time to market for core providers and core users.

The data transfer and reuse from the core provider to the core user are based on the premise that the core test data are left unchanged, while the test protocol is adapted from the IEEE 1500 hardware interface to the SoC.

1.6 Access protocols

Access protocols that reside above the IEEE 1500 core might need to be activated before accessing the core wrapper and core data registers. These protocols are outside the scope of this standard. An example of such protocols might be one that provides security access to the SoC.

2. Normative references

The following referenced documents are indispensable for the application of this document (i.e., they must be understood and used, so each referenced document is cited in text and its relationship to this document is explained). For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments or corrigenda) applies.

IEEE Std 1149.1™, IEEE Standard Test Access Port and Boundary-Scan Architecture.[9, 10]

IEEE Std 1450.6™, IEEE Standard Test Interface Language (STIL) for Digital Test Vector Data—Core Test Language (CTL).

IEEE Std 1687™, IEEE Standard for Access and Control of Instrumentation Embedded within a Semiconductor Device.

3. Definitions, acronyms, and abbreviations

3.1 General

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause.[11]

This clause lists some definitions of terms that have been used throughout this standard. In addition, a list of acronyms and abbreviations is also provided. The criteria for differentiating various terms are based on the following rules:

[9] The IEEE standards or products referred to in Clause 2 are trademarks owned by The Institute of Electrical and Electronics Engineers, Incorporated.
[10] IEEE publications are available from The Institute of Electrical and Electronics Engineers (https://standards.ieee.org/).
[11] *IEEE Standards Dictionary Online* is available at: http://dictionary.ieee.org. An IEEE Account is required for access to the dictionary, and one can be created at no charge on the dictionary sign-in page.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

a) General terminology in the scope of electrical engineering and/or test technology that do not influence the definition of this standard itself. These terms are used in this standard without further explanation. It is assumed that the readership has sufficient background knowledge to understand these terms.

b) Terms that are specific to this standard.

   1) These terms are defined in this clause.

   2) General definitions are valid throughout this standard.

   3) Local definitions are relevant only to a specific clause of this standard.

Effort has been made to achieve consistent usage of all terms (e.g., wrapper, TAM).

3.2 Definitions

access mechanisms: Mechanisms by which signals are propagated to and from a core, from either embedded circuitry or from the primary inputs and outputs of the system chip. There are two types of access mechanisms:

a) Functional access: The mechanism for moving stimuli to and observing responses from a core or user-defined logic (UDL) during functional operation or normal mode.

b) Test access: The mechanism for moving stimuli to and observing responses from a core or UDL during nonfunctional operation or test mode.

auxiliary clock (AUXCK): A functional clock that can be used in conjunction with wrapper clock (WRCK) during core test for capturing, shifting, and updating test data in a wrapper.

bypass: As applied to core wrappers, an abbreviated sequential path connecting a wrapper serial input (WSI) to a wrapper serial output falling and rising (WSO[R]).

CaptureWR: A wrapper terminal used to enable and control a Capture operation in the selected IEEE 1500 wrapper register (WR).

cell functional input (CFI): For input wrapper cells, the cell's input, which is connected to a wrapper functional input (WFI); for output wrapper cells, the cell's input, which is connected to a core output.

NOTE—See CFI pin in Figure 17.

cell functional output (CFO): For input wrapper cells, the cell's output, which is connected to a core input; for output wrapper cells, the cell's output, which is connected to a wrapper functional output (WFO).

NOTE—See CFO pin in Figure 17.

cell test input (CTI): A wrapper boundary register (WBR) cell's test data input.

cell test output (CTO): A wrapper boundary register (WBR) cell's test data output.

core: Predesigned circuit block that can be tested as an individual unit.

core data register (CDR): Optional data register that belongs to a core being wrapped.

core input: An input terminal of an unwrapped core.

core integrator: An entity that incorporates one or more cores into a system on chip (SoC).

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

core isolation: A test mode feature preventing core-to-core or core-to-UDL (i.e., user-defined logic) interaction.

core output: An output terminal of an unwrapped core.

core provider: An entity that designs cores that can be reused in other designs.

core test: A test methodology that is applied to an embedded core.

core test language (CTL): A standard language for core suppliers to provide test data that can be used to test a core once it is integrated into a system on chip (SoC). The language presents a format to describe test and support data so that the core can be effectively integrated, reused, and tested.

NOTE—See IEEE Std 1450.6 reference documentation.

dedicated shift path: A shift path comprising storage elements that do not participate in functional operation.

dedicated wrapper (cell): A wrapper style that does not share hardware with core functionality. This style allows certain test operations to occur concurrently and transparently during functional operation. This definition could apply to individual cells.

event: An uninterrupted, predefined sequence of one or more steps that apply to a data register, e.g., wrapper boundary registry (WBR), as a whole. Within the interval of the predefined sequence there can be a particular instant that characterizes the event when the nominal action of the event occurs. This instant, if it exists for a particular event, is referred to as the *characteristic instant*. Predefined events are Shift, Update, Capture, and Apply.

external safe state: A configuration of safe state in which the outputs of a core are in a state that prevents them from interfering with a block of logic outside the core. *See also:* **internal safe state**; **safe state**.

firm core: A predesigned block of functional logic, such as a macro, megacell, or memory, that has a process technology-dependent netlist representation and so is amenable to some modification.

hard core: A predesigned block of functional logic, such as a macro, megacell, or memory, that has a physical implementation that cannot be modified.

hybrid instruction: A wrapper instruction that has mixed use of wrapper serial port (WSP) and wrapper parallel port (WPP) terminals.

input cell: A wrapper boundary register (WBR) cell that is provided on a core input.

internal safe state: A configuration of safe state whereby a core is protected from the impact of a test outside the core. *See also:* **external safe state**; **safe state**.

interoperability: *See:* **plug-and-play (PnP)**.

inward facing (IF) mode: The test mode where core inputs are controlled by the wrapper boundary register (WBR) and core outputs are observed by the WBR.

mergeable core: With respect to testability, a core that can be integrated with other cores and user-defined logic (UDL) into a system on chip (SoC) so that a uniform design-for-test (DFT) methodology can be applied across the entire system. A typical mergeable core is provided using a register transfer level (RTL) or gate-level description.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

merged core: With respect to testability, a core that is integrated with other cores and user-defined logic (UDL) into a system on chip (SoC) so that a uniform design-for-test (DFT) methodology could be applied across the entire system.

nonmergeable core: With respect to testability, a core that cannot be integrated to apply a uniform design-for-test (DFT) methodology to the entire system on chip (SoC). A typical non-mergeable core comes with a physical design implementation that does not accommodate modification of the test methodology. A nonmergeable core might be represented as a black-box design, making standard automatic test pattern generation (ATPG) impossible on such a core.

nonmerged core: With respect to testability, a core that has not been integrated with other cores and user-defined logic (UDL) into a system on chip (SoC) so that a uniform design-for-test (DFT) methodology could be applied across the entire system.

normal mode: The mode in which the wrapper boundary register (WBR) does not interfere with the functional operation of a wrapped core.

output cell: A wrapper boundary register (WBR) cell that is provided for a core output.

outward facing (OF) mode: The test mode where wrapper functional outputs (WFOs) are controlled by the wrapper boundary register (WBR) and wrapper functional inputs (WFIs) are observed by the WBR.

parallel instruction: A wrapper instruction that uses wrapper parallel port (WPP) terminals and also configures the wrapper bypass register (WBY) between wrapper serial input (WSI) and wrapper serial output falling and rising (WSO[R]).

pattern set: A collection of test vectors intended for manufacturing test. In the context of core test language (CTL), a pattern set is a collection of pattern constructs and their associated macros and procedures brought together with PatternBurst and PatternExecs.

plug-and-play (PnP): A minimum level of required interoperability between various core wrappers in a system on chip (SoC). See Clause 16 for detailed requirements.

safe data: Data that satisfy safe state configuration requirements. These data are user-defined.

safe state: A property whereby a test of one block of logic is prevented from interfering with or damaging another block of logic. *See also:* **external safe state**; **internal safe state**.

segment: A set of contiguous (in terms of the scan path) data register bits obeying the rules for wrapper data registers and from which one or more data registers can be assembled.

segment insertion bit (SIB): A scan cell with at least shift and update storage elements and a scan-multiplexor that is used to include or exclude a data register segment from the active scan path. SIBs can be in-line with the data register segments, or on a separate path in order to remotely control the segments.

SelectWIR: The IEEE 1500 wrapper terminal that determines the selection of a wrapper register (WR). A value of 1 represents selection of the wrapper instruction register (WIR), and a value of 0 represents selection of a wrapper data register (WDR).

serial instruction: A wrapper instruction that exclusively uses wrapper serial port (WSP) terminals.

serial scan chain: The scan chain configuration inside a wrapped core where an internal scan chain is concatenated with the wrapper boundary register (WBR) chain for the purpose of running the WS_INTEST_SCAN instruction.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

shared wrapper (cell): The wrapper style that shares logic between the test and functional modes of operation. Shared cells typically include registered core inputs and outputs that can be used in test mode to control and observe core test data. This style prevents simultaneous functional and test operation uses of the shared register.

ShiftWR: The wrapper terminal used to enable and control a Shift operation in the selected IEEE 1500 wrapper register (WR).

silent shift path: A wrapper boundary register (WBR) shift path comprising a dedicated shift path and implemented to support a Shift operation that keeps wrapper functional output (WFO) terminals static.

soft core: A predesigned block of functional logic such as a macro, megacell, or memory with a register transfer level (RTL) representation. Soft cores are inherently process technology independent.

standard test interface language (STIL): The language in IEEE Std 1450™ for representing digital test vector data. The core test language (CTL) is an extension of STIL to support a standard way of representing test architecture and optionally the test data for a core.

system on chip (SoC): An entire system integrated on a single chip. It can include one or more cores with user-defined logic (UDL) integrated by the core user or system integrator.

terminal: A connection point from the core or wrapper to the logic outside of the core or wrapper.

test access mechanism (TAM): A feature of a system-on-chip (SoC) design that enables the delivery of test data to and from cores or core wrappers.

test access mechanism (TAM) harness: Wrapper boundary register (WBR) logic that enables the coupling of a TAM to cell test inputs (CTIs) and cell test outputs (CTOs).

test input (TI): The serial test data input of a wrapper boundary register (WBR).

NOTE—See TI in Figure 17.

test mode: A state whereby a block of logic is made ready for test.

test output (TO): The serial test data output of a wrapper boundary register (WBR).

NOTE—See TO in Figure 17.

test protocol: A sequence of control operations required for a test. A test protocol comprises functions and/or sequences. Functions can consist of other functions and/or sequences, while sequences comprise a series of logic 0 and 1 values applied to specified terminals. At the lowest level, a test protocol is just a series of logic 0 and 1 applied to specified test control terminals. The protocol would typically also contain symbolic references to the test data that are to be applied to or observed at a specified test data or system data port. For example, a scan protocol might involve the repetition of the following operations:

a) Apply a logic level to assert an internal scan chain SCAN_ENABLE control signal.

b) Apply a sequence of *n* clock pulses to a clock port while applying data to the SCAN_DATA_IN of the scan chain(s). Observe data at the SCAN_DATA_OUT of the scan chain(s) as the data are clocked through the scan chains.

c) De-assert SCAN_ENABLE and clock the scan chains one or more times while controlling primary inputs and observing primary outputs.

d) Repeat steps a), b), and c) until the application of the scan chain patterns is complete.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

test reuse: The ability to apply a predetermined test pattern associated with a core after this core has been integrated into a system on chip (SoC). Test reuse is a consequence of design reuse and often requires the adaptation of a test protocol to reflect the core's new environment within an SoC.

Test Technology Standards Committee (TTSC): Organization within IEEE that promotes standards in the test industry.

update register: The register used to prevent the outputs of a shift register from propagating to other circuitry during Shift operation. After shifting is complete, the content of the associated shift register is parallel-loaded (updated) into the update register.

UpdateWR: The wrapper terminal used to enable and control an Update operation in the selected IEEE 1500 wrapper register (WR).

user-defined logic (UDL): Logic added by the system chip integrator (i.e., not a reused design), as interface circuitry or part of the feature set that differentiates the system-on-chip (SoC) product.

wrapper: Circuitry added around an embedded core to facilitate test reuse and to interface between a test access mechanism (TAM) and the embedded core.

wrapper boundary register (WBR): The portion of a wrapper comprising wrapper cells. *See:* **wrapper cell**.

wrapper bypass register (WBY): The IEEE 1500 register providing an abbreviated data register between wrapper serial input (WSI) and wrapper serial output (WSO[R]).

wrapper cell: The wrapper element associated with at least one core terminal.

wrapper clock (WRCK): The IEEE 1500 clock.

wrapper configuration: An arrangement of interconnected wrappers in a system on chip (SoC).

wrapper data register (WDR): The IEEE 1500 register (e.g., wrapper boundary register [WBR], wrapper bypass register [WBY]) used to perform IEEE 1500 operations.

wrapper functional input (WFI): The wrapper input terminal corresponding to the functional core input of a wrapped core.

wrapper functional output (WFO): The wrapper output terminal corresponding to the functional core output of a wrapped core.

wrapper input: An input terminal to a wrapped core. In the case where there is a shared wrapper cell, the term *wrapper input* takes precedence over the term *core input*.

wrapper instruction register (WIR): The IEEE 1500 register used to serially load and store IEEE 1500 instructions.

wrapper output: An output terminal to a wrapped core. In the case where there is a shared wrapper cell, the term *wrapper output* takes precedence over the term *core output*.

wrapper parallel port (WPP): A user-defined set of wrapper terminals corresponding to a test access mechanism (TAM) and providing parallel access to the wrapper boundary register (WBR) and/or an embedded core.

wrapper register (WR): A wrapper instruction register (WIR) or wrapper data register (WDR).

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

wrapper reset (WRSTN): The IEEE 1500 active low reset.

wrapper serial input (WSI): The wrapper serial data input.

wrapper serial output (WSO): The wrapper serial data output.

wrapper serial output rising (WSOR): An optional second wrapper serial data output. When implemented, the WSOR terminal outputs the same value as WSO, but changing on the rising edge of WRCK rather than the falling edge of WRCK.

wrapper serial output falling and rising (WSO[R]): WSO(R) is used in this document when it applies to both the WSO and optional WSOR terminals.

wrapper serial port (WSP): Standard wrapper terminals providing serial access to the IEEE 1500 wrapper. The WSP comprises the following signals: wrapper reset (WRSTN), wrapper clock (WRCK), SelectWIR, ShiftWR, CaptureWR, UpdateWR, wrapper serial input (WSI), and wrapper serial output (WSO[R]). Additionally, user-defined auxiliary clock(s) (AUXCKs) can be part of the WSP.

Wx_: Shorthand nomenclature to refer to multiple instructions that instructions that have the same suffix. For example, Wx_PRELOAD refers to WS_PRELOAD, WP_PRELOAD, etc.

3.3 Acronyms and abbreviations

ASIC	application-specific integrated circuit
ATPG	automatic test pattern generation
AUXCK	auxiliary clock
CDR	core data register
CFI	cell functional input
CFO	cell functional output
CSU	CaptureWR, ShiftWR, UpdateWR
CTI	cell test input
CTL	core test language
CTO	cell test output
DFT	design-for-test
DR	data register
EDA	electronic design automation
FPGA	field-programmable gate array
IC	integrated circuit
IF	inward facing

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

I/O	input/output
IP	intellectual property
IR	instruction register
OF	outward facing
PnP	plug-and-play
RF	radio frequency
RTL	register transfer level
SIB	segment insertion bit
SoC	system on chip
STIL	standard test interface language
TAM	test access mechanism
TAP	test access port
TCK	IEEE 1149.1 test clock
TI	test input
TO	test output
TTSC	Test Technology Standards Committee
UDL	user-defined logic
WBR	wrapper boundary register
WBY	wrapper bypass register
WDR	wrapper data register
WFI	wrapper functional input
WFO	wrapper functional output
WIR	wrapper instruction register
WPC	wrapper parallel control
WPI	wrapper parallel input
WPO	wrapper parallel output
WPP	wrapper parallel port
WR	wrapper register

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

WRCK	wrapper clock
WRSTN	wrapper reset
w.r.t	with respect to
WSC	wrapper serial control
WSI	wrapper serial input
WSO	wrapper serial output
WSOR	wrapper serial output rising
WSO(R)	wrapper serial output falling and/or rising
WSP	wrapper serial port

4. Structure of this standard

4.1 Introduction

The three mandatory hardware blocks along with the standard instructions and the mandatory serial port are described in separate clauses of this standard:

- Standard instructions: Clause 7
- Wrapper serial port (WSP): Clause 8 and Clause 14
- Wrapper instruction register (WIR): Clause 10
- Wrapper bypass register (WBY): Clause 11
- Wrapper boundary register (WBR): Clause 12

The above hardware clauses are complemented with information model and compliance discussion split across the following clauses:

- Compliance definitions common to wrapped and unwrapped cores: Clause 17
- Compliance definitions specific to unwrapped cores: Clause 18
- Compliance definitions specific to wrapped cores: Clause 19

4.2 Specifications

Subclauses entitled “Specifications” contain the rules, recommendations, and permissions that define this standard:

a) *Rules* specify the mandatory aspects of this standard. Subclauses that are rules contain the word *shall*.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

b) *Recommendations* indicate preferred practice for designs that seek to conform to this standard. Subclauses that are recommendations contain the word *should*.

c) *Permissions* show how optional features may be introduced into a design that seeks to conform to this standard. These features extend the application of the test circuitry defined by this standard. Subclauses that are permissions contain the word *may*.

It is assumed throughout this standard that rules take precedence over recommendations and recommendations take precedence over permissions.

4.3 Descriptions

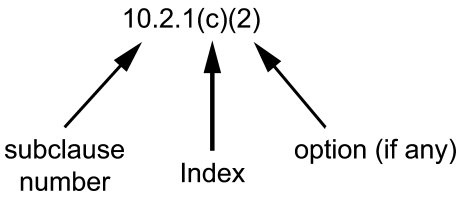
Material not contained in subclauses entitled "Specification" is descriptive material that illustrates the need for the features being specified or their application. This material includes schematics that illustrate a possible implementation of the specifications in this standard. Annex clauses to this standard contain alternative implementation examples. The descriptive material also discusses rationale for inclusion of certain features.

The descriptive material contained in this standard is for illustrative purposes only and does not define a preferred implementation. Most examples provided are intentionally somewhat ambiguous to not display bias toward a particular implementation style. Where discrepancies between examples and specifications occur, the specifications take precedence. Readers should exercise caution when using these examples to facilitate full compliance in their specific applications. In particular, it is emphasized that the examples are designed to effectively communicate the meaning of this standard. As always, a particular implementation might not operate properly with respect to timing and other parametric characteristics.

4.4 Conventions

The following structural conventions are used in this standard:

— The rules, recommendations, and permissions in each specifications subclause are contained in a single alphabetically indexed list. References to each rule, recommendation, or permission are shown in the form:



— When the reference and the referring text belong to the same subclause, only the index is indicated (i.e., the subclause number is omitted).

5. Introduction and motivations for two compliance levels

IEEE Std 1500 recognizes the heterogeneous aspect of the embedded core market, i.e., a diversity driven by the need to cover a variety of design functions implemented in digital logic, analog logic, memory, radio frequency (RF), field-programmable gate arrays (FPGAs), or combinations of the above. Cores come in many different "flavors" (e.g., hard, firm, soft) and are being used and/or sold within companies as well as between companies. Within its focus on nonmerged digital logic and memory cores, IEEE Std 1500 intends

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

to support the above diversity and associated business models. This requirement poses a need for flexibility that was translated into the definition of the following two levels of compliance to this standard:

— **IEEE 1500 unwrapped compliance:** This compliance level refers to a core that does not have a (complete) IEEE 1500 wrapper, but does have a IEEE 1450.6 CTL description on the basis of which the core could be made IEEE 1500 wrapped compliant, either manually or by using dedicated tools. The CTL test data describes the core test knowledge at the bare core terminals.

— **IEEE 1500 wrapped compliance:** This compliance level refers to a core that incorporates an IEEE 1500 wrapper function and comes with necessary test data described using a combination of IEEE 1450.6 CTL and other IEEE Test Technology Standards Committee (TTSC) (or successor) sponsored standard test data language. All hardware-related test information pertinent to core integration, which includes describing the wrapper and the wrapper's external terminals, shall be described in CTL, while test pattern data and test protocols pertinent to core integration can be described in CTL or another IEEE TTSC (or successor) sponsored standard test data language.

The motivation behind the two different levels of compliance is a need to match the core-provider/core-user business model and provide the flexibility that is required in testing core-based system chips. The ultimate compliance goal should be to make a core compliant to the IEEE 1500 wrapped level. Nevertheless, the two levels of compliance provide the option to become IEEE 1500 wrapped directly or via the intermediate step of being IEEE 1500 unwrapped.

The first alternative, i.e., direct generation of an IEEE 1500 wrapped core, provides the possibility to integrate the wrapper functionality with the core itself and hence reduce the performance and area impact of the wrapper. The second alternative is to first create an IEEE 1500 unwrapped core, which is then, in a separate step, turned into an IEEE 1500 wrapped core. This allows the SoC integrator to take advantage of the scalability of the standardized wrapper and instantiate the wrapper with particular parameter values, which takes into account certain aspects of the system chip environment in which this particular core version is used.

6. Overview of the IEEE 1500 scalable hardware architecture

6.1 Introduction

The IEEE 1500 core wrapper comprises the following:

— Serial interface terminals forming the WSP

— An optional user-defined set of wrapper terminals forming the wrapper parallel port (WPP) and providing parallel access to the wrapper

— A WIR

— A WBY

— A WBR

Figure 1 illustrates the standard components of the IEEE 1500 wrapper.

6.2 Wrapper serial port (WSP)

The WSP terminals serve as the primary interface to the IEEE 1500 wrapper. This set of serial terminals could be sourced from chip-level pins, an embedded controller such as an IEEE 1149.1™-based controller,

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

or a serial TAM as described in Annex C. The WSP is used to load and unload instructions and data into and out of the IEEE 1500 registers. In addition to the wrapper serial input (WSI) and wrapper serial output (WSO and optional WSOR) terminals shown in Figure 1, the WSP contains wrapper serial control (WSC) terminals used to control the operation of all IEEE 1500 registers.

The WSP is described with further details in Clause 8.

6.3 Wrapper parallel port (WPP)

The WPP is a user-defined set of wrapper terminals providing a parallel interface to the IEEE 1500 wrapper. These terminals are used when the wrapper is configured into parallel mode. The WPP terminals consist of the wrapper parallel input (WPI) terminal(s), wrapper parallel output (WPO) terminal(s), and wrapper parallel control (WPC) terminals. The set of WPC terminals may include elements of the set of WSC terminals (i.e., wrapper clock [WRCK], auxiliary clock [AUXCK], wrapper reset [WRSTN]). The WPP is described with further details in Clause 9.

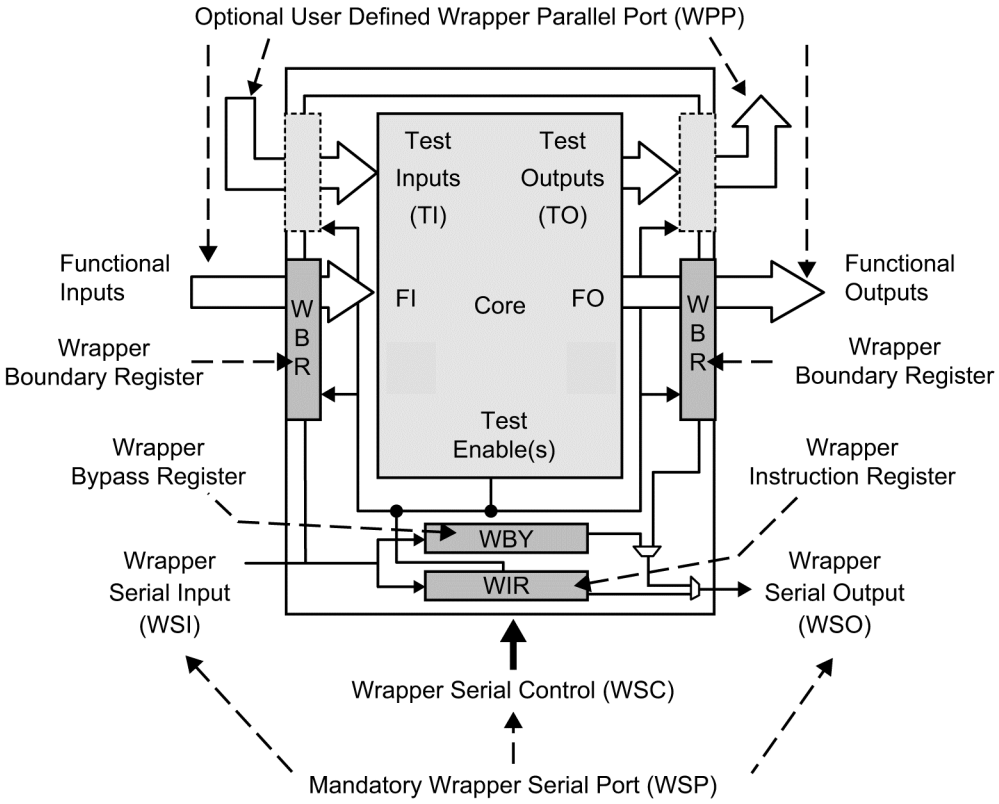


Figure 1 —Standard IEEE 1500 wrapper components

6.4 Wrapper instruction register (WIR)

The WIR enables all IEEE 1500 wrapper operations. This register is loaded via the WSP with instructions that select an IEEE 1500 data scan path register. The WIR can optionally be interfaced to the core for establishing test mode or functional operation. The WIR is described with further details in Clause 10.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

6.5 Wrapper bypass register (WBY)

The WBY provides a bypass path for the WSI-WSO(R) terminals of the WSP. The WBY is the default data register between WSI and WSO(R) and should be selected by the current wrapper instruction when no other data register is selected. The WBY is intended to provide a minimum length scan path through the wrapper, so that when several IEEE 1500 wrappers are serially chained together in a system on chip (SoC), the wrappers that do not require a data register to be accessed can be bypassed with a short scan path through their WSI-WSO(R) terminals. The WBY is described with further details in Clause 11.

6.6 Wrapper boundary register (WBR)

The WBR is the data register through which test data stimuli are applied and pattern responses are captured. This register allows internal testing of the core, as well as testing of external connectivity to other cores and SoC integration circuitry, in response to an instruction loaded into the WIR. The WBR is described with further details in Clause 12.

7. Wrapper instructions

7.1 Introduction

The WIR allows instructions to be serially entered into the wrapper circuitry via the WSI and the WSC. This clause defines the minimum set of instructions that shall be supplied and the operations that occur in response to those instructions. Optional instructions and the resulting operation of the wrapper circuitry are also defined, together with the requirements for extensions to the instruction set defined in this standard. The tests applied using IEEE 1500 instructions can be specified with CTL or a combination of CTL and another IEEE TTSC (or successor) sponsored standard test data language. The CTL identifies attributes of the wrapped core such as the following:

— Location and number of core wrapper input and output terminals interfacing with external circuitry

— Protocol used to input and output the wrapper scan data

— Condition or state of the core input and output terminals during the external circuitry test, excluding the test input (TI) and test output (TO) terminals and the terminals interfacing with the external circuitry

— Configuration of the WBR

The IEEE 1500 wrapper has various modes of operation. There are modes for functional (nontest) operation, inward facing (IF) test operation, and outward facing (OF) test operation. Different test modes also determine whether the serial test data mechanism (WSI–WSO[R]) or the parallel test data mechanism (WPI–WPO), if present, is being utilized.

Instructions loaded into the WIR, together with the IEEE 1500 wrapper signals, determine the mode of operation of the wrapper and possibly the core itself. In addition, the selected data register path can be determined by the contents and operation of the data register itself, or other data registers. There is a minimum set of instructions and corresponding operations that shall be supplied. Optional instructions and their corresponding behavior are also defined, together with the requirements for extension of the instruction set. All instructions that establish test modes that utilize the parallel port WPI and WPO are optional, as the presence of this port is optional. Furthermore, IEEE Std 1500 also allows for user-defined instructions.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

IEEE Std 1500 has a set of instructions that are defined to use only the serial interface (WSP) and a corresponding set of instructions that are defined for the parallel interface. IEEE Std 1500 shall allow accessibility to test the core. There is one main core test instruction—Wx_INTEST (user-specified core-test instruction)—that is flexible enough to allow any core test to execute. IEEE Std 1500 does not define the test for the core, and this very flexible instruction was specified so that it could be defined differently for each core test. There are two other instructions that are mandatory: an instruction for functional mode (WS_BYPASS) and an instruction for external test mode (WS_EXTEST). WS_BYPASS puts the wrapper into the bypass configuration and allows access to all functional terminals of the core. WS_EXTEST is the serial EXTEST configuration of the wrapper. Even if there is a WP_EXTEST mode (for parallel access), there should still be a WS_EXTEST instruction capability.

7.2 Response of the wrapper circuitry to instructions and data registers

7.2.1 Specifications

Rules

a) Each instruction shall select the wrapper register(s) (WRs) that operate while the instruction is active.

b) WRs and WR segments that are not selected by the active instruction shall be controlled so that operation of selected WRs, selected WR segments, and operation of the core circuitry can continue unhindered.

c) Each instruction shall cause a WR path to be selected to shift data between WSI and WSO(R).

d) During parallel and serial instructions, the proper WSC signals shall be used to shift data between WSI and WSO(R).

e) The data register path selected between WSI and WSO shall remain constant for the duration of the Shift operation.

f) If a data register path configuration is controlled by an inline segment insertion bit (SIB); the update of the segment insertion select signal shall not change until at least one-half clock cycle after the data register Update.

g) The WBY shall be selected to shift data between WSI and WSO(R) while parallel instructions are active.

h) Data that are loaded per an instruction into the WBR and to be used by a subsequent instruction shall be held until the subsequent instruction becomes active.

Permissions

i) The WR path between WSI and WSO(R) may comprise one or more serially connected wrapper data registers (WDRs), core data registers (CDRs), or data register segments.

j) As the result of a data register Update operation, data register path configurations may change.

k) A user-defined wrapper instruction may enable an alternate mechanism for access to WDRs and/or CDRs.

7.2.2 Description

The instructions loaded into the WIR are interpreted in order to achieve two key functions: configure the WDRs and control the mode of the core. The active instruction identifies the wrapper or CDRs that operate

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

while the instruction is active. Several wrappers or CDRs can be set into test modes simultaneously. Also, an instruction identifies the WR path that is used to shift data between WSI and WSO(R) during scan operation. The selected data register paths can also be controlled by data registers, e.g., SIBs. A particular instruction, or data register contents, can result in one or more wrapper or CDRs being serially connected between WSI and WSO(R) or WPI and WPO. Further, the active instruction can select one or more other registers, separate from the register(s) between WSI and WSO(R) or WPI and WPO, to perform other test functions. An unselected wrapper or CDR should be controlled so that they do not interfere with the operation of the core circuitry or with the operation of a WR path. Rule 7.2.1(h) allows for interdependent instructions such as WS_PRELOAD and WS_CLAMP to properly establish and use setup data.

During all instructions, there shall always be at least one register between WSI and WSO. During parallel and serial instructions, the WSC shall control the shift path between WSI and WSO(R). Hybrid instructions do not require that the WSC be used to control the shift path between WSI and WSO(R).

Permission 7.2.1(i) and 7.2.1(j) allows data registers (i.e., WDRs and CDRs) to be segmented and/or accessed hierarchically. This may be done by adding SIBs or other scan control bits that, when updated, cause the selected data register path configuration to change. Note that in order to access such data register segments, a sequence of multiple instructions, multiple shift, and update operations might be required.

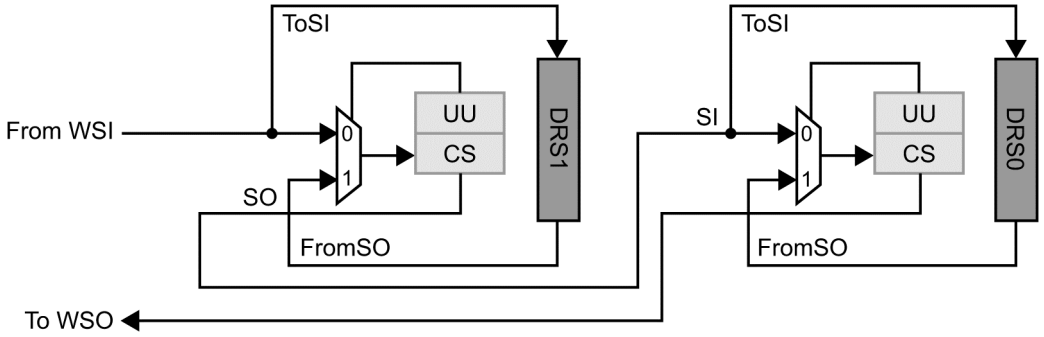


Figure 2 —DR segments accessed by inline SIBs

Figure 2 shows an example of a segmented data register that is accessed using two inline SIBs. Each of the two data register segments, DRS1 and DRS0, are either selected to be in the WR path, or bypassed through their respective SIB, depending on the Update value of the SIB.

Rule 7.2.1(e) requires that the selected data register path remain constant during the Shift operation, the path configuration need not change until the Update operation, per permission 7.2.1(j). Rule 7.2.1(f) requires that when using inline SIBs, the segment select signal shall not change until at least one-half clock cycle after the data register Update. This is required to prevent the inline SIB updating the select too early and causing the controlled data register segment to deselect before it updates its data register. Note that the two SIBs are in the data register scan path, even when both data register segments are unselected. See Figure 53 for an example of an inline SIB example. Also reference IEEE Std 1687 for other aspects and examples of SIB designs.

In Figure 2, CS is a single storage element for the shift and capture events, and UU is two storage elements, for the update event and the delay of the segment select. Figure 53 also uses a second update storage element to delay the data register path select. The second update storage element can be clocked with the subsequent rising or falling edge of the WRCK or AUXCK clock, following the data register update from the assertion of the WSP UpdateWR signal. It is also possible to satisfy rule 7.2.1(f) by adding at least one-half clock cycle delay instead of a second storage element.

The standard instructions are defined in Table 1. Only the WSP is utilized during the serial instructions.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Table 1—Instruction list

Instruction	Mandate	Description
WS_BYPASS	Required	Allows normal (functional) mode and puts the wrapper into bypass mode.
WS_EXTEST	Required	Allows external test using a single chain configuration in the WBR.
WP_EXTEST	Optional	Allows external test using a multiple scan chain configuration in the WBR.
Wx_EXTEST	Optional	A user-specified external test instruction.
WS_SAFE	Optional	Puts the core into a quiet mode and outputs a predefined static (safe) state from all output ports. It also puts the WBR into bypass mode.
WS_CLAMP	Optional	Outputs a programmable static (safe) state from all output ports. It also puts the wrapper into bypass mode. This is presumed to be preceded by either a Wx_PRELOAD instruction or a Wx_EXTEST instruction.
WS_PRELOAD	Conditionally required[a]	Loads data into the single silent shift path of the WBR.
WP_PRELOAD	Optional	Loads data into the multiple silent shift paths of the WBR.
WS_INTEST_RING	Optional[b]	Allows internal testing using a single chain configuration in the WBR.
WS_INTEST_SCAN	Optional[b]	Allows internal testing by concatenation of the wrapper chain with a single internal chain.
Wx_INTEST	Required	A user-specified core test instruction.

[a] WS_PRELOAD is required only if there is a silent shift path in the WBR.

[b] This instruction is also considered a Wx_INTEST instruction, just more completely (specifically) defined. This instruction can be used as the required Wx_INTEST instruction.

7.3 Wrapper instruction rules and naming convention

7.3.1 Introduction

There is a naming convention for the instructions.

W<Parallel/Serial/Hybrid>_<Mode>_<Configuration> (e.g., WS_INTEST_SCAN)

— **W:** Prefaces all standard IEEE 1500 instructions.

— **Parallel/Serial/Hybrid:** An S denotes a serial mode instruction. A P denotes a parallel mode instruction. An H denotes a hybrid instruction in which both the serial and parallel data ports are utilized. Note that a standard parallel instruction (e.g., WP_EXTEST) shall have the WBY between WSI and WSO(R) but remain distinct from a hybrid instruction. An instruction by which any register other than WBY is configured between WSI and WSO(R) and that also uses the parallel port is classified as a hybrid instruction, except for the WP_EXTEXT instruction [see permission 7.6.2(j)].

— **Mode:** A shortened description of the instruction mode, such as BYPASS, PRELOAD, etc.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

— **Configuration:** A shortened description of the configuration selected by a particular instruction. For instance, during the serial instructions WS_INTEST_SCAN and WS_INTEST_RING, SCAN denotes that internal scan chains are included in the single scan chain between WSI and WSO(R). RING indicates that only the wrapper chain is between the WSI and WSO(R).

7.3.2 Specifications

Rules

a) All serial instructions (e.g., WS_INTEST) shall use the WSP solely, except for WP_EXTEST [see permission 7.6.2(j)].

b) All parallel instructions (e.g., WP_EXTEST) shall use the WPP solely.

c) At least one instruction with mode INTEST shall be provided.

d) Each instruction name in Table 1 shall be used for the standard instruction associated with it.

e) No user instruction shall utilize the standard instruction names.

f) When Wx is stated in this standard, the x shall be replaced with P, S, or H.

g) The WS_EXTEST instruction shall be provided.

h) The WS_BYPASS instruction shall be provided and shall be used during any operation where the wrapper is disabled.

i) All standard instructions shall have a unique opcode.

j) Standard instruction names defined in this standard, such as WP_EXTEST, shall not be reused as names for instructions with functionality other than the functionality described in this standard. The standard instruction names are reserved.

Recommendations

k) User and private instructions should follow the same naming convention as the standard instructions.

Permissions

l) User-defined instructions may share opcodes with standard instructions.

m) Standard instruction names may be used as a prefix for other instructions defined by the user, such as WP_EXTEST_**BIST**.

7.3.3 Description

Rules 7.3.2(a) and 7.3.2(b) are meant to specify usage of the WSP and WPP for serial and parallel instructions. Additional instruction-specific description is provided in 7.4 through 7.14.

Where permission 7.3.2(l) is exercised, the corresponding user-defined instruction is expected to be identical or a superset of the standard instruction that has the same opcode as this user-defined instruction.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

7.4 WS_BYPASS instruction

7.4.1 Introduction

The mandatory WS_BYPASS instruction enables the functional configuration of the wrapper. WS_BYPASS is selected when no test operation of that core is required and allows only the WBY to be selected. The WBY provides a minimum-length serial path between the wrapper's WSI and the WSO(R). This allows more rapid movement of test data to and from other core wrappers, provided the wrappers are connected serially.

7.4.2 Specifications

Rules

a) Each wrapper shall provide a WS_BYPASS instruction.

b) The WS_BYPASS instruction shall select the WBY to be connected for serial access between the WSI and WSO(R) of the wrapper.

c) While the WS_BYPASS instruction is active, the operation of the wrapper circuitry shall have no effect on the operation of the core circuitry.

d) While the WS_BYPASS instruction is active, all wrapper boundary cells that can operate in either system or test modes shall perform their system function.

e) While the WS_BYPASS instruction is active, the WSC shall be utilized for test control.

Permissions

f) The binary code for the WS_BYPASS instruction may be selected by the wrapper designer.

7.4.3 Description

During testing of a particular core or cluster of cores on an SoC, it might be inconvenient to drive data through the entire length of a WBR. Instead, the WBR is replaced by the WBY. This allows data to transfer through the wrappers of the cores that are not being tested, more directly to and from the wrappers of the cores that are being tested, provided these wrappers are serially connected. The bypassed cores will have the ability to continue to run in system mode. The WS_BYPASS instruction could be used for this purpose. This instruction is the active instruction after wrapper reset (WRSTN) is asserted. Figure 3 shows an example of the configuration of the wrapper during the WS_BYPASS instruction. In the figure, the WSP_path goes from WSI, through Bypass to WSO, and is controlled by WSC.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

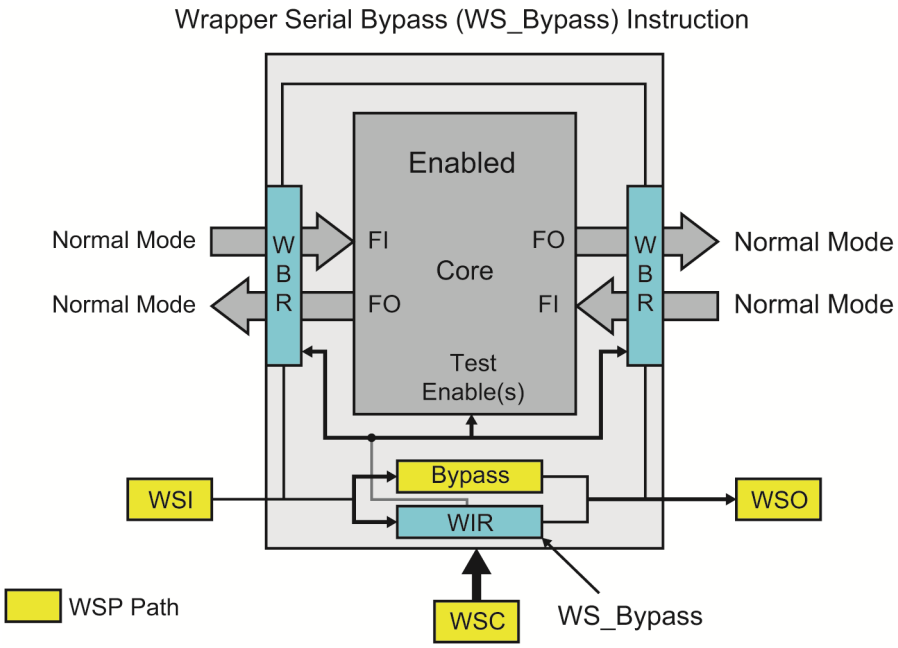


Figure 3 —WS_BYPASS example

7.5 WS_EXTEST instruction

7.5.1 Introduction

The mandatory WS_EXTEST instruction allows testing of off-core circuitry and core-to-core interconnections.

NOTE 1—Wrapper boundary cells that support the Wx_PRELOAD instruction (see 7.9 and 7.10) are presumed to have data loaded into the WBR using the Wx_PRELOAD instruction prior to loading the Wx_EXTEST instruction.[12]

NOTE 2—Following use of the WS_EXTEST instruction, the core circuitry can be in an indeterminate state that would persist until a system reset is applied. Therefore, the core circuitry might need to be reset upon return to normal (i.e., nontest) operation.

7.5.2 Specifications

Rules

a) Each wrapper shall provide a WS_EXTEST instruction.

b) While the WS_EXTEST instruction is active, only the WBR shall be connected for serial access between WSI and WSO(R) during the Shift event.

c) While the WS_EXTEST instruction is active, the core circuitry shall be controlled so that it cannot be damaged as a result of signals received at core input or core clock input terminals.

[12] Notes in text, tables, and figures are given for information only and do not contain requirements needed to implement this standard.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

NOTE—This might be achieved by placing the core circuitry in a reset or hold state while the WS_EXTEST instruction is active.

d) While the WS_EXTEST instruction is active, the state of all signals driven from the WBR wrapper functional outputs (WFOs) shall be completely defined by the data held in the WBR cell(s) associated with that terminal. The data at the output shall be valid with the Apply event and remain stable until the next WBR event.

NOTE 1— Where the WBR provides a silent shift path, the data held in the WBR are presumed to have been loaded by the previous use of an instruction such as WS_PRELOAD.

NOTE 2— Where the WBR comprises shared wrapper boundary cells (i.e., the wrapper boundary cells are shared between functional and test operation) that do not have a silent shift path, the data held in the WBR exist as a result of the previous instruction or of entering the WS_EXTEST instruction.

e) While the WS_EXTEST instruction is active, the state of all signals received at the core input terminals required to be provisioned with wrapper boundary cells per rule 12.2.1(a) shall be loaded into the WBR during the Capture event.

f) While the WS_EXTEST instruction is active, the state of WFO terminals shall not change to an unknown logic state in response to the Capture event.

g) While the WS_EXTEST instruction is active, the WBR shall be in OF mode.

h) While the WS_EXTEST instruction is active, the WSC shall be utilized for test control.

Recommendations

i) While the WS_EXTEST instruction is active, the core should be put into a quiet mode (e.g., reset or clock off).

j) While the WS_EXTEST instruction is active, all output cells that are in the WBR shift path should control the associated WFOs to safe values during shift.

Permissions

k) A binary code for the WS_EXTEST instruction may be selected by the wrapper designer.

7.5.3 Description

The WS_EXTEST instruction allows circuitry external to the core wrapper (typically the interconnects and user-defined logic [UDL]) to be tested. The wrapper boundary cells at WFOs are used to apply test stimuli, while the cells at wrapper input terminals capture test results. This instruction also allows testing of blocks of UDL between cores that do not themselves incorporate wrappers.

While the WS_EXTEST instruction is active, the core circuitry might receive input signals that differ significantly from those expected during normal operation. If the core circuitry can tolerate any permutation of input signals that is received, then no specific design changes are required. However, for some cores, there can be input sequences that could place the core circuitry in a state where damage can result, or excessive current can be drawn. In these cases, it is the responsibility of the wrapper designer to prevent the core circuitry from processing the “illegal” inputs while the WS_EXTEST instruction is active. This can be achieved by placing the core circuitry into a reset or hold state during WS_EXTEST or by controlling the CFOs of the WBR cells to present safe values to the core.

The wrapper input terminals provisioned with wrapper boundary cells per rule 12.1.1(a) may optionally be designed to allow signals to be driven into the core circuitry when the WS_EXTEST instruction is active. This allows user-defined values to be established at the core input terminals, preventing undesired operation of the core in response to unknown signals arriving at core inputs during operation of the WS_EXTEST

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

instruction. The values driven can either be constant for the duration that WS_EXTEST is selected (i.e., by using signal blocking gates at the core input terminals) or be loaded serially through the WBR. Figure 4 shows an example of the configuration of the wrapper during the WS_EXTEST instruction.

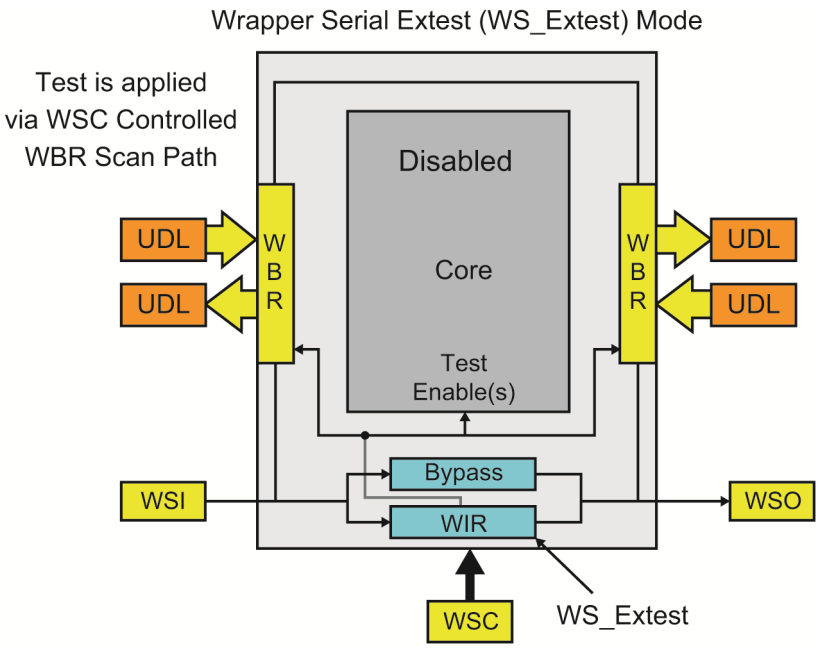


Figure 4 —WS_EXTEST example

7.6 WP_EXTEST instruction

7.6.1 Introduction

The WP_EXTEST instruction allows testing of off-core circuitry and core-to-core interconnections. The WP_EXTEST instruction allows the WBR to be divided into segments (multiple scan chains). It also allows wrapper inputs and wrapper outputs, other than the WSP, to control WBR inputs and observe WBR outputs. In all other behaviors, WP_EXTEST mimics the WS_EXTEST instruction.

NOTE 1—Wrapper boundary cells that support "preload" are presumed to have data loaded into the WBR using the WS_PRELOAD or WP_PRELOAD instruction prior to loading the WP_EXTEST instruction.

NOTE 2—Following use of the WP_EXTEST instruction, the core circuitry can be in an indeterminate state that would persist until a system reset is applied. Therefore, the core circuitry might need to be reset upon return to normal (i.e., nontest) operation.

7.6.2 Specifications

Rules

a) While the WP_EXTEST instruction is active, only the WBR shall be selected between WPI and WPO during the Shift event.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

b) While the WP_EXTEST instruction is active, the core circuitry shall be controlled so that it cannot be damaged as a result of signals received at core input or core clock input terminals.

NOTE—This might be achieved by placing the core circuitry in a reset or hold state while the WP_EXTEST instruction is active.

c) While the WP_EXTEST instruction is active, the state of all signals received at the wrapper input terminals required to be provisioned with wrapper boundary cells per rule 12.3.2(a) shall be loaded into the WBR during the Capture event.

d) While the WP_EXTEST instruction is active, the state of all signals driven from the WBR WFOs shall be completely defined by the data held in the WBR cell(s) associated with that terminal. The data at the output shall be valid with the Apply event and remain stable until the next WBR event.

NOTE 1—Where the WBR provides a silent shift path, the data held in the WBR are presumed to have been loaded by the previous use of an instruction such as WS_PRELOAD.

NOTE 2—Where the WBR comprises shared wrapper boundary cells (i.e., the wrapper boundary cells are shared between functional and test operation) that do not have a silent shift path, the data held in the WBR shall exist as a result of either the previous instruction or of entering the WP_EXTEST instruction.

e) While the WP_EXTEST instruction is active, the WBR shall be in OF mode.

f) While the WP_EXTEST instruction is active, the state of WFO terminals shall not change to an unknown logic state in response to the Capture event.

Recommendations

g) While the WP_EXTEST instruction is active, the core should be put into a quiet mode (e.g., reset or clock off).

h) While the WP_EXTEST instruction is active, where output cells that are in the WBR shift path are used, the WFOs should be controlled to safe values during shift.

Permissions

i) The WBR may be segmented into one or more scan chains with the input and output port of the scan chains interfaced to the TAM.

j) WP_EXTEST may use the WSC in place of the WPC.

k) The binary code for the WP_EXTEST instruction may be selected by the wrapper designer.

7.6.3 Description

The optional WP_EXTEST instruction allows circuitry external to the core wrapper (typically the interconnects and glue logic) to be tested. Wrapper boundary cells at WFOs are used to apply test stimuli, while the cells at wrapper input terminals capture test results. This instruction also allows testing of blocks of UDL between cores that do not themselves incorporate wrappers.

While the WP_EXTEST instruction is active, the core circuitry might receive input signals that differ significantly from those expected during normal operation. If the core circuitry can tolerate any permutation of input signals that is received, then no specific design changes are required. However, for some cores, there can be input sequences that could place the core circuitry in a state where damage can result, or excessive current can be drawn. In these cases, it is the responsibility of the designer to prevent the core circuitry from processing the “illegal” inputs while the WP_EXTEST instruction is active. This can be achieved by placing the core circuitry into a reset or hold state during WP_EXTEST or by controlling the cell functional outputs (CFOs) connected to the core.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

ShiftWR, CaptureWR, and UpdateWR terminals are defined in 8.2.3.

The wrapper input terminals provisioned with wrapper boundary cells per rule 12.2.1(a) may optionally be designed to allow signals to be driven into the core circuitry when the WP_EXTEST instruction is active. This allows user-defined values to be established at the core input terminals, preventing undesired operation of the core in response to unknown signals arriving at core inputs during operation of the WP_EXTEST instruction. The values driven can either be constant for the duration that WP_EXTEST is selected (e.g., by using signal blocking gates at the core input terminals) or be loaded serially through the WBR. Figure 5 shows an example of the configuration of the wrapper during the WP_EXTEST instruction. Note that both the WPC and the WSC control pins are shown in the figure; however, either the WPC or the WSC can be used during the WP_EXTEST instruction, not a combination of both. This is the only parallel standard instruction that can utilize the WSC. The reason that WSC is allowed to be used only for the WP_EXTEST instruction is because it can help to ensure plug-n-play between different wrappers and can be a common strategy for this instruction.

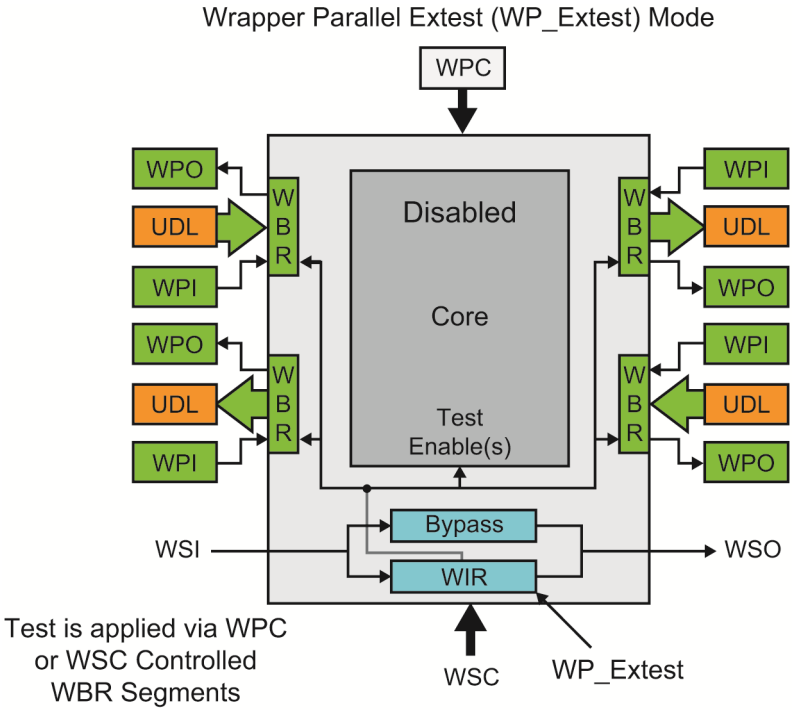


Figure 5 —WP_EXTEST example

7.7 Wx_EXTEST instruction

7.7.1 Introduction

Wx_EXTEST is an external test instruction that allows the circuitry external to the core to be tested according to the system integrator's requirements. The x in Wx is a place holder for an S, P, or H to indicate whether the instruction is serial, parallel, or hybrid, respectively.

The Wx_EXTEST instruction encompasses the WS_EXTEST and the WP_EXTEST instructions. This instruction provides the same flexibility as the Wx_INTEST instruction. The difference between the two instructions is that Wx_INTEST is an IF test and Wx_EXTEST is an OF test.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

NOTE—Following use of the Wx_EXTEST instruction, the core circuitry can be in an indeterminate state that would persist until a system reset is applied. Therefore, the core circuitry might need to be reset upon return to normal (i.e., nontest) operation.

The rules in 7.7.2 apply where a Wx_EXTEST instruction is provided.

7.7.2 Specifications

Rules

a) While the Wx_EXTEST instruction is active, the WBR shall be in OF mode.

Recommendations

b) While the Wx_EXTEST instruction is active, the core should be put into a quiet mode (e.g., reset or clock off).

c) While the Wx_EXTEST instruction is active, the testing of the circuitry external to the core should not disturb circuitry internal to the core.

d) While the Wx_EXTEST instruction is active, for core output terminals provisioned with wrapper boundary cells per rule 12.2.1(a), the data loaded during the Capture event should be independent of the operation of core circuitry.

e) WS_EXTEST or WP_EXTEST instructions should be utilized unless the more flexible capabilities of the Wx_EXTEST instruction are needed.

Permissions

f) The binary code(s) for the Wx_EXTEST instruction may be selected by the wrapper designer.

g) Any number of Wx_EXTEST instructions may be included by the wrapper designer.

7.7.3 Description

The Wx_EXTEST instruction allows testing of circuitry external to the core. While the Wx_EXTEST instruction is active, the WBR assumes the role of test points. Cells at core output terminals are used to apply the test stimulus, while those at core input terminals capture the test response. Stimuli and responses are moved into and out of the circuit by shifting through the WBR.

7.8 WS_SAFE instruction

7.8.1 Introduction

The optional WS_SAFE instruction allows the state of the signals driven from WFOs to be determined from the WBR while the WBY is selected as the serial path between the serial input (WSI) and serial output (WSO[R]) of the wrapper. The signals driven from the WFOs shall not change while the WS_SAFE instruction is active. The WS_CLAMP instruction can also be used to put the wrapper into a safe state by utilizing a Wx_PRELOAD instruction to shift in the proper states and then clamping them with the WS_CLAMP instruction. However, it is important to allow a straightforward way for the core integrator to put wrappers into a safe state. This is why there is a separate instruction for WS_SAFE. This instruction allows the core integrator to more easily put various wrappers into safe states while other portions of an SoC are being tested.

The rules in 7.8.2 apply where the WS_SAFE instruction is provided.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

NOTE—Following use of the WS_SAFE instruction, the core circuitry can be in an indeterminate state that would persist until a system reset is applied. Therefore, the core circuitry might need to be reset upon return to normal (i.e., nontest) operation.

7.8.2 Specifications

Rules

a) While the WS_SAFE instruction is active, the WBY shall be connected for serial access between WSI and WSO(R) during wrapper data scan operations.

b) While the WS_SAFE instruction is active, the state of all signals driven from WFOs shall be hardwired to fixed values that have been predetermined at wrapper instantiation.

c) NOTE—The data output from the WBR shall be safe and established as a result of entering the WS_SAFE instruction.

d) When the WS_SAFE instruction is active, the core circuitry shall be controlled so that it cannot be damaged as a result of signals received at the core data input or core clock input terminals.

e) While the WS_SAFE instruction is active, the WSC shall be utilized for test control.

Recommendations

f) While the WS_SAFE instruction is active, the core circuitry should be put into a quiet mode (e.g., reset or clock off).

Permissions

g) The binary code for the WS_SAFE instruction may be selected by the wrapper designer.

7.8.3 Description

During testing of a particular core or cluster of cores on a system chip, it might be necessary to place static values on signals that control operation of circuitry not involved in the test, e.g., to place the core in a state where it does not respond to signals received from the circuitry under test.

The optional WS_SAFE instruction is similar to the WS_CLAMP instruction in that it allows safe static values to be applied using the WBRs of the appropriate core wrapper, but does not retain these registers in the serial path during test application. In a case in which the WS_SAFE instruction is used to create safe static signals, the following process would be used.

It is presumed in the following example that every wrapper implements the optional WS_SAFE instruction.

— Shift the WS_SAFE instruction into all core wrappers that will provide safe static signals during the upcoming test. Call this group of wrappers *G*. If test setup data are required in wrappers not in *G* (i.e., in the wrappers that will participate actively in the upcoming test), another instruction (e.g., WS_EXTEST) can be loaded into these core wrappers simultaneously with the loading of WS_SAFE into core group *G*.

— From this point on, until the test is concluded, every time instructions are to be scanned into core wrappers in the SoC, enter the WS_SAFE instruction into the wrappers in *G*. As long as the WS_SAFE instruction is maintained as the active instruction in the wrappers of *G*, the output signal values of these wrappers will be determined by the safe data at their WBR outputs. Also, as a consequence of the use of the WS_SAFE instruction, the wrappers in *G* all have their WBYs selected throughout the test; thus, they contribute very little to the overall test time. Figure 6 shows an example of the configuration of the wrapper during the WS_SAFE instruction.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

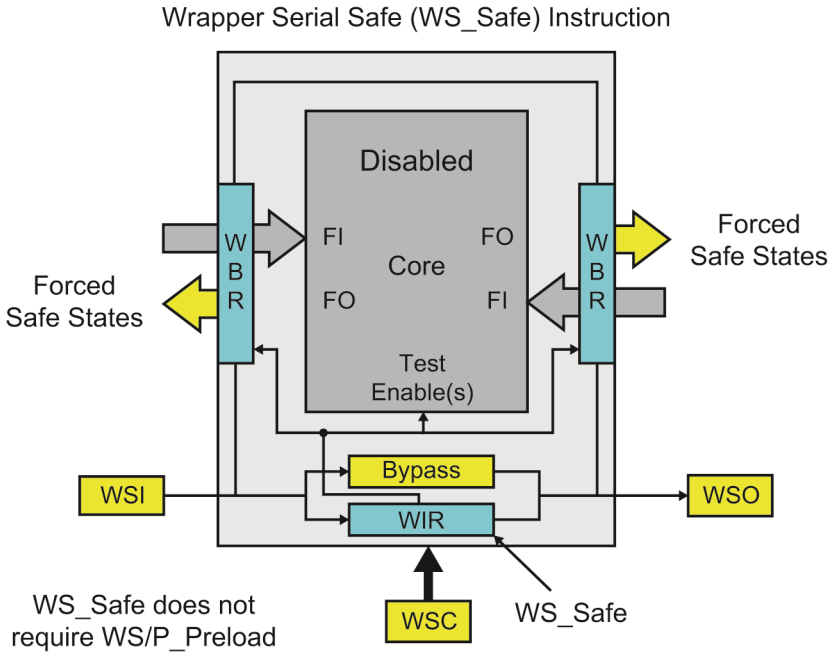


Figure 6 —WS_SAFE example

7.9 WS_PRELOAD instruction

7.9.1 Introduction

The WS_PRELOAD instruction enables the wrapper to be functionally configured. Wrappers that have a WBR that contains a silent shift path shall include the WS_PRELOAD instruction. The WS_PRELOAD instruction allows data values to be loaded into the latched parallel outputs of the registers in the silent shift path of the WBR. The data are loaded into the silent shift path of wrapper cells, and then the data are loaded into the update stage, if present, during the Update event. This instruction would typically be utilized before other instructions are selected (e.g., WS_EXTEST, WS_CLAMP).

7.9.2 Specifications

Rules

a) If the WBR contains a silent shift path, there shall be a WS_PRELOAD instruction.

b) While the WS_PRELOAD instruction is active, only the WBR shall be connected for serial access between WSI and WSO(R) during the Shift event.

c) While the WS_PRELOAD instruction is active, the Shift event of the WBR shall have no effect on the operation of the core or UDL.

d) While the WS_PRELOAD instruction is active, the WSC shall be utilized for test control.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Recommendations

e) While the WS_PRELOAD instruction is active, if the core is not being used in its normal operation, the core circuitry should be put into a quiet mode (e.g., reset or clock off).

Permissions

f) The binary code for the WS_PRELOAD instruction may be selected by the wrapper designer.

7.9.3 Description

The WS_PRELOAD instruction is used to allow shifting of the WBR, via WSI to WSO(R), without causing interference to the operation of the core or UDL attached to the WBR. WS_PRELOAD allows an initial data pattern to be placed at the latched parallel outputs of the silent shift path register cells through a scan load operation. For example, prior to selection of the WS_EXTEST instruction, data can be loaded onto the latched parallel outputs using WS_PRELOAD. As soon as the WS_EXTEST instruction has been transferred to the update register of the instruction register, the preloaded data are driven from the WFOs. This should result in known data being driven immediately when the WS_EXTEST instruction is entered. Without WS_PRELOAD, indeterminate data might be driven until the first scan load is complete. Figure 7 shows an example of the configuration of the wrapper during the WS_PRELOAD instruction.

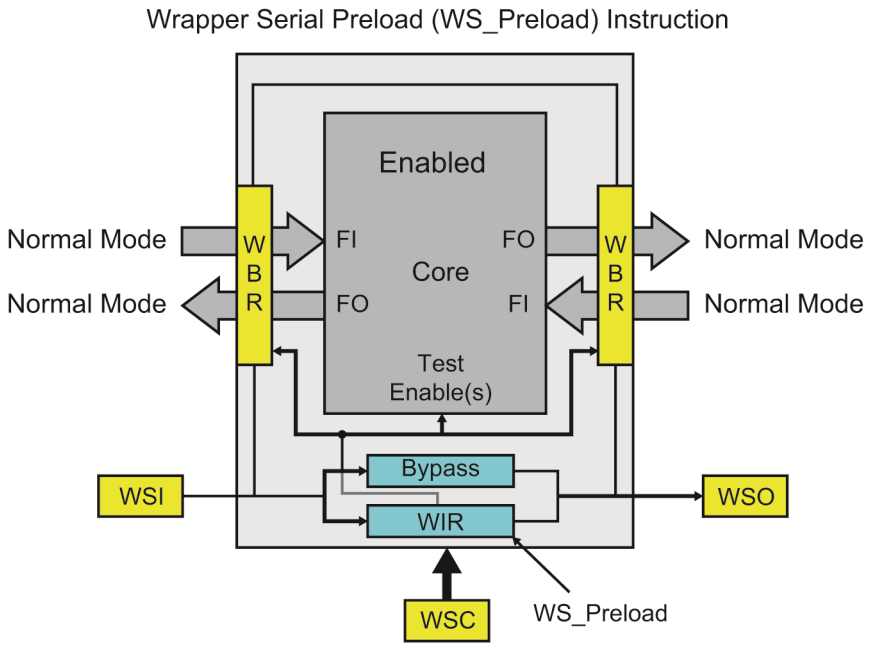


Figure 7 —WS_PRELOAD example

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

7.10 WP_PRELOAD instruction

7.10.1 Introduction

The WP_PRELOAD instruction enables the wrapper to be functionally configured. Wrappers that have a WBR composed entirely of cells with a silent shift path and have parallel instruction capability, can include the WP_PRELOAD instruction. The WP_PRELOAD instruction allows data values to be loaded into the latched parallel outputs of the registers in the silent shift path(s) of the WBR. The data are loaded into the silent shift paths of wrapper cells, and then the data are loaded into the update stage, if present, during the Update event. This instruction would typically be utilized before other defined instructions are selected (e.g., WP_EXTEST). The WP_PRELOAD instruction allows the WBR to be divided into segments (multiple scan chains) and might be preferred over the WS_PRELOAD instruction before entering the WP_EXTEST instruction. It also allows wrapper inputs and outputs, other than the WSP, to control WBR inputs and observe WBR outputs. In all other behaviors, WP_PRELOAD mimics the WS_PRELOAD instruction.

7.10.2 Specifications

Rules

a) While the WP_PRELOAD instruction is active, only the WBR shall be connected during the Shift event.

b) While the WP_PRELOAD instruction is active, the data in the wrapper boundary cell shall be available for the Update event, if applicable.

c) While the WP_PRELOAD instruction is active, the Shift event of the WBR shall have no effect on the operation of the core or UDL.

d) While the WP_PRELOAD instruction is active, the WPC shall be utilized for test control.

Recommendations

e) While the WP_PRELOAD instruction is active, if the core is not being used in its normal operation, the core circuitry should be put into a quiet mode (e.g., reset or clock off).

Permissions

f) The WBR may be segmented into one or more scan chains with the input and output port of the scan chains interfaced to the TAM.

g) The binary code for the WP_PRELOAD instruction may be selected by the wrapper designer.

7.10.3 Description

The WP_PRELOAD instruction is used to allow shifting of the WBR without causing interference to the operation of the core or UDL attached to the WBR. WP_PRELOAD allows an initial data pattern to be placed at the latched parallel outputs of the silent shift path register cells through a scan load operation. For example, prior to selection of the WP_EXTEST instruction, data can be loaded onto the latched parallel outputs using WP_PRELOAD. As soon as the WP_EXTEST instruction has been transferred to the parallel output of the instruction register, the preloaded data are driven from the output wrapper terminals. This should result in known data being driven immediately when the WP_EXTEST instruction is entered. Without WP_PRELOAD, indeterminate data might be driven until the first scan load is complete. Figure 8 shows an example of the configuration of the wrapper during the WP_PRELOAD instruction. In the figure, the WPP Paths go from a WPI, through a WBR, to a WPO and controlled by WPC.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

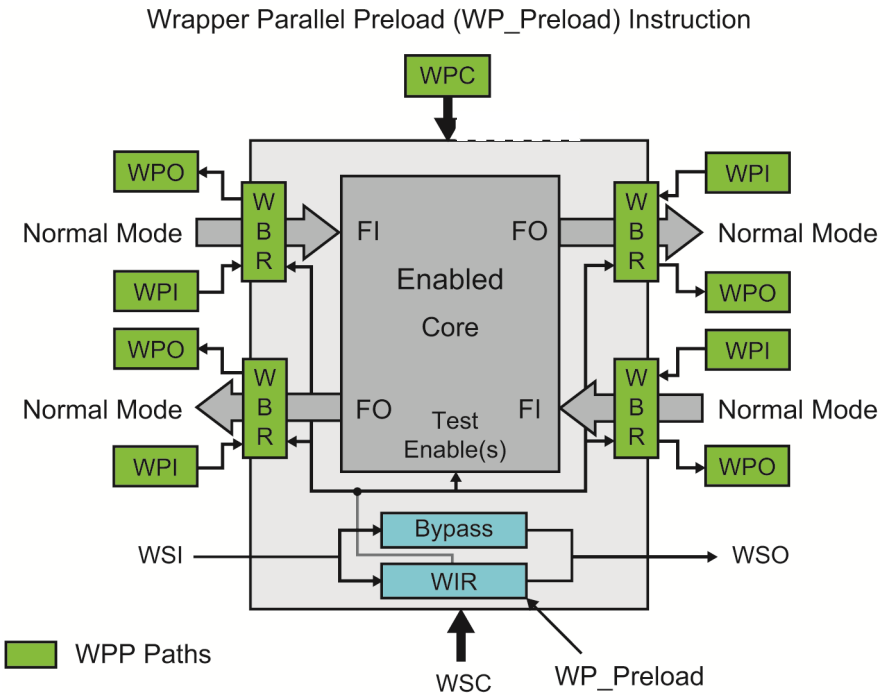


Figure 8 —WP_PRELOAD example

7.11 WS_CLAMP instruction

7.11.1 Introduction

The optional WS_CLAMP instruction allows the state of the signals driven from WFOs to be determined from the WBR while the WBY is selected as the serial path between WSI and WSO(R). The signals driven from the WFOs shall not change while the WS_CLAMP instruction is active.

The rules in 7.11.2 apply where the WS_CLAMP instruction is provided.

NOTE—Following use of the WS_CLAMP instruction, the core circuitry can be in an indeterminate state that would persist until a system reset is applied. Therefore, the core circuitry might need to be reset upon return to normal (i.e., nontest) operation.

7.11.2 Specifications

Rules

a) While the WS_CLAMP instruction is active, the WBY shall be connected for serial access between WSI and WSO(R).

b) While the WS_CLAMP instruction is active, the state of all signals driven from WFOs shall be completely defined by the data held in the WBR.

c) NOTE—Data held in the WBR might be achieved by the previous use of an instruction (e.g., WS_PRELOAD).

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

d) While the WS_CLAMP instruction is active, the WSC shall be utilized for test control.

Recommendations

e) While the WS_CLAMP instruction is active, the core circuitry should be put into a quiet mode (e.g., reset or clock off).

Permissions

f) The binary code for the WS_CLAMP instruction may be selected by the wrapper designer.

7.11.3 Description

During testing of a particular core or cluster of cores on a system chip, it might be necessary to place static values on signals that control operation of circuitry not involved in the test, e.g., to place the aforementioned circuitry in a state where it cannot respond to signals received from the circuitry under test.

The optional WS_CLAMP instruction allows static values to be applied using the WBRs of the appropriate core wrapper, but does not retain these registers in the serial path during test application. In a case in which the WS_CLAMP instruction is used to create static values, the following processes would be used.

It is presumed in the following example that every wrapper implements the optional WS_CLAMP instruction.

Example: WBRs having dedicated wrapper boundary cells

— Prior to the test, a Wx_EXTEST or Wx_PRELOAD (the x denotes S, P, or H) instruction would be loaded into all core wrappers that will provide specific values during the upcoming test. Call this group of wrappers *G*. If test setup data are required in wrappers not in *G* (i.e., in the wrappers that will participate actively in the upcoming test), a Wx_EXTEST or Wx_PRELOAD instruction can be loaded into these core wrappers simultaneously with the loading of WS_CLAMP into core group *G*.

— Shift the desired pattern into all relevant wrapper boundary cells of the wrappers in *G*. Any test setup data required for the SoC to be tested are also loaded.

— Load WS_CLAMP instruction into the wrappers in *G*.

— From this point on, until the test is concluded, every time instructions are to be scanned into core wrappers in the SoC, enter the WS_CLAMP instruction into the wrappers in *G*. As long as the WS_CLAMP instruction is maintained as the active instruction in the wrappers of *G*, the output signal values of these wrappers will be determined by the data in their WBRs. Also, as a consequence of the use of the WS_CLAMP instruction, the wrappers in *G* all have their WBYs selected throughout the test; thus, they contribute very little to the overall test time.

Figure 9 shows an example of the configuration of the wrapper during the WS_CLAMP instruction. This instruction would be loaded serially into the core wrappers that drive the signals on which static values are required. The required signal values could be loaded as a part of the complete serial data stream shifted into the chip-level WBR path, both at the start of the test and each time a new test pattern is entered. However, a limitation of this approach is that the length of the data pattern to be shifted for each test is increased by inclusion of the WBR of each core wrapper involved in the process. As a result, the test application rate is reduced. WS_CLAMP reduces overall shift length when the target wrapper does not take an active part in the test.

Note that this is just one example of how to clamp a signal. Instructions such as Wx_PRELOAD or Wx_EXTEST can be used to load specific values into the wrapper before WS_CLAMP is enabled.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

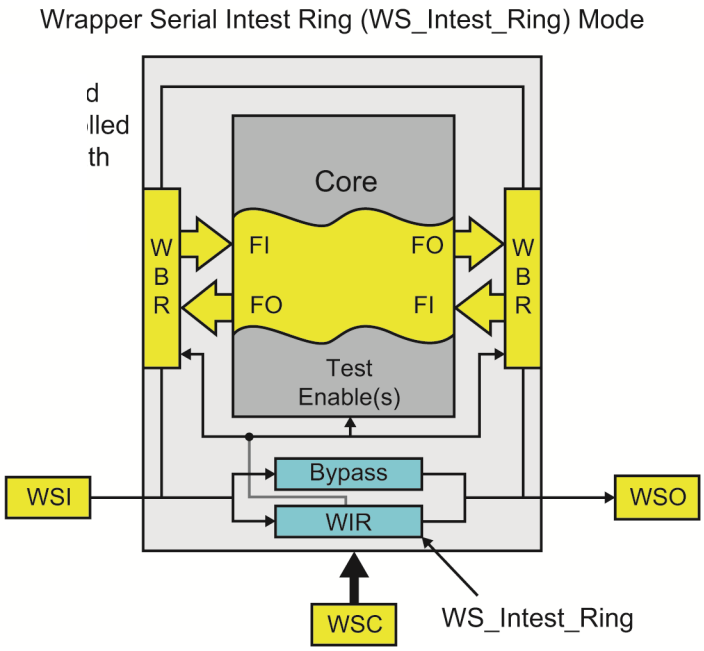


Figure 9 —WS_CLAMP example

7.12 WS_INTEST_RING instruction

7.12.1 Introduction

The optional serial wrapper core test WS_INTEST_RING instruction allows testing of the core circuitry. During the WS_INTEST_RING instruction, test stimuli are shifted in 1 bit at a time and applied to the core functional terminals via the WBR only. The test results are captured into the WBR and are shifted out 1 bit at a time for examination. The rules in 7.12.2 apply where the WS_INTEST_RING instruction is provided.

NOTE—Following the use of the WS_INTEST_RING instruction, the core circuitry can be in an indeterminate state that would persist until a core reset is applied. Therefore, the core circuitry might need to be reset upon return to normal operation.

7.12.2 Specifications

Rules

a) The WS_INTEST_RING instruction shall select only the WBR to be connected for serial access between WSI and WSO(R).

b) While the WS_INTEST_RING instruction is active, the WBR shall be in IF mode.

c) While the WS_INTEST_RING instruction is active, the WSC shall be utilized for test control.

Recommendations

d) While the WS_INTEST_RING instruction is active, all WFOs should be placed in a safe or static drive state.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

e) While the WS_INTEST_RING instruction is active, for core input terminals provisioned with wrapper boundary cells per rule 12.2.1(a), the data loaded during the Capture event should be independent of the operation of off-core circuitry or chip-level interconnections.

Permissions

f) The binary code for the WS_INTEST_RING instruction may be selected by the wrapper designer.

7.12.3 Description

The WS_INTEST_RING instruction allows quasi-functional single-step testing of core circuitry with each test pattern and response being shifted through the WBR. The WS_INTEST_RING instruction requires that the core circuitry can be operated in a single-step mode, where the circuitry moves one step forward in its operation each time shifting of the WBR is completed.

While the WS_INTEST_RING instruction is active, the WBR assumes the role of test points. Cells at nonclock core input terminals are used to apply the test stimulus, while those at core output terminals capture the response. Stimuli and responses are moved into and out of the circuit by shifting through the WBR. Note that this requires that the core input terminals provisioned with wrapper boundary cells per rule 12.2.1(a) are able to drive signals into the core circuitry.

Typically, the core circuitry will receive a sequence of clock events between application of the stimulus and capture of the response so that single-step operation is achieved. The specification of wrapper boundary cells for core clock input terminals allows the clocks for the core circuitry to be obtained in several ways while the WS_INTEST_RING instruction is active. The following are offered as examples:

a) The signals received at core clock terminals can be fed directly to the core circuitry as during normal operation of the core. Where this option is selected, the core design would be such that precisely one single step of operation of the core circuitry occurs between each shifting (filling) of the WBR.

b) Circuitry can be built into the core that allows the core circuitry to complete one step of operation on completion of shifting in data into the WBR. If the core were a microprocessor, it would be permitted to complete a single processing cycle by, for example, internal generation of a pulse on the hold signal. In this case, the clock(s) applied at the core clock terminal(s) during the test could be free-running.

While the WS_INTEST_RING instruction is active, the state of all WFOs is determined by the wrapper circuitry. Every WFO should be forced to a static or inactive drive state. This causes the surrounding circuitry on an SoC to be supplied with known, safe signal levels while the core circuitry test is in progress.

Recommendation 7.12.2(e), where followed, helps ensure that data shifted out of the core in response to the WS_INTEST_RING instruction are not altered by the presence of defects in off-core circuitry, chip-level interconnections, etc. This simplifies diagnosis, since any errors in the output bit stream can be caused only by defects in the core circuitry or in the WBR. Figure 10 shows an example of the configuration of the wrapper during the WS_INTEST_RING instruction.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

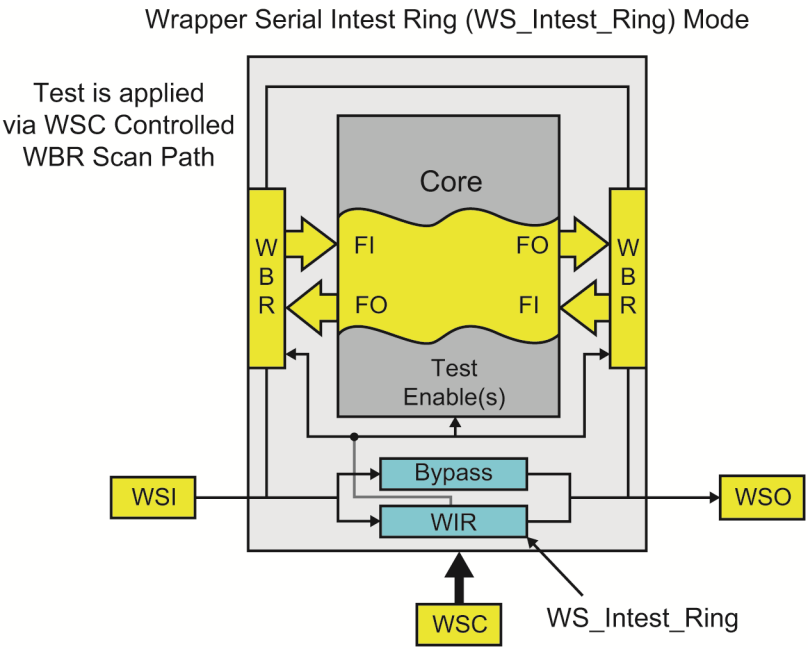


Figure 10 —WS_INTEST_RING example

7.13 WS_INTEST_SCAN instruction

7.13.1 Introduction

The optional serial core test WS_INTEST_SCAN instruction allows structural scan testing of the core circuitry with a single serial chain comprised of the WBR concatenated with all the core's internal scan elements. A detailed example of a WS_INTEST_SCAN implementation is described in section 12.9.

The rules in 7.13.2 apply where the WS_INTEST_SCAN instruction is provided.

NOTE—Following the use of the WS_INTEST_SCAN instruction, the core circuitry can be in an indeterminate state that would persist until a core reset is applied. Therefore, the core circuitry might need to be reset upon return to normal operation.

7.13.2 Specifications

Rules

a) While the WS_INTEST_SCAN instruction is active, the WBR concatenated with an internal scan chain, which includes all internal scan cells, shall be connected for serial access between WSI and WSO(R).

b) While the WS_INTEST_SCAN instruction is active, the WBR shall be in IF mode.

c) While the WS_INTEST_SCAN instruction is active, the WSC shall be utilized for test control.

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Recommendations

d) While the WS_INTEST_SCAN instruction is active, all WFOs should be placed in a safe or static drive state.

e) While the WS_INTEST_SCAN instruction is active, for core input terminals provisioned with wrapper boundary cells per rule 12.2.1(a), the data loaded during the Capture event should be independent of the operation of off-core circuitry or chip-level interconnections.

Permissions

f) The binary code for the WS_INTEST_SCAN instruction may be selected by the wrapper designer.

7.13.3 Description

The WS_INTEST_SCAN instruction allows scan-based structural testing of core circuitry with each test pattern and response being shifted through the serial scan chain.

While the WS_INTEST_SCAN instruction is active, the serial scan chain assumes the role of test points. Cells at nonclock core input terminals are used to apply the test stimulus, while those at core output terminals capture the response. Stimuli and responses are moved into and out of the circuit by shifting the serial scan chain. This operation requires that the core input terminals provisioned with wrapper boundary cells per rule 12.2.1(a) are able to drive signals into the core circuitry.

Typically, the core circuitry will receive a sequence of clock events between application of the stimulus and capture of the response so that single-step operation is achieved. The specification of wrapper boundary cells for core clock input terminals allows the clocks for the core circuitry to be obtained in several ways while the WS_INTEST_SCAN instruction is active. The following are offered as examples:

a) The signals received at core clock terminals can be fed directly to the core circuitry as during normal operation of the core. Where this option is selected, the core design would be such that precisely one single step of operation of the core circuitry occurs between each shifting (filling) of the serial scan chain.

b) Circuitry can be built into the core that allows the core circuitry to complete one step of operation on completion of shifting in data into the serial scan chain. If the core were a microprocessor, it would be permitted to complete a single processing cycle by, for example, internal generation of a pulse on a hold signal. In this case, the clock(s) applied at the core clock terminal(s) during the test could be free-running.

While the WS_INTEST_SCAN instruction is active, the state of all WFOs is determined by the wrapper circuitry. Every WFO should be forced to a static or inactive drive state. This causes the surrounding circuitry on an SoC to be supplied with known, safe signal levels while the core circuitry test is in progress.

Recommendation 7.13.2(e), where followed, helps ensure that data shifted out of the core in response to the WS_INTEST_SCAN instruction are not altered by the presence of defects in off-core circuitry, chip-level interconnections, etc. This simplifies diagnosis, since any errors in the output bit stream can be caused only by defects in the core circuitry or in the WBR. Figure 11 shows an example of the configuration of the wrapper during the WS_INTEST_SCAN instruction.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

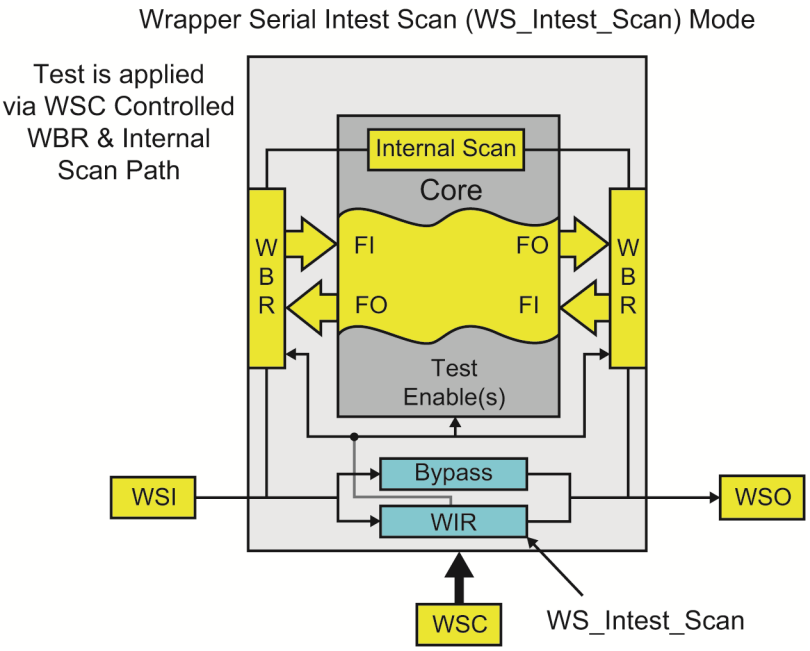


Figure 11 —WS_INTEST_SCAN example

7.14 Wx_INTEST instruction

7.14.1 Introduction

One core test instruction that allows the core to be tested according to a test procedure specified by the core provider or core user is required. IEEE Std 1500 does not describe how to test individual cores; this is the responsibility of the core provider. The core test invoked by the Wx_INTEST instruction (the x in Wx is a place holder for an S, P, or H to indicate whether the instruction is serial, parallel, or hybrid) is specified with the CTL test mode plus test data provided for the core in CTL or another IEEE TTSC (or successor) sponsored standard test data language. The Wx_INTEST CTL describes the attributes of the core test such as the following:

a) Number of cell test input (CTI) and cell test output (CTO) terminals

b) Protocol used to input and output the test patterns

c) Condition or state of core input and output terminals during the core test, excluding the TI and TO terminals

d) Whether the WBR is used during the core test, and if used, in what configuration it is placed

The Wx_INTEST instruction includes the WS_INTEST_RING and the WS_INTEST_SCAN instructions.

The pattern data provided along with the CTL for the core, which can be described in CTL or another IEEE TTSC (or successor) sponsored standard test data language, describes the following:

a) Test pattern set, including control and/or data that is applied to the core via associated TI and TO terminals.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

The rules in 7.14.2 apply for the Wx_INTEST instruction.

7.14.2 Specifications

Rules

a) Each wrapper shall provide at least one Wx_INTEST instruction.

b) While the Wx_INTEST instruction is active, the WBR shall be in IF mode.

Recommendations

c) While the Wx_INTEST instruction is active, the operation of the core should not disturb circuitry external to the core.

d) While the Wx_INTEST instruction is active, all WFOs should be placed in a safe or static drive state.

e) While the Wx_INTEST instruction is active, for core input terminals provisioned with wrapper boundary cells per rule 12.2.1(a), the data loaded during the Capture event should be independent of the operation of off-core circuitry or chip-level interconnections.

Permissions

f) The WBR may be segmented into one or more scan chains with the input and output port of the scan chains interfaced to the TAM.

g) The binary code for the Wx_INTEST instruction may be selected by the wrapper designer.

h) Any number of Wx_INTEST instructions (Wx_INTEST_user_defined) may be included by the wrapper designer.

7.14.3 Description

The Wx_INTEST instruction allows testing of core circuitry.

While the Wx_INTEST instruction is active, the WBR can participate and assume the role of test points. Cells at nonclock core input terminals are used to apply the test stimulus, while those at core output terminals capture the response. Stimuli and responses are moved into and out of the circuit by shifting the WBR. Note that this requires that the core input terminals provisioned with wrapper boundary cells per rule 12.1.1(a) are able to drive signals into the core circuitry.

Typically, the core circuitry will receive a sequence of one or more clock events between application of the stimulus and capture of the response. The specification of wrapper boundary cells for core clock input terminals allows the clocks for the core circuitry to be received in several ways while the Wx_INTEST instruction is active. The following are offered as examples:

a) The signals received at core clock terminals can be fed directly to the core circuitry as during normal operation of the core.

b) Circuitry can be built into the core that allows the core circuitry to complete one step of operation on completion of shifting in data into the WBR. If the core were a microprocessor, it would be permitted to complete a single processing cycle by, for example, internal generation of a pulse on the hold signal. In this case, the clock(s) applied at the core clock terminal(s) during the test could be free-running.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

While the Wx_INTEST instruction is active, the state of all WFOs is determined by the wrapper circuitry. Every WFO should be forced to a static or inactive drive state. This causes the surrounding circuitry on an SoC to be supplied with known, safe signal levels while the core circuitry test is in progress.

Recommendation 7.14.2(e), where followed, helps ensure that data shifted out of the core in response to the Wx_INTEST instruction are not altered by the presence of defects in off-core circuitry, chip-level interconnections, etc. This simplifies diagnosis, since any errors in the output bit stream can be caused only by defects in the core circuitry or in the WBR.

8. Wrapper serial port (WSP)

8.1 Introduction

The WSP is a set of wrapper terminals that facilitate standard plug-and-play (PnP) operation of a core that implements the IEEE 1500 architecture. IEEE Std 1500 defines mandatory WSP terminals to control serial access to the WIR, the WBY, and the WBR.

The WSP includes the following mandatory terminals: WSI, WSO, and a set of WSC terminals. The WSC comprises the wrapper clock (WRCK), wrapper reset (WRSTN), SelectWIR, CaptureWR, ShiftWR, and UpdateWR terminals. The WSP can also include the optional WSOR terminal. Some WBR implementations can also operate using one or more optional auxiliary clock (AUXCK) inputs, which are depicted as the AUXCKn terminal(s) in Figure 12. The AUXCKn terminal(s) are included in the WSC. The above standard WSP terminals are described in further detail in 8.2.

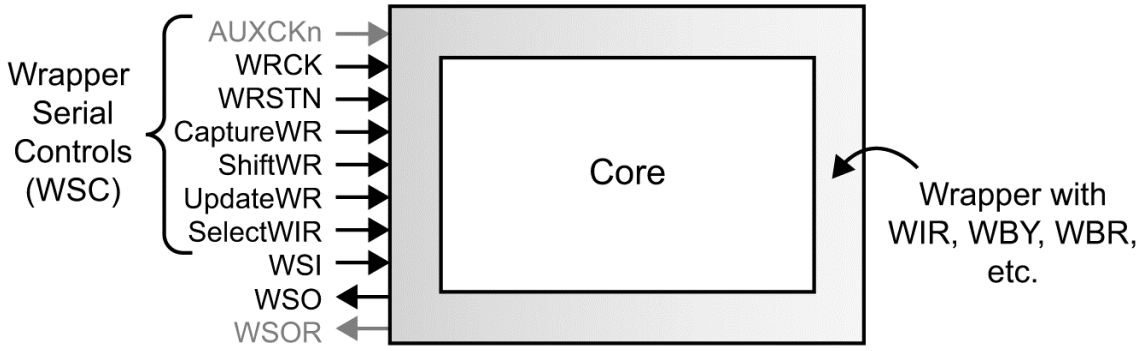


Figure 12—WSP

8.2 WSP terminals

8.2.1 Introduction

This subclause defines the standard WSP terminals of the IEEE 1500 wrapper. Note that only the WSP's terminals are specified in this subclause, whereas the WSP timing and protocol, as used to operate the standard features of the IEEE 1500 architecture, are specified and described in subsequent clauses of this standard.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

8.2.2 Specifications

Rules

a) The WSP is mandatory and shall include the following mandatory terminals: WSI, WSO, WRCK, WRSTN, SelectWIR, CaptureWR, ShiftWR, and UpdateWR.

b) All mandatory WSP terminals shall be dedicated wrapper terminals (i.e., they shall not be used for any other system function).

c) If user-defined AUXCK(s) are used, per permission 8.2.2(d), the timing relationship between WRCK and the AUXCK(s) shall be sufficiently specified and documented for use by the SoC integrator.

Permissions

d) The WSP may include AUXCKn terminal(s) in addition to the dedicated WRCK for operation of registers other than the WBY and WIR. Such clocks may be shared with other system clocks and may be used to operate other core features.

e) The user may select a user-defined name for the AUXCK terminal(s).

f) When the mandatory WSO terminal is specified to change on the falling edge of WRCK, the WSP may include a WSOR terminal that provides a second wrapper serial output that changes on the rising edge of WRCK.

8.2.3 Description

The mandatory WSP terminals are WSI, WSO, WRCK, WRSTN, SelectWIR, CaptureWR, ShiftWR, and UpdateWR. The signals connected to these WSP terminals are used to select whether the WIR or a WDR is connected between WSI and WSO(R) and to select and operate either the WIR or a selected WDR between WSI and WSO(R). The WSP can also include the optional WSOR and AUXCKn terminal(s).

The following definitions apply to the rules specified in 8.2.2.

WSI, **WSO, and optionally WSOR:** These WSP terminals are used to scan in and scan out wrapper instructions and data.

WRCK: The signal connected to the WRCK terminal is a dedicated clock used to operate IEEE 1500 functions.

AUXCKn: AUXCKn are optional auxiliary clocks (AUXCK) used [per permission 8.2.2(d)] with some implementations of the WBR. These can also be shared with system clocks and shall have a user-specified timing relationship to WRCK for operation of the WBR. It is anticipated that more than one AUXCK terminal could exist in an IEEE 1500 wrapper. In such a case, the integer n is used to differentiate these terminals (e.g., AUXCK1, AUXCK2). Subclause 16.2 provides further discussion on AUXCKs.

WRSTN: When asserted, WRSTN puts the wrapper into its normal system mode. The signal can be used to reset other WRs or wrapper circuitry as needed.

SelectWIR: SelectWIR determines what type of WR operation, i.e., instruction or data, is to be performed. The signal value selects between the WIR and the data registers, e.g., WBY or WBR. While SelectWIR is asserted to logic 1, the WIR is selected and connected between WSI and WSO(R). SelectWIR shall be de-asserted to logic 0 in order for any WDR or CDR to be selected and connected between WSI and WSO(R).

CaptureWR, ShiftWR, UpdateWR: These terminals control and enable WR operations. When one of these signals is asserted to logic 1 and the other two signals are deasserted to logic 0, a corresponding

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Capture, Shift, or Update operation will be enabled for the selected WR. While the WIR or WBY is selected, the enabled operation occurs synchronously to WRCK.

The operation of the WIR, WBY, and WBR using the WSP is described in further detail in Clause 10, Clause 11, and Clause 12, respectively.

9. Wrapper parallel port (WPP)

9.1 WPP terminals

9.1.1 Introduction

In addition to the mandatory WSP, IEEE Std 1500 supports an optional WPP as depicted in Figure 1.

WPP terminals are user-defined with the specifications in 9.1.2.

9.1.2 Specifications

Rules

a) If a user-defined WPP is used per permission (c), the timing relationships between its constituent signals shall be sufficiently specified and documented for use by the SoC integrator.

b) A WPP shall not include the ShiftWR, CaptureWR, UpdateWR, SelectWIR, WSI, WSO, or WSOR terminals.

Permissions

c) The wrapper designer may define one or more dedicated WPP(s).

d) A WPP may include the WRCK, AUXCK, and WRSTN terminals.

9.1.3 Description

It is permitted to have a parallel access mechanism for increased data bandwidth to the wrapped core. Figure 1, Figure 18, and Figure 20 show the WPP and its usage. A WPP port is distinct from the WSP, but can share the WRCK and AUXCKs with the WSP.

10. Wrapper instruction register (WIR)

10.1 Introduction

This clause specifies and describes the WIR. The WIR is an instruction register in which IEEE 1500 wrapper instructions are serially loaded through the standard WSP. The WIR also comprises interface circuitry to other IEEE 1500 components, such as the WBR and WBY, and can also interface to the circuitry of the core.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

10.2 WIR configuration and DR selection

10.2.1 Introduction

Figure 13 illustrates the WIR within the IEEE 1500 architecture. This figure shows how the standard data registers (i.e., WBR and WBY) and other optional data registers (i.e., WDRs and CDRs) are configured for access via the WSP and how they are selected by the WIR. The WIR contains a shift stage, instruction decode, and update stage. The WIR interfaces to the WSP, the IEEE 1500 wrapper, and conditionally the core per permission 10.3.2(i).

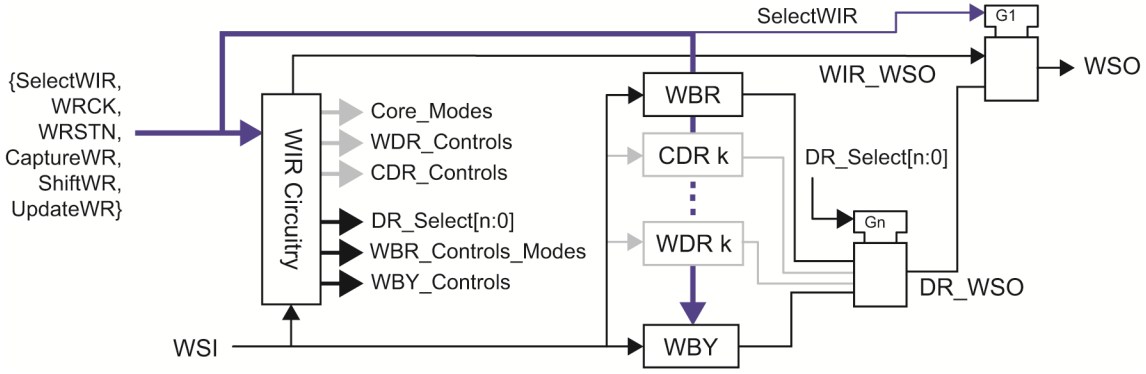


Figure 13—Example of WIR interface to WBY, WBR, and core

10.2.2 Specifications

Rules

a) The IEEE 1500 wrapper shall contain a single WIR.

b) The WIR shift register shall be selected and connected between WSI and WSO(R) when SelectWIR is asserted to logic 1, regardless of the current wrapper instruction.

c) Selection of registers other than the WIR (i.e., WDRs or CDRs) between WSI and WSO(R) shall be determined by the current wrapper instruction, and these registers are selected when SelectWIR is asserted to logic 0.

Permissions

d) Alternate mechanisms for access to WDRs or CDRs may be enabled by a user-defined wrapper instruction.

10.2.3 Description

The WIR is unconditionally accessible by the WSP by asserting SelectWIR. SelectWIR takes precedence in selecting the WIR, so that when it is logic 1, the WIR is connected between WSI and WSO(R) and is enabled to shift, update, or capture using the WSP. Thus, access to the WIR for loading wrapper instructions can be realized simply by asserting SelectWIR to logic 1, regardless of the current wrapper instruction and the selected WDR or CDR.

The access of any WDR or CDR through the WSP requires that SelectWIR be logic 0. The register accessed through WSI and WSO(R) is then determined based on the wrapper instruction previously updated in the WIR.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

User-defined wrapper instructions can enable an optional user-defined parallel port. Such a user-defined parallel interface can encompass an alternative selection mechanism for access to WDRs and/or CDRs. Permission 10.2.2(d) establishes that control of such alternate access mechanisms originates and terminates exclusively from the WIR.

10.3 WIR design

10.3.1 Introduction

A block diagram of a WIR design is depicted in Figure 14. The WIR is controlled and clocked by the standard WSC terminals.

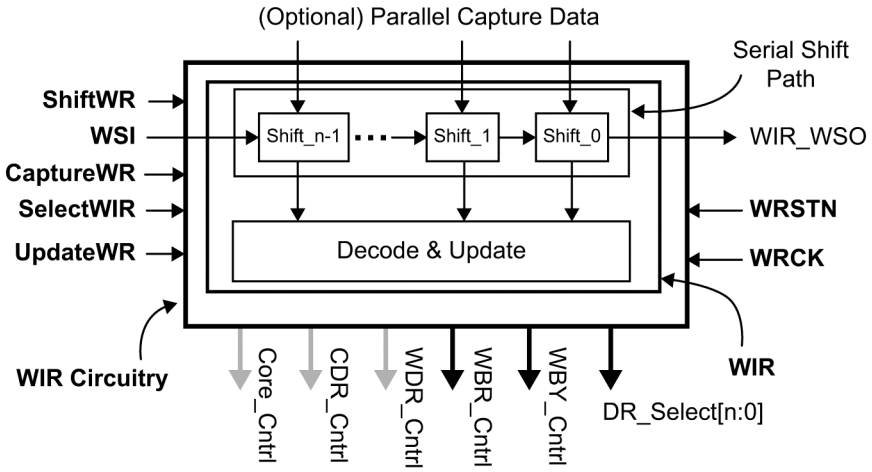


Figure 14—WIR circuitry design

Figure 14 shows an example of the interface signals generated by the WIR circuitry. This interface circuitry shall be designed to provide the necessary control signals for standard data register selection and operation and to control the standard wrapper modes. The WIR circuitry can be optionally extended to generate controls for the selection and operation of other data registers, for the enabling of a WPP, or for any other core modes that might be needed.

Control signals generated by the WIR circuitry are derived from the current wrapper instruction and the states of the signals connected to the WSC terminals. For the standard IEEE 1500 data registers (i.e., the WBR and the WBY), signals generated by the WIR circuitry shall include controls for connecting the WBR or WBY between the WSI and WSO(R) terminals. For the WBY, the WIR circuitry shall generate controls for its Shift event, e.g., a ShiftWBY shift enable signal. For standard serial instructions, the WIR circuitry should also generate control signals for operation of the WBR and its wrapper modes. For example, the WIR would generate wrapper modes for multiplexing the WBR cells between the test and normal modes of the wrapper.

Figure 14 shows a block diagram of the WIR circuitry. The WIR is controlled and clocked by the standard WSC terminals, as shown in the figure. Circuitry in the WIR shall generate the necessary controls for its operation from the signals connected to the standard WSC terminals. For example, a ShiftWIR enable signal can be derived from the signals connected to the SelectWIR and ShiftWR terminal. The specific circuitry required for WIR operation depends on the design implementation of the WIR and is not shown in Figure 14.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

As illustrated in Figure 14, the WIR circuitry includes an *n*-bit serial shift stage, logic for (optionally) decoding the shift stage contents, and circuitry for updating the wrapper instruction that was loaded into the WIR. To accommodate the mandatory wrapper instructions, the WIR serial shift stage shall be at least 2 bits long. The wrapper instruction opcode is serially shifted into the *n*-bit shift stage and can be encoded, partially encoded, or unencoded. Thus, logic for decoding the wrapper instruction is optional, depending on implementation. The update stage latches the wrapper instruction that was loaded into the WIR to control that the current (i.e., previously updated) instruction does not change during subsequent WIR Shift operations.

The WIR can optionally include parallel capture inputs, as shown in Figure 14. This permits the WIR to capture test control information or to capture data that can be used for testing the WIR, or other IEEE 1500 circuitry. Further specification and description of the WIR design are given in 10.3.2 and 10.3.3.

10.3.2 Specifications

Rules

a) The WIR shall provide signals for selecting the WBR or WBY between WSI and WSO(R) as required by the wrapper instructions defined in this standard.

b) The WIR shall provide signals to enable and control the operational modes of the WBR, as defined by this standard.

c) The WIR shall be dedicated IEEE 1500 logic.

d) The shift path through the WIR shall include at least 2 bits.

e) There shall be no inversion of logic values from WSI to WSO(R) for WIR Shift operations.

f) The WIR shall be designed so that data shifted into the WIR shift register do not affect the currently active wrapper and core modes until a WIR Update operation occurs.

g) Where optional parallel inputs are provided to the WIR, the data at the parallel inputs shall be captured into the WIR shift register upon execution of a WIR Capture operation.

h) Where cores are provided with test mode enable input terminals, these terminals shall be directly controlled by the WIR circuitry.

Recommendations

i) When parallel capture inputs are provided to the WIR, any shift register stages that are unused for capture should be designed to capture a fixed binary logic value (i.e., either a 0 or a 1) when a WIR Capture operation occurs. The fixed binary code captured can be determined by the designer.

j) Operation of the WRSTN to reset the wrapper should load fixed logic values into the WIR shift path.

Permissions

k) Modes of user-defined WDRs may be enabled and controlled by the current instruction in the WIR.

l) Core system operation modes may be enabled and controlled by the current instruction in the WIR.

m) Signals applied to the terminals of the WSC may be used to directly control and/or clock WDRs, CDRs, or core test functions without going through the WIR.

n) The WIR may capture data values during WIR Capture operations.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

o) Control signals provided by the WIR may be output to IEEE 1500 wrapper terminals in order to facilitate external circuitry that may be required for PnP operation of the WBR or other data registers.

10.3.3 Description

The WIR circuitry as shown in Figure 13 and Figure 14 is loaded and updated with a wrapper instruction via the WSP terminals. Based on the protocol applied to the signals connected to the WSC terminals, the WIR circuitry generates the necessary control signals for shifting and updating wrapper instructions or the data register specified by the current wrapper instruction. The WIR circuitry also generates the necessary signals for controlling any wrapper and core modes associated with the instruction.

Interface signals generated by the WIR circuitry for the standard IEEE 1500 instructions shall include controls for the following:

— Selection of which data register is to be connected between WSI and WSO(R). This includes controls to select between the WBY and the WBR.

— Control signals for the WBY Shift event and the optional WBY Capture event.

— Control signals for the wrapper modes of the WBR cells.

The WIR circuitry can optionally generate other control signals, such as signals to configure core test modes, controls to select and enable CDRs and/or WDRs, or controls to select and enable the optional WPP and its associated data registers.

Control signals generated by the WIR are static control signals, i.e., signals that change relatively infrequently. Depending on the implementation of the WBR or other data registers, WSC signals might interface directly to the data registers or the core. For example, the WRCK can directly clock the WBR cells. The state of the control signals output from the WIR circuitry, as shown in Figure 14, is generated based on the state of the WSC terminals and the currently active wrapper instruction in the WIR.

As an example, consider the standard WS_EXTEST instruction. When this instruction is loaded into the WIR, the WBR shall be accessed between the WSI and WSO(R) terminals of the wrapper, and the WBR Cells are put into their OF wrapper mode. Similarly, when the WS_BYPASS instruction is loaded in the WIR, the WBY is to be accessed between WSI and WSO(R), and the WBR shall be put into a state so that it allows normal operation of the core. Figure 14 shows a set of control signals called DR_Select[n:0], as being generated by the WIR circuitry. These controls are then connected to the DR_WSO multiplexer. In this example of WIR implementation, the DR_Select[n:0] controls are used for selecting the data register to be connected between WSI and WSO(R). The DR_Select[n:0] signals select among the WBY, the WBR, and any number of other optional registers, per the currently active wrapper instruction. The scan output of the selected data register is then output onto the DR_WSO output of the multiplexer, and the DR_WSO multiplexer output would then be selected at the wrapper's WSO(R) multiplexer by the SelectWIR signal. The WIR circuitry shall also generate signals to control WBY operations, e.g., a ShiftWBY signal to enable shifting the WBY.

Figure 14 shows optional parallel inputs to the WIR. These inputs can be provided to permit data to be captured into the WIR. These capture data can be used for test control, testing of the WIR circuitry, or testing of other IEEE 1500 circuitry.

When optional parallel capture inputs are provided to the WIR, it is recommended that any unused capture inputs be designed to capture fixed binary logic values. This is intended to prevent the capture of unknown data into any unused bits, which might then need to be masked when examining the captured data that were scanned out of the WIR. In addition, the designer can consider choosing fixed logic values that will aid in testing and diagnosis of the WIR logic and SoC scan path integrity. However, care should be taken in the values that are captured into the WIR, and the operation of the WIR, so that unknown or harmful instructions

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

are not inadvertently updated in the WIR. This can be avoided after a WIR Capture operation, by first shifting in a known instruction, prior to performing a WIR Update operation.

10.4 WIR operation

10.4.1 Introduction

Operation of the WIR is controlled by the signals connected to the WSC terminals of the IEEE 1500 wrapper. Figure 15 shows a timing diagram of WIR Shift operation followed by a WIR Update operation. Standard WIR operations are specified and described in 10.4.2 and 10.4.3.

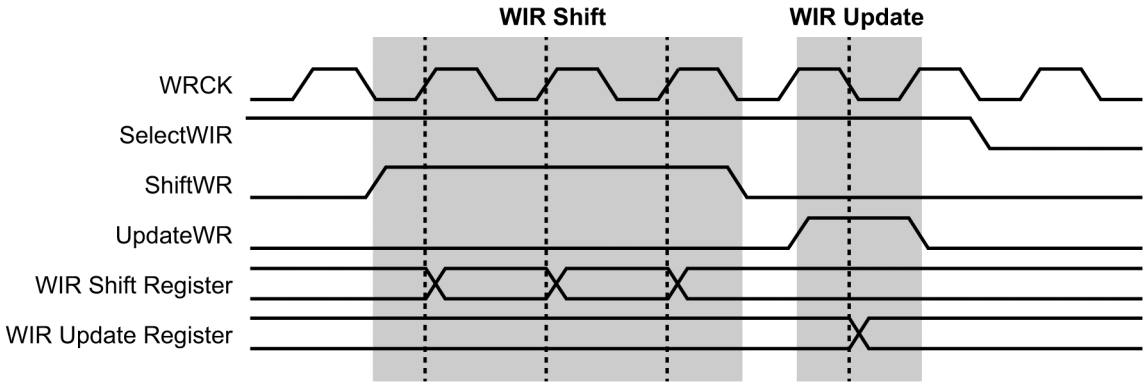


Figure 15—Timing for WIR Shift operation followed by WIR Update operation

10.4.2 Specifications

Rules

a) The behavior and operation of the WIR shall be defined by WSP protocol applied to the signals connected to the WSC terminals, as follows:

   1) When a WRSTN occurs, the WIR update register shall be set to WS_BYPASS, and the WBR shall be set to normal mode.

   2) When a WIR Shift operation occurs, the WIR shift register shall shift instruction data from WSI to WSO(R).

   3) When a WIR Update operation occurs, the wrapper instruction shall be updated from the WIR shift register into the WIR update register, and updated wrapper and core modes shall be set.

   4) When the optional WIR Capture operation occurs, the WIR shift register shall be loaded with parallel capture data.

   5) During all other WSP operations, the WIR shift and update registers shall retain their current state or instruction.

b) The WIR and its associated circuitry shall be clocked and controlled only with the dedicated WSC signals, as defined by this standard.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

c) A single WSP protocol (i.e., WRSTN, WIR shift, WIR update, or WIR capture) shall be applied, as defined by this standard, to operate the WIR so that only one WIR operation is enabled for the given protocol.

d) The current wrapper modes and core modes shall remain active until either a WRSTN or WIR Update operation occurs, which shall cause the specified wrapper instruction and modes to take effect and become the currently active instruction and modes.

NOTE—The previous instruction and modes shall become inactive subsequent to the new Instruction and Modes becoming active.

e) The WIR circuitry shall retain its current state (i.e., shift stage values and currently active modes) indefinitely while the WRCK signal is stopped (i.e., WRCK held at a fixed logic value of 1 or 0) and the signal connected to the WRSTN terminal is logic 1.

f) The WIR circuitry shall retain its current state while there are no active WIR operations in progress.

g) A WRSTN shall occur when the signal connected to the WRSTN terminal transitions to logic 0 and shall cause the wrapper modes and core modes to be set to normal system mode.

h) When the signals connected to the WSC's SelectWIR and ShiftWR terminals are logic 1, a WIR Shift operation shall occur on the next rising edge of WRCK and shall cause instruction data to shift through the WIR, from WSI to WSO(R). The shift stage data shall shift 1 bit toward WSO(R) for each rising edge of WRCK of the WIR Shift operation. SelectWIR and ShiftWR shall remain logic 1 during the WIR Shift operation.

i) During a WIR Shift operation, WSI shall be sampled and WSO(R) shall change on the specified WRCK edge(s) per the rules and permissions for WSP timing given in Clause 14.

j) When the signals connected to the WSC's SelectWIR and UpdateWR terminal are logic 1, a WIR Update operation shall occur on the next falling edge of WRCK and shall cause the wrapper modes and core modes, as determined by the wrapper instruction in the WIR serial shift stage, to take effect.

k) When the signals connected to the WSC's SelectWIR and CaptureWR terminals are logic 1, a WIR Capture operation, if implemented, shall occur on the next rising edge of WRCK and shall cause the data at the parallel inputs of the WIR to be latched into the serial shift stage of the WIR.

NOTE—For all WIR operations, the appropriate WSC terminals shall be at valid logic levels prior to the corresponding clock edge of WRCK specified for the operation, and they shall meet the setup and hold timing specifications of the design. Per rule (c), the signals connected to the WSC terminals that are not specified in the protocol shall be de-asserted in the given protocol.

Recommendations

l) WRSTN should not be used to initialize any other system logic within the core.

m) When a WIR Capture operation has been performed, a WIR Shift operation should be performed prior to a WIR Update operation.

NOTE—This will help prevent an unintended wrapper instruction (i.e., based on the WIR's capture data) from being updated in the WIR. Either a WIR Shift operation should precede a WIR Update operation, or the designer should implement the WIR so that unintended wrapper instructions are not updated in the WIR.

n) When a WRSTN occurs, the WIR shift register should be set to WS_BYPASS.

Permissions

o) The sequences and number of WIR operations may occur in any order.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

10.4.3 Description

Protocol applied to the WSP terminals controls three mandatory operations of the WIR (i.e., WRSTN, WIR Shift, and WIR Update) and an optional operation (i.e., WIR Capture). For all other data register operations, the WIR is required to retain the wrapper instruction last updated in the WIR. This causes the wrapper modes or core modes controlled by the instruction to hold their value until a new wrapper instruction is updated in the WIR. Further, when the WRCK is stopped, the WIR shall retain its current state; thus the WRCK is not required to be free-running.

The WIR circuitry outputs WS_BYPASS when the WSC's WRSTN terminal transitions to logic 0. This causes the mode of the core to be set to normal system operation.

A wrapper instruction is loaded into the WIR by asserting the signals connected to the SelectWIR and ShiftWR terminals to logic 1 and then shifting in the instruction opcode on the WSI terminal with WRCK. The WIR will shift 1 bit of data toward WSO(R) for each rising edge of WRCK. The WSI data shall be sampled and WSO(R) shall change according to the rules and permissions for WSP timing given in Clause 14. Once the proper number of wrapper instruction bits has been shifted into the serial shift stage of the WIR, the ShiftWR signal should be deasserted before the next rising edge of WRCK. Following this, the signal connected to the UpdateWR terminal is asserted to logic 1, and the WIR will then update on the next falling edge of WRCK. A timing diagram with a WIR Shift operation, followed by a WIR Update operation, is shown in Figure 15. This falling-edge behavior was adopted in order to prevent race conditions between shift and update and as an alternative to additional protocol restrictions. This falling-edge behavior also improves PnP.

When a wrapper instruction is updated in the WIR, the corresponding data register scan path is enabled to be selected, and the appropriate wrapper modes and core modes are set. Once the new instructions and modes are active, deasserting the SelectWIR signal selects operation of the data register scan path specified by the wrapper instruction.

11. Wrapper bypass register (WBY)

11.1 WBY register configuration and selection

11.1.1 Introduction

This clause specifies and describes the WBY.

Figure 13 illustrates how the WBY fits into the IEEE 1500 WR configuration and is connected between the WSI and WSO(R) of the WSP. The WBY is the WDR selected when the WIR contains the WS_BYPASS instruction. Once selected, the WBY is connected between the WSI and WSO(R) terminals ofthe WSP and can be shifted using protocol applied to the signals of the WSC terminals.

11.1.2 Specifications

Rules

a) The IEEE 1500 wrapper shall contain a single WBY.

b) The WBY shall be selected for access between WSI and WSO(R) when the currently active wrapper instruction is WS_BYPASS and the signal connected to the WSC's SelectWIR terminal is logic 0.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

c) The WBY shall be selected for access between WSI and WSO(R) when the currently active wrapper instruction is WS_CLAMP or WS_SAFE, if support for those instructions is provided in the wrapper.

Recommendations

d) The WBY should be selected for all unused wrapper instruction opcodes or for when a wrapper instruction does not require a WDR for its operation.

Permissions

e) The WBY may be selected by wrapper instructions other than the WS_BYPASS instruction.

11.1.3 Description

The WBY is a mandatory WDR and shall be selected by the WS_BYPASS instruction. The WBY can also be selected by other wrapper instructions and should be considered as the default data register (i.e., for unused wrapper instruction opcodes or for when the instruction does not require a specific data register) selected between the WSI and WSO(R) terminals of the WSP. This will help ensure that there is a default WDR connected between the WSI and WSO(R) terminals so that the scan path for the WSP is never undefined. Once selected, the WBY can be utilized when the signal connected to the WSC's SelectWIR terminal is deasserted to logic 0.

11.2 WBY design

11.2.1 Introduction

A block diagram for the design of the WBY is shown in Figure 16. The WBY is an *n*-bit serial shift register. It can optionally capture parallel data into its shift register stage and does not have an update stage. Further specification and description of the WBY design are given in 11.2.2 and 11.2.3.

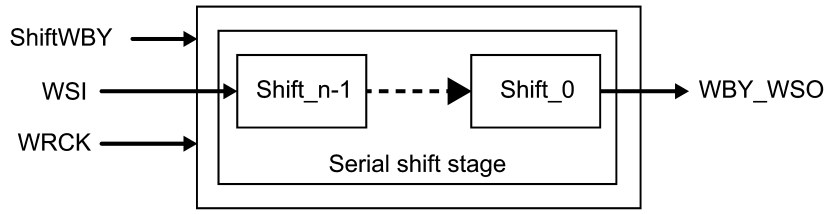


Figure 16—WBY

11.2.2 Specifications

Rules

a) The circuitry that forms the WBY shall be dedicated IEEE 1500 logic (i.e., the circuitry shall not be shared with, or used to perform, system functions).

b) The WBY shall include at least 1 bit of serial shift data.

c) There shall be no inversion of logic values between WSI and WSO(R) for WBY Shift events.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Recommendations

d) The serial shift stage of the WBY should be kept to a minimal length in order to provide the shortest possible bypass path through the WSI-WSO(R) terminals of the WSP. A 1-bit WBY is the preferred length.

Permissions

e) Parallel input(s) to the WBY that permit capture of data value(s) during WBY Capture operations may be provided.

11.2.3 Description

The WBY provides an *n*-bit bypass of the WSP. In most IEEE 1500 wrapper implementations, the WBY should be a single bit in length; however, it is permitted to be multiple bits in length to facilitate connection of the WSP at the system chip level (see Clause 15). In the case where more than a single bit WBY is implemented, it is recommended that the WBY length be kept as short as possible.

The WBY interfaces to the WSC terminals and to the controls generated by the WIR circuitry. Particular control signals for the WBY circuitry will depend on the actual implementation of the WBY.

11.3 WBY operation

11.3.1 Introduction

Operation of the WBY register involves first loading a wrapper instruction to select the WBY and then applying the proper protocol to the WSC terminals to shift the WBY or optionally capture parallel data into its shift stage. Detailed specifications and description on operation of the WBY are given in 11.3.2 and 11.3.3.

11.3.2 Specifications

Rules

a) The operation of the WBY shall be defined by protocol applied to the dedicated WSP signals (e.g., WRCK, ShiftWR).

b) The WBY shall retain its current shift stage value indefinitely while the WRCK signal is stopped (i.e., WRCK held at a fixed logic value of 1 or 0).

c) The WBY shall retain its current shift stage value while there are no active WBY operations in progress.

d) While the WBY is selected and when the signals connected to the WSC's SelectWIR and ShiftWR terminals are logic 0 and 1, respectively, a WBY Shift event shall occur on the next rising edge of WRCK.

e) A WBY Shift event shall cause data to shift through the WBY, from WSI to WSO(R), and the shift stage data shall shift 1 bit toward WSO(R) for each rising edge of WRCK.

f) During a WBY Shift event, WSI shall be sampled and WSO(R) shall change on the specified WRCK edge(s) per the rules and permissions for WSP timing given in Clause 14.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

NOTE—For all WBY operations, the appropriate WSP terminals shall be at valid logic levels prior to the corresponding clock edge of WRCK specified for the operation, and they shall meet the setup and hold timing specifications of the design.

11.3.3 Description

The operation of the WBY is controlled and clocked with protocols applied to the WSP signals. The WBY interfaces to the WIR circuitry, where clock and control inputs to the WBY from the WIR will depend on the actual WBY implementation. For example, in the WBY register illustrated in Figure 16, a ShiftWBY control input is generated by the WIR circuitry and the WBY receives the WRCK and WSI signals directly from the WSP. The WBY-WSO output of the WBY is then connected to the DR_WSO multiplexer in Figure 13 and will be output on the DR_WSO signal when the WBY is selected as the data register.

12. Wrapper boundary register (WBR)

12.1 Introduction

IEEE Std 1500 mandates a serial interface and supports a parallel interface for accessing a wrapped core. The serial interface is characterized by a single chain composed of all WBR cells, whereas the parallel interface is characterized by one or multiple IEEE 1500 wrapper cell chains surrounding a core. Figure 17 shows the WBR in an IEEE 1500 serial interface architecture. The WRSTN shown in this figure can optionally be used to reset the WBR cells to a known state.

The WBR is constructed of WBR cells, each of which [except for those exempted by permission 12.3.2(g)] has four data terminals: cell functional input (CFI), cell functional output (CFO), cell test input (CTI), and cell test output (CTO).

Figure 17 shows the following:

a) Wrapper functional inputs (WFIs) are connected to the CFI terminals of WBR cells provided for core input terminals.

b) CFO terminals of WBR cells provided for core input terminals are connected to the corresponding core input terminals.

c) Core output terminals are connected to the CFI terminals of corresponding WBR cells provided for those core output terminals.

d) The CFO terminals of cells provided for core output terminals are connected to corresponding WFOs.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

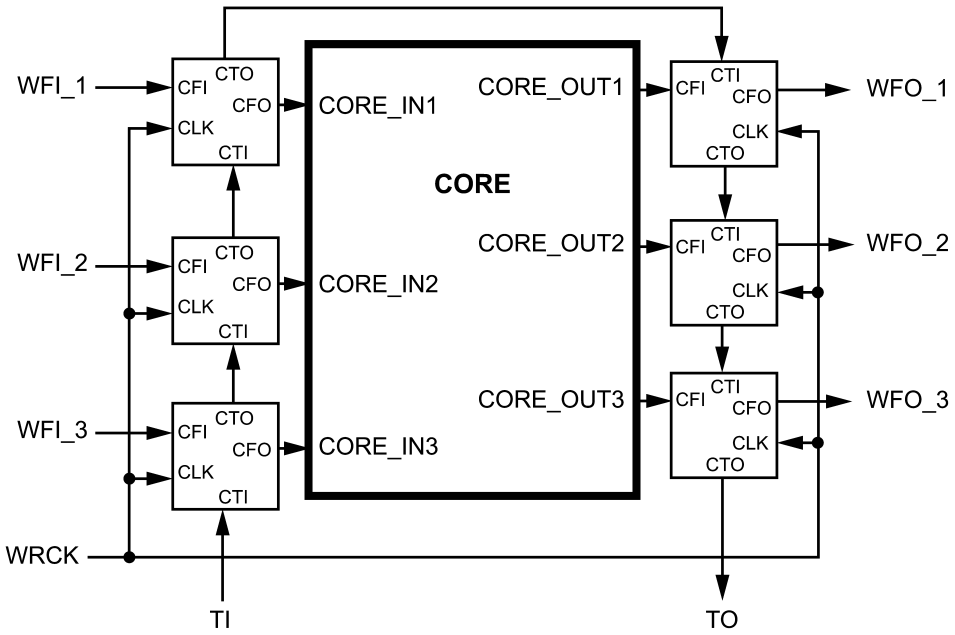


Figure 17 —Example of WBR in serial interface configuration to be accessed from WSP

WFI and WFO are generic names for wrapped core terminals that are intended to be connected within an SoC to integrate the wrapped core within the SoC. It is often the case that identical WBR cells are used for wrapping both core input terminals and core output terminals. These WBR cells are connected differently and might receive different control signals (not shown). The WBR shift path is formed by connecting WBR cells CTO to CTI. TI and TO are defined to be the scan input and scan output terminals of the WBR as a whole.

In Figure 17 and Figure 18, the WFI_k terminals are WFI terminals, while the WFO_k terminals are WFO terminals. In the case where an AUXCK is used to operate the WBR, Figure 55 shows an example timing relationship between WRCK and the AUXCK.

In Figure 18, a parallel interface implementation using two scan chains is depicted. Although this figure shows all inputs placed on one scan chain and all outputs on a second scan chain, the IEEE 1500 parallel interface is not limited to this configuration. In addition, although WRCK is shown in both Figure 17 and Figure 18, IEEE Std 1500 does not mandate the use of WRCK for clocking the wrapper cells. A different clock, i.e., AUXCK, may be used although the WRCK is required at the wrapper level.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

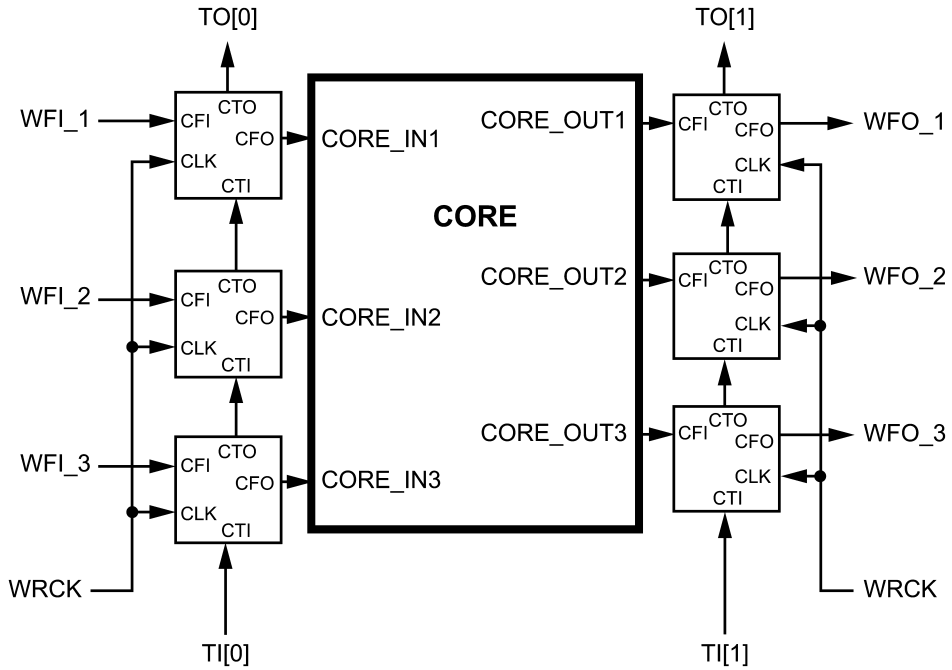


Figure 18—Example of WBR in parallel interface configuration to be accessed from WPP

The following definitions apply to the rules specified in 12.3.2 and in 12.4.2.

Shift event: An event whereby the data stored in the WBR shift path are advanced one storage position closer to the WBR's TO. The data present at the WBR's TI are loaded into the shift path storage element closest to the WBR's TI. If multiple WBR segments are implemented per Figure 18, this definition applies individually to each WBR segment.

Capture event: An event whereby the value present on the CFI, or on the CFO, at a characteristic instant is stored in a sequential element within a WBR cell.

Update event: An optional event whereby data stored in a WBR cell's shift path storage element closest to CTO at a characteristic instant are loaded into an off-shift-path storage element.

Apply event: A derivative event (in the sense that it is inferred from the operation of the other three events: Shift, Update, and Capture), whereby test data become active and effective as test stimuli. While the wrapper is in IF mode, the Apply event causes test data to be applied from input cells onto core inputs. While the wrapper is in OF mode, the Apply event causes test data to be applied from output cells onto WBR functional outputs. The test data are the data stored in the shift path storage element closest to CTO unless the Update event is supported, in which case the test data shall be the data stored in the off-shift-path storage element described in the Update event. The Apply event is only meaningful in the context of the WBR as a whole and not on a cell-by-cell basis.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

12.2 WBR structure and operation

12.2.1 Specifications

Rules

a) Every core signal terminal that is digital, except for signals that cause data to be loaded into a sequential element (e.g., the clock of a flip-flop, the gate of a latch, or the asynchronous set or reset of either a flip-flop or a latch) or dedicated test signal, shall be provisioned with a WBR cell.

b) The WBR shall have at least one configuration in response to the state of the WIR, and any SIBs, allowing serial access to and from all WBR cells between TI and TO.

c) Every wrapped terminal shall be associated with at least one WBR cell.

d) The WBR shall respond to shift, capture, and apply as defined in 12.4.2, except as exempted under permission 12.3.2(g).

e) Core terminals that have been provided for the purpose of statically enabling one or more core test modes shall be provisioned with a WIR control.

Recommendations

f) Core terminals exempted from wrapper cell insertion per rule 12.2.1(a) should be provided with direct access from the SoC to their corresponding wrapped core terminals, except where doing so would conflict with rule 12.2.1(e).

Permissions

g) Any core terminal may be provisioned with a WBR cell, provided doing so does not alter the behavior of standard instructions.

h) In a case in which a core input terminal is used solely as the source of data for an output terminal, a single WBR cell may be shared between the input and the output terminals.

i) A core input terminal may be provisioned with more than one WBR cell.

j) There may be combinational logic between the core terminal and the WBR cell.

12.2.2 Description

For the IEEE 1500 serial interface, the WBR should have a single, uniform shift path between its TI and TO. WBR structure rules 12.2.1(a) and 12.3.2(b) support this. A shift path in a WBR cell is composed of one or more storage elements serially connected between CTI and CTO. Likewise, for the serial mode of the IEEE 1500 wrapper, a shift path is the same storage elements of all the cells in the WBR concatenated into a single serial path.

In the case where core terminals are exempted from wrapper cell insertion, the wrapped core patterns will likely presume that a direct access is provided from the SoC to these terminals except where prohibited by rule 12.2.1(e) since those terminals are controlled by the WIR. The SoC integrator has to take this into account for proper operation at the SoC level.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

12.3 WBR cell structure and operation

12.3.1 Introduction

The WBR cell operation relies on the Shift, Capture, Update, and Apply events more fully defined in 12.4.2.

12.3.2 Specifications

Rules

a) Every WBR cell shall have at least one storage element connected between its CTI and CTO terminals.

b) Every WBR cell required per rule 12.2.1(a) shall have at least one storage element connected between its CFI and CFO terminals.

c) Every WBR cell required per rule 12.2.1(a) shall have a storage element provisioned for the purpose of servicing the Capture event and this element shall be in the shift path.

d) Every WBR cell required per rule 12.2.1(a) shall have a storage element provisioned for the purpose of servicing the Apply event.

e) Storage elements in the shift path of a WBR cell shall not respond to the Update event.

Permissions

f) Any WBR cell may have a storage element provisioned for servicing the Update event.

g) For core terminals exempted from being wrapped per rule 12.2.1(a), reduced-functionality WBR cells may be provisioned, provided that they support the Shift event and at least one of the following:

   1) The Capture event to observe the CFI terminal or the CFO terminal

   2) The Apply event to control the CFO terminal

12.3.3 Description

WBR cell structure rules 12.2.1(a) and 12.3.2(b) provide for a minimal implementation consisting of only a means to select test data or functional data as input to a shared storage element.

WBR cell structure rule 12.2.1(a) allows for multiple storage elements in the shift path to support test methodologies requiring the application of sequential patterns (e.g., path-delay, transition delay, piece-wise functional).

WBR cell structure rule 12.3.2(c) requires that captured data enter the shift path of a cell at a storage element.

Unidirectional IEEE 1149.1 boundary-scan cells are usable as IEEE 1500 WBR cells. WBR cell structure permission 12.3.2(f) supports this.

Cells provisioned on core terminals that need not be wrapped per rule 12.2.1(a) can have reduced-functionality cells as described in permission 12.3.2(g). For instance, one might wish to have an observe-only cell on clocks as defined in rule 12.2.1(a) or other types of observe cells supporting the Shift and

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Capture events in OF mode. If the core terminal provided with a reduced-functionality cell is a clock terminal and gets connected to the clock input of the reduced-functionality cell, then this clock would be considered an AUXCK and would have to follow AUXCK requirements and allow for proper operation of the WBR.

12.4 WBR operation events

12.4.1 Introduction

An event is an uninterrupted, predefined sequence of one or more steps. Within the interval of the predefined sequence, there is a particular instant that characterizes the event when the nominal action of the event occurs. This is referred to as the *characteristic instant*. Predefined events are Shift, Update, Capture, and Apply. Some events may overlap with others as indicated in permission 12.4.2(e). In addition, user-defined events are allowed as indicated in permission 12.4.2(f). Events apply to the WBR as a whole.

12.4.2 Specifications

Rules

a) The Shift event shall not occur simultaneously with either Capture or Update events.

b) While the SelectWIR signal is low and in the absence of an active value on other WSC or any WPC terminal (as defined in the CTL), WRCK shall not cause a WBR storage element to change state.

c) For serial instructions, in the absence of an active edge on WRCK or pulse on WRSTN, WBR storage elements shall not change state.

Recommendations

d) While the SelectWIR signal is high and in the presence of active values on WSC terminals, with the exception of UpdateWR, WRCK should not cause a WBR storage element to change state.

Permissions

e) Except where excluded by rule (a), events may be discrete, simultaneous, or overlapping.

f) WBR cells may respond to user-defined events, provided that these events are documented and distinguished in their behavior from standard IEEE 1500 events.

12.4.3 Description

WBR events are enabled by the active edge of the clock (rising edge of WRCK for Capture and Shift; falling edge of WRCK for Update) in conjunction with the assertion of a WSC signal (CaptureWR, ShiftWR, or UpdateWR). WRCK pulses or edges applied in the absence of active WSC signals do not result in any event. WRCK can be held in a high or low state for arbitrarily long intervals without loss of WBR state. PnP requirements can restrict the degree to which events might be overlapped. Subclause 16.4 discusses such restriction. Recommendation 12.4.2(d) allows the WBR to maintain its state for subsequent instructions that might require the current WBR data.

There are Shift, Capture, and Update events that are defined and have rules mandating what shall happen during these events. User-defined events are also permitted per 12.4.2(f). Delay testing using the wrapper cells is described in 12.9. The second example in that section, depicted in Figure 39, requires a user-defined event. In this particular example, the user-defined event allows the input wrapper cells to receive the input

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

data from CTI, rather than CFI, while the remainder of the design, including the output wrapper cells, are allowed to capture from their respective CFI inputs.

During delay testing, a data pair is required. When in inward facing mode, the input wrapper cell shown in part a) of Figure 19 receives an X during capture. However, if the input wrapper cell receives data during capture from CTI, rather than CFI, as shown in part b) of Figure 19, it can deliver data pairs to the initial cloud of logic. Of course, the shift paths between the input wrapper cells should be able to be operated at-speed to enable at-speed delay testing of this logic. This user-defined event helps prevent Xs during any test, so it is helpful for testing in general, but especially LBIST testing.

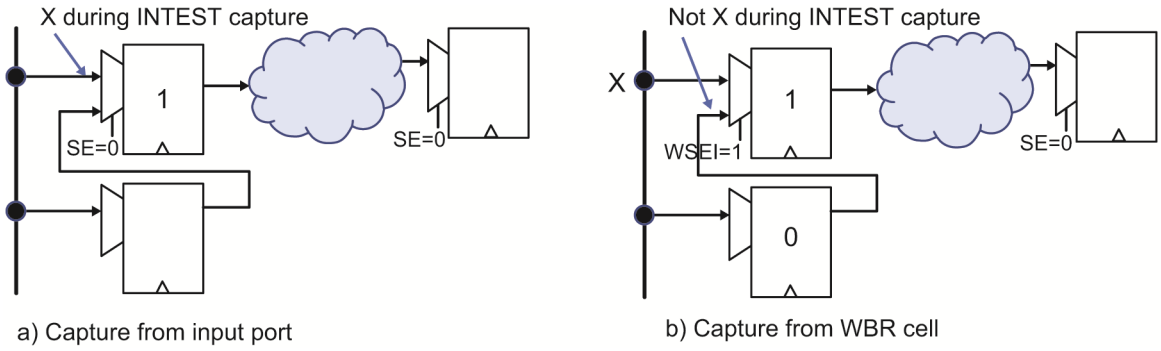


Figure 19—Input wrapper cell control options

12.5 WBR operation modes

12.5.1 Introduction

A mode is a static condition or configuration of the WBR that exists in response to the state of the WIR. This subclause describes the normal mode, the IF mode, the OF mode, and the nonhazardous mode.

General permissions

a) IF mode and OF mode may be operative at the same time.

b) Whereas the four modes defined in this subclause (i.e., normal, IF, OF, and nonhazardous) are applied homogeneously across the entire WBR, other modes may be defined in which the cells respond on an individual basis.

12.5.2 Normal mode

12.5.2.1 Introduction

The normal mode is the mode in which the WBR does not interfere with the functional operation of the core.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

12.5.2.2 Specifications

Rules

a) While in normal mode, the WBR shall not interfere with the operation of the core or with the flow of signals to and from the core.

Permissions

b) While in normal mode, the WBR may respond to the Capture, Shift, or Update events, provided that such response does not conflict with rule (a).

12.5.3 Inward facing (IF) mode

12.5.3.1 Introduction

The IF mode is a test mode where core inputs are controlled by the WBR and core outputs are observed by the WBR.

12.5.3.2 Specifications

Rules

a) While in IF mode, cells provided for core inputs shall respond to Shift, Apply, and, if provisioned for them [under permission 12.3.2(f)] the Update events, respectively.

b) While in IF mode, cells provided for core outputs shall respond to Capture and Shift events.

Recommendations

c) While in IF mode, cells provided for core outputs should present safe data at their CFO terminals (i.e., external safe state).

12.5.4 Outward facing (OF) mode

12.5.4.1 Introduction

The OF mode is a test mode where WFOs are controlled by the WBR and WFIs are observed by the WBR.

12.5.4.2 Specifications

Rules

a) While in OF mode, cells provided for core inputs shall respond to Capture and Shift events.

b) While in OF mode, cells provided for core outputs shall respond to Shift, Apply, and, if provisioned for them [under permission 12.3.2(f)], the Update events, respectively.

Recommendations

c) While in OF mode, cells provided for core inputs should present safe data at their CFO terminals (i.e., internal safe state).

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

12.5.5 Nonhazardous mode

12.5.5.1 Introduction

In nonhazardous mode, core inputs are controlled to safe data by the WBR, and WFOs are controlled to safe data by the WBR.

12.5.5.2 Specifications

Recommendations

a) While in nonhazardous mode, all cells should present safe data at their CFO terminals.

Permissions

b) While in nonhazardous mode, the WBR may respond to any events.

12.5.5.3 Description

Safe data are data that cannot harm circuitry internal or external to the core. These data can be static or dynamic.

12.6 Parallel access to the WBR

12.6.1 Introduction

In addition to its mandatory serial wrapper access mechanism, IEEE Std 1500 provides for an optional parallel wrapper access mechanism. There are two possible ways to implement parallel access of the IEEE 1500 WBR, distinguished by the presence or absence of an IEEE 1500 wrapper cell on every core terminal:

— In the case where every core terminal contains an IEEE 1500 wrapper cell, as shown in Figure 18, parallel access of the WBR is achieved via segmentation of the WBR chain.

— In the case where not every core terminal has a WBR cell, per rule 12.2.1(a), parallel harnessing is implemented, as shown in Figure 20, in order to allow enabling of the parallel test being applied to the core as a response to a parallel instruction in the WIR.

NOTE—Harnessing logic enables coupling of a TAM to CTIs and CTOs. This includes any logic that is needed between the core and a TAM provided that such logic obeys wrapper states rules defined in Clause 13.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

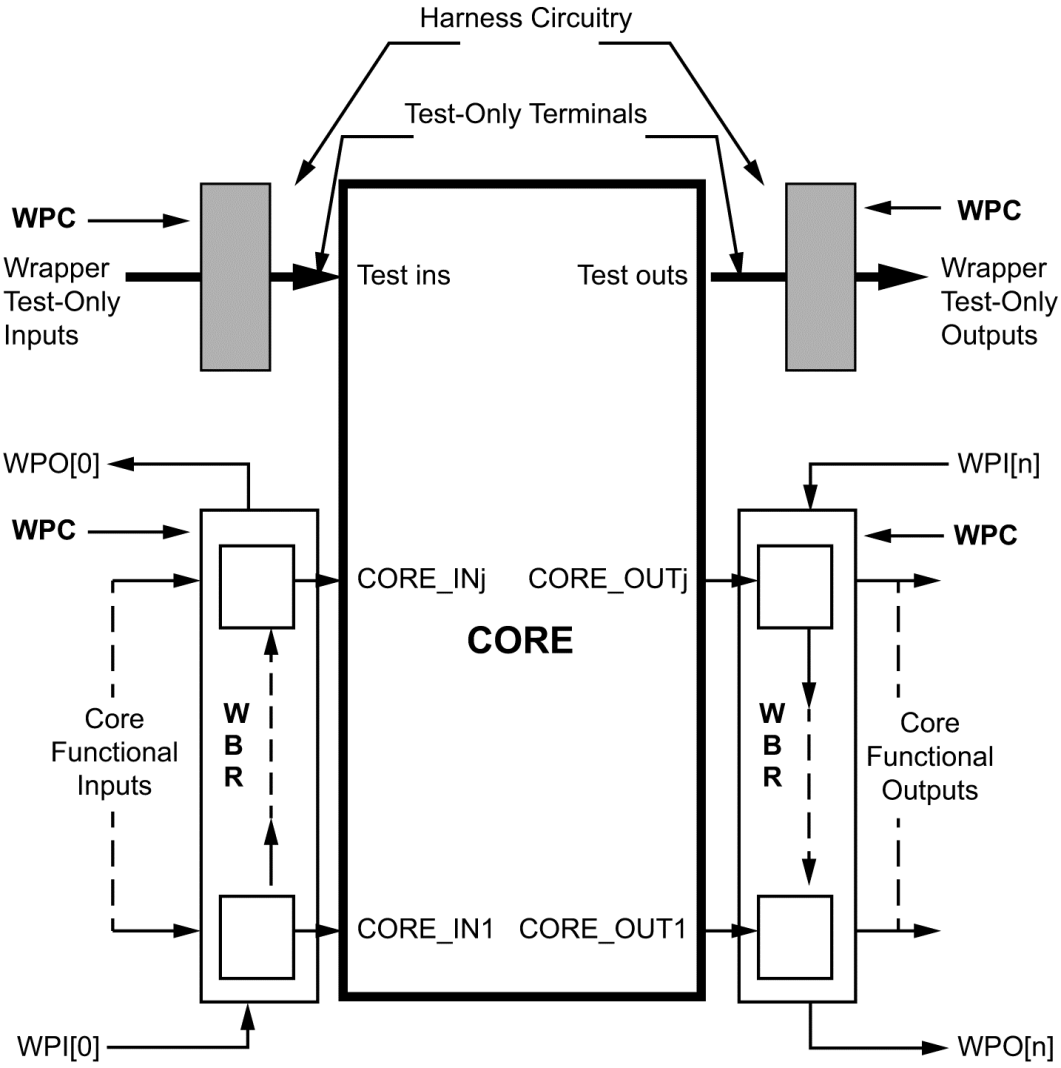


Figure 20—WBR harnessing via test-only terminals

12.6.2 Parallel configuration of the WBR

12.6.2.1 Introduction

This subclause applies to both parallel access types described in 12.6.

12.6.2.2 Specifications

Rules

a) Parallel configuration of the WBR shall be enabled by the WIR, as a response to a parallel or hybrid instruction, e.g., WP_PRELOAD, WH_EXTEST, WP_INTEST_SCAN.

b) WPI and WPO terminals shall be distinct from the mandatory WSP terminals, provided that the parallel interface exists.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

c) For every segmentation of the WBR in every parallel configuration, every WBR cell shall be a member of a unique WBR segment and be provided a path from a WPI and to a WPO.

d) In parallel mode, all WBR segments shall be under the control of the WPC, except when WP_EXTEST is in use [see permission 7.6.2(j)].

Recommendations

e) In cases where a WBR segment drives an internal scan chain, circuitry should be inserted to prevent certain wrapper events, such as Update, from causing data corruption in the internal scan chain.

Permissions

f) The WBR may be segmented into subsets of WBR scan chains accessible via WPI and WPO.

g) When the WBR is configured into parallel mode, the serial input (TI) and serial output (TO) of each segment may be connected to one of the following:

1) Another WBR segment from the same wrapper (to create a larger segment)

2) a TAM via WPI or WPO, respectively

3) a core internal scan chain output or input, respectively

h) There may be more than one parallel configuration of the WBR, allowing different parallel instructions to access different segmentations of the WBR.

12.6.2.3 Description

IEEE Std 1500 allows for a configuration of the WBR as a parallel interface to a core, in order to provide an increase in data bandwidth to the core. This parallel configuration of the WBR allows flexibility for the connectivity of the WBR segments and for how the parallel test applied to the core is controlled. The flexibility in connectivity allows WBR segments to be connected via their WPI and WPO to a TAM interface or to internal scan chains. For standard parallel instructions, nonclock WPC signals that are distinct from WSC signals are used to control the WBR segments during the application of a parallel test. User-defined parallel instructions are not required to use WPC signals that are distinct from WSC signals for controlling the parallel test.

12.6.3 Harnessing of the WBR

12.6.3.1 Introduction

This subclause discusses additional rules and permissions specific to the second type of parallel access described in 12.6, which is the parallel harnessing of the WBR.

12.6.3.2 Specifications

Rules

a) Harnessing logic inserted on test-only terminals shall maintain compliance with Wrapper Disabled state rules described in 13.1.

b) Parallel harnessing shall be configured only in response to an instruction loaded in the WIR.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Permissions

c) Test-only terminals [which are exempted from the requirement for wrapper cell insertion per rule 12.2.1(a)] may be provided with wrapper harnessing logic that, in response to an instruction present in the WIR, couples these terminals to a TAM for the application of parallel tests.

d) In response to nonstandard or hybrid/parallel instructions in the WIR, the configuration of the harnessing logic may be actuated via a TAM signal, a WBR cell, or a WPC signal.

e) Parallel harnessing may be reduced to a direct connection from a TAM to a core input, provided that compliance to Wrapper Disabled state rules (described in 13.1) is maintained via control from the WIR.

f) Parallel harnessing may be configured by a combination of WBR storage elements provided uniquely for this purpose, provided that the configuration is enabled in response to the WIR. Such WBR storage elements need not correspond uniquely (one to one) to the core terminals so harnessed.

12.6.3.3 Description

Rules defined in this subclause complement those defined in 12.6.2 in that harnessing requires compliance to both 12.6.2 and 12.6.3.

Although flexibility is provided to the user in terms of defining the nature of the harnessing logic, the WIR remains the enabler of the parallel mode. In particular, core terminals with direct harnessing (direct wire) to a TAM should not interfere with the Wrapper Disabled state mode of the IEEE 1500 wrapper. Test modes specific to the parallel mode of the wrapper should be enabled only as a response to the WIR. Therefore, appropriate gating logic is expected to be inserted whereby the connection between the TAM and the core terminal is enabled by the WIR as part of a parallel test instruction. WBR storage elements can be used to provide decoding logic for the configuration of parallel harnessing, assuming that this configuration is requested from the WIR.

12.7 WBR cell naming

12.7.1 Introduction

IEEE Std 1500 mandates a naming convention for WBR cells.

12.7.2 Specifications

Rules

a) All IEEE 1500 WBR cell names shall match the following regular expression: /((WC)|(WH))((_S[DF]\d+)|(_CI?))(_C([IOB][IO])\d+|N)?(_U[DF])?(_O)?(_G[01]?)?/.

12.7.3 Description

WBR cell structures are defined in Perl regular expression format to ease software tool automation as well as integration with the CTL. The regular expression defined in 12.7.2(a) allows for the identification of any IEEE 1500 WBR cell by parsing CTL code.

The naming of the various types of WBR cells shall be done in a descriptive, parsable method. Each cell type name shall begin with WC (WBR cell) or WH (WBR harness cell). This prefix is followed by a

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

sequence of characters that describe the capabilities and structure of the cell. The information shall indicate whether a particular storage element is shared or dedicated to wrapper operation, how many shift path storage elements exist, the existence and type of the optional update cell, and the existence of safe data support. In addition, for harness cells, the presence of an inversion shall be noted and also if the cell is combinational or sequential. To support this, from one to five fields should exist in the name in a specified order. Each field begins with an underscore.

— The first field is mandatory and describes the nature and number of shift path storage elements or the combinational and/or inverted nature of a harness cell. The first two characters of this field are _S (shift) or _C (combinational). If _S is selected, the third character is either D (dedicated) or F [(shared) functional], followed by an integer indicating the number of shift path storage elements. If _C is selected, a third character, I, may be used to indicate an inversion in the data path. This first field has the following regular expression format: /(_S[DF]\d+)|(_CI?)/.

— The second field indicates the site where data are captured. The first two characters of this field are _C (capture). The third character of this field specifies the origin of captured data: I (CFI), O (CFO), or B (selectable by design, i.e., can be CFI or CFO), followed by an integer indicating location of the capture site counting from CTI. In cases where the wrapper cell being described does not perform the capture function, both third and fourth characters in this field should be replaced by N (none) to indicate that there is no origin for captured data and that the capture site does not exist. When _C or _CI is selected for the first field, _CN is mandatory for the second field. The regular expression matching this second field is /(_C([IOB][IO])\d+|N)?/.

— The third field describes the nature of the update element. It is composed of three characters, the first two of which are _U (update). The third character is either D (dedicated) or F [(shared) functional]. Its regular expression format is /(_U[DF])?/. The absence of this field indicates the absence of an update storage element.

— The fourth field indicates, by its presence or absence, the presence or absence of an observe-only characteristic. It is composed of an underscore followed by an O. Its regular expression format is /_O/.

— The final field indicates, by its presence or absence, the presence or absence of safe data support in nonhazardous mode. It is composed of an underscore followed by a G (guarded data) and optionally by a 0 or 1 indicating the static value. Its regular expression format is /_G[01]?/.

The following is a generic CTL code describing IEEE 1500 WBR cells:

```
Environment(Env_name) {
   CTL (CTL_name) {
    External {
     (sigref_expr {
       (connectTo {
         Wrapper IEEE1500
         (CellID cell_enum | PinID user_defined);
       })*
      })+
     }
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
    }
}
cell_enum = /((WC)|(WH))((_S[DF]\d+)|(_CI?))(_C([IOB][IO])\d+|N)?(_U[DF])?(_O)?(_G[01]?)?/
```

IEEE 1450.6 (CTL) documentation contains an in-depth description of the above CTL syntax.

12.8 WBR cell examples

Table 2 describes IEEE 1500 WBR cell example names, and a gallery of bubble diagrams depicting each example follows the table. The bubble diagrams used in these examples are defined in Annex A. It is understood that the means of data path selection shown in these bubble diagram figures is to be configured by the content of the WIR.

Table 2—WBR cell example list

Cell description	Name	Figure number
One storage element shared with functional operation.	WC_SF1_CI1	Figure 21
One storage element in shift path dedicated to wrapper function, capturing from CFO.	WC_SD1_CO1	Figure 22
Two dedicated shift path storage elements, capturing from CFI into the shift storage element closest to CTI. This cell captures data into the storage element closest to the scan input.	WC_SD2_CI1	Figure 23
One dedicated storage element in the shift path and a shared update storage element.	WC_SD1_CB1_UF	Figure 24
A reduced-functionality cell with one dedicated storage element in the shift path, performing observe-only function.	WC_SD1_CI1_O	Figure 25

The WC_SF1_CI1 cell types have the least circuitry. As this cell services both normal operation and test operation, the two operations are likely mutually exclusive. Also, the CFO terminal toggles as data are shifted from CTI to CTO and when a capture occurs. WC_SF1_CI1_G can drive a safe value in addition to supporting the functionality described for WC_SF1_CI1. See Figure 21.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

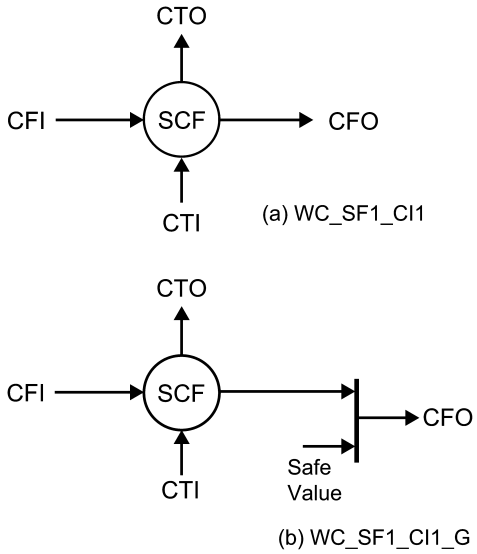


Figure 21—WC_SF1_CI1 WBR cells

WC_SD1_CO1 has a dedicated shift path, and its CFO might toggle during certain events, e.g., Shift or Capture. However, compared to the WC_SD1_CII cell, the CFI to CFO connection has superior testability. See Figure 22.

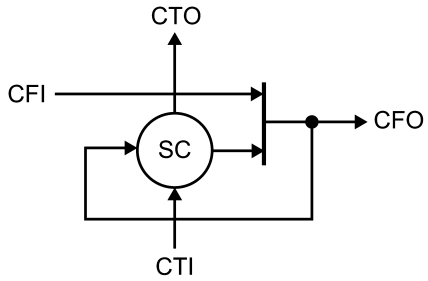


Figure 22—WC_SD1_CO1 WBR cell

The WC_SD2_CI1 cell has the capability to support delay testing applications. Note that this cell has a dedicated shift path and two shift elements. See Figure 23.

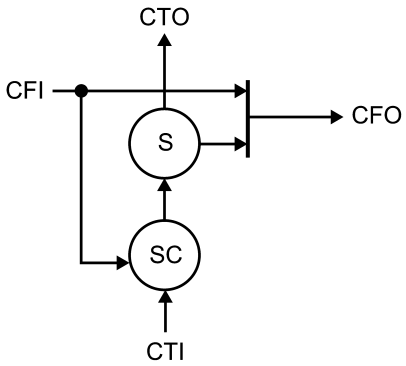


Figure 23—WC_SD2_CI1 WBR cell

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

WC_SD1_CB1_UF has a dedicated shift path. An update storage element is shared with functional operation. It has a selectable capture, from either CFI or CFO, to capture data into the shift path. WC_SD1_CB1_UF_G can be forced to a safe value in addition to supporting the functionality described for WC_SD1_CB1_UF. See Figure 24.

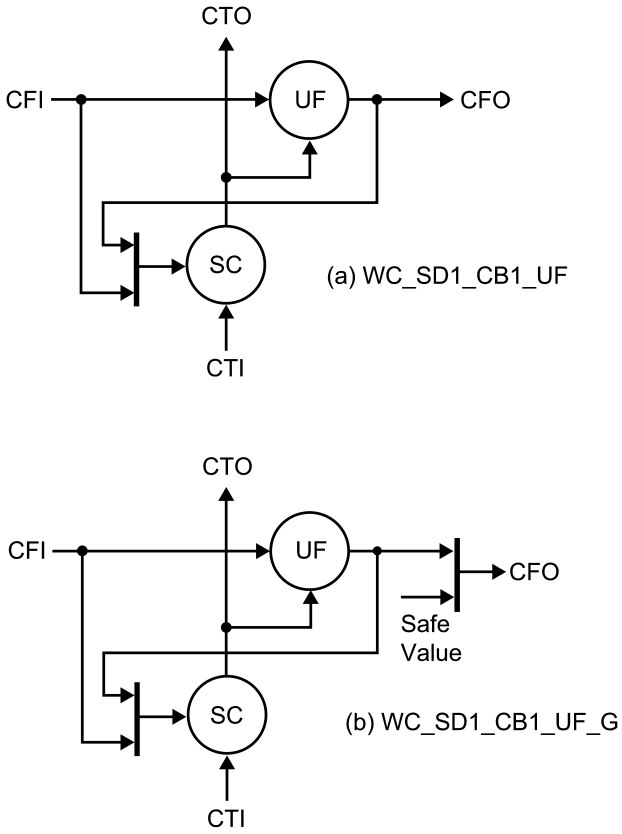


Figure 24—WC_SD1_CB1_UF WBR cell

WC_SD1_CI1_O is a observe-only harnessing cell. This cell is to be used only in accordance with permission 12.3.2(g). See Figure 25.

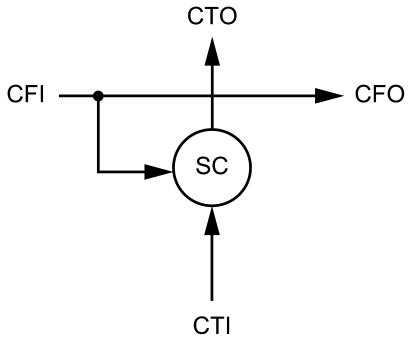


Figure 25—WC_SD1_CI1_O WBR cell

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

12.9 IEEE 1500 WBR example

This subclause describes an example of how the WBR and WBR events can be used to perform a delay test in conjunction with the WS_INTEST_SCAN and WP_INTEST_SCAN instructions. Figure 26 depicts a simple example of core logic prior to being wrapped or any other DFT treatment being applied. The circuit is a four-bit ripple adder that adds registered inputs A[3:0] to internal registers and generates registered output named CO.

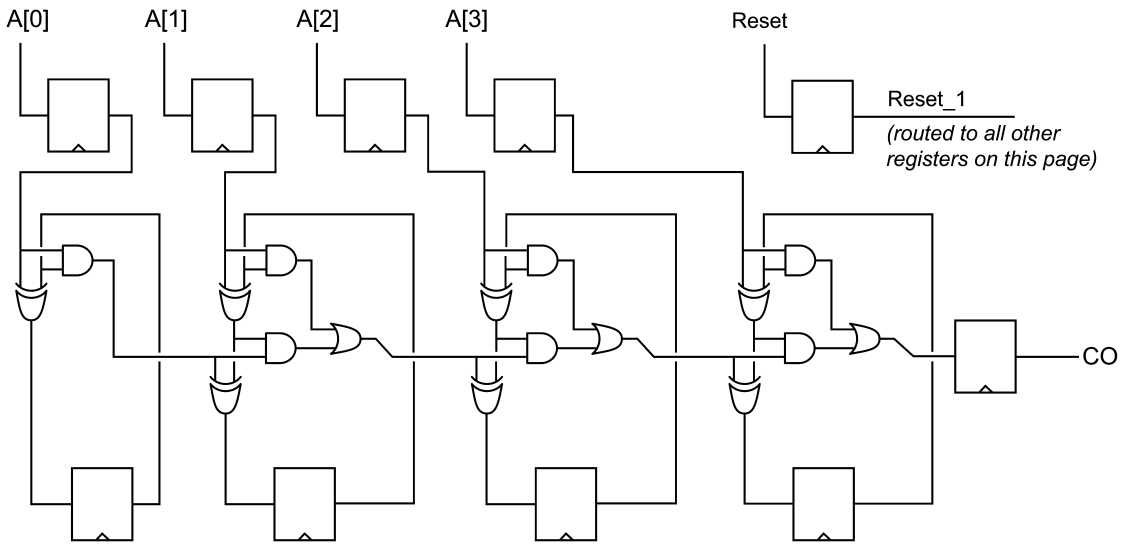


Figure 26—Core logic

The clock for all the flip-flops, which is not shown for the sake of visual simplicity, is called Core_Clk. Core_Clk is a gated version of a high-speed clock called 16xClk. Each of the flip-flops (except for the register for input Reset) receives the signal Reset_1 as a synchronous reset. For this example the registered inputs A[3:0] and Reset and registered output CO are converted to shared WBR cells of type WC_SF2_CI2_G with detailed schematics shown in Figure 27, Figure 28, and Figure 29 respectively and the four internal registers (at the bottom) are conventional scan flip-flops as shown in Figure 30. In Figure 27, Figure 28, and Figure 29, signals shown as Intest and Extest respectively are mode configuration outputs of the WIR decode.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

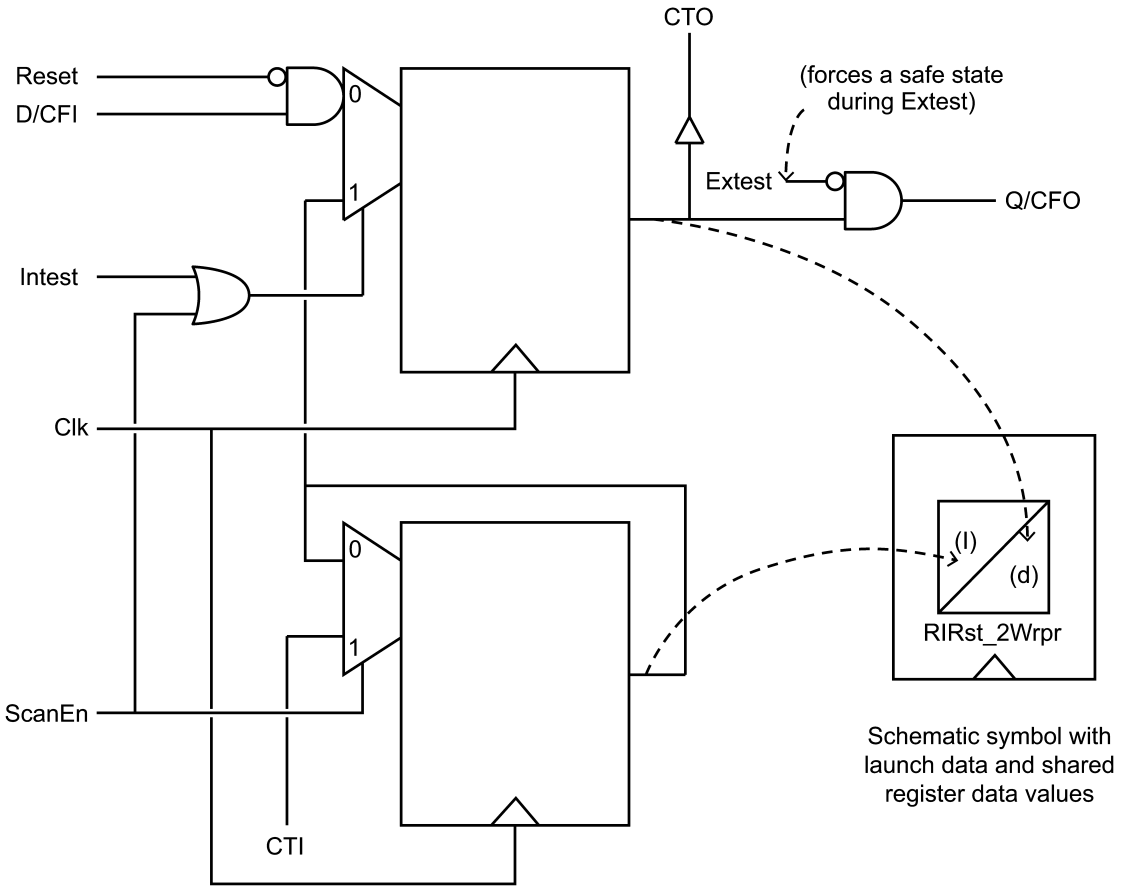


Figure 27—RIRst_2Wrpr

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

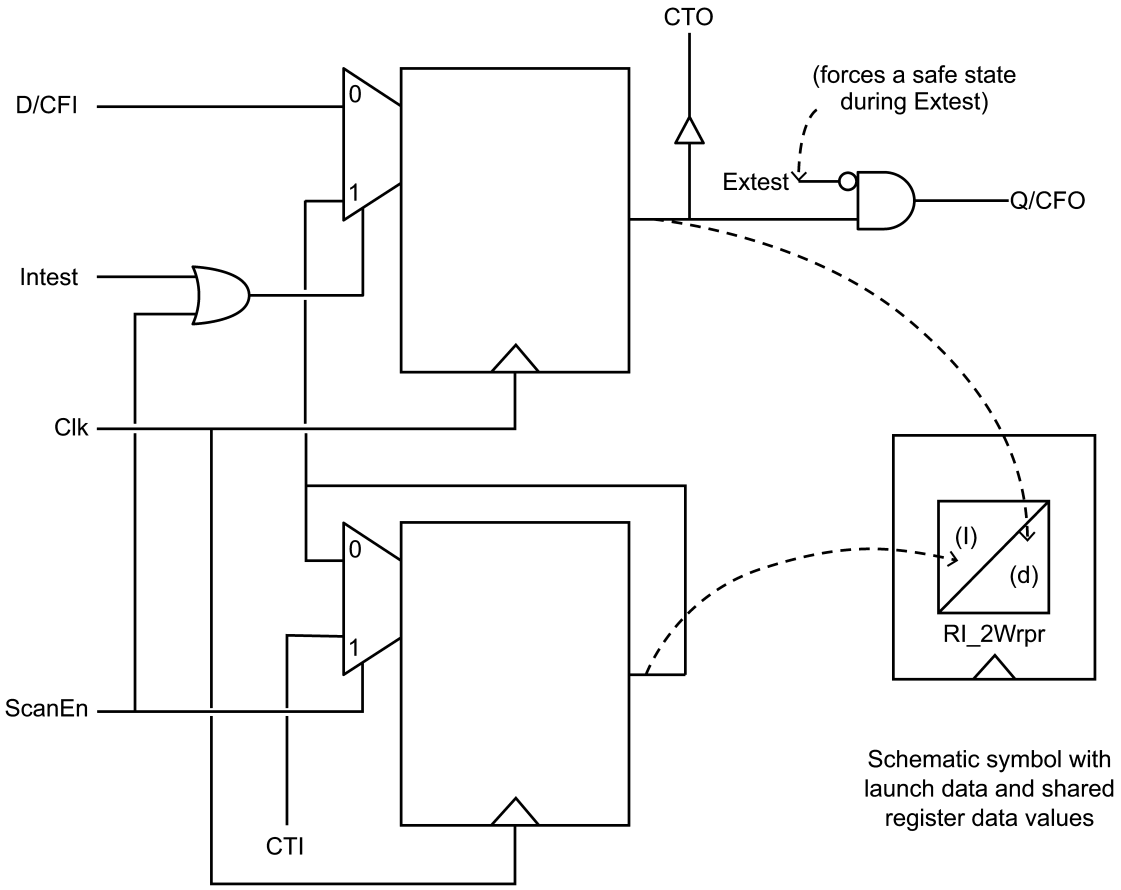


Figure 28 —RI_2Wrpr

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

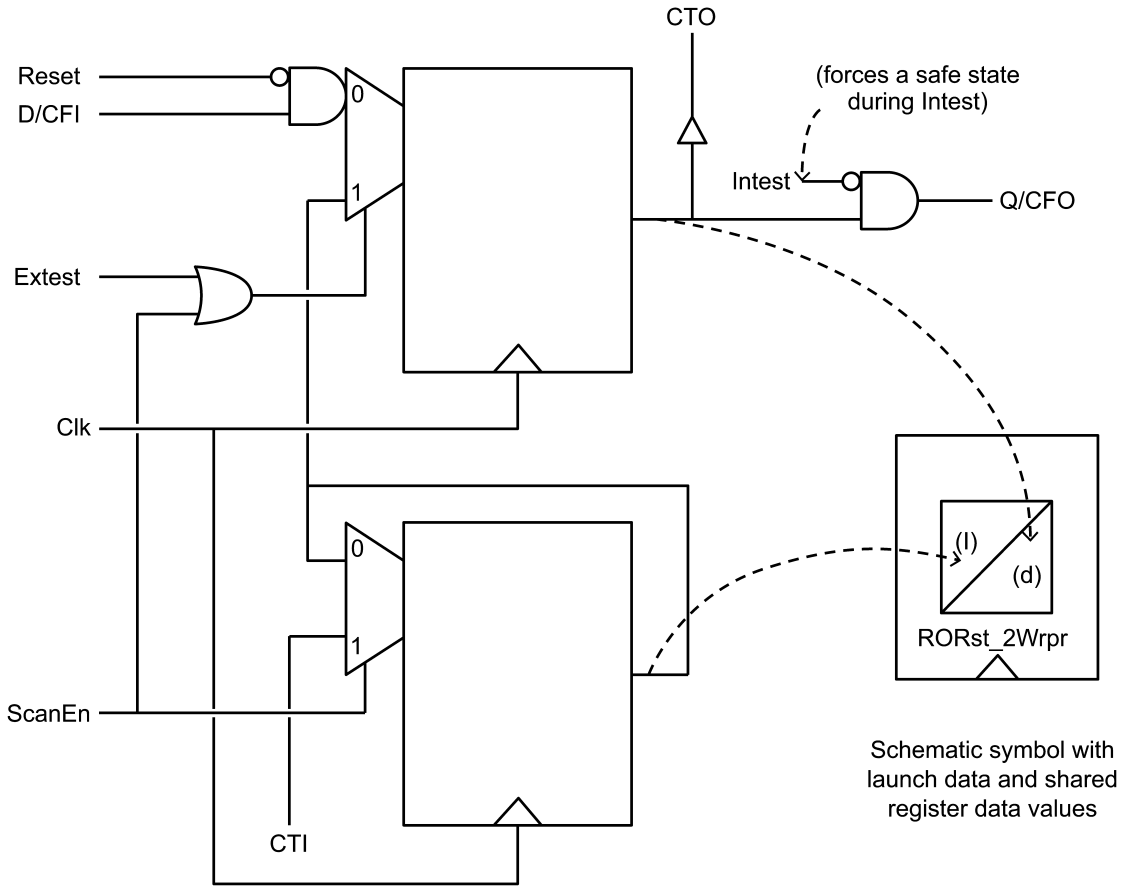


Figure 29—RORst_2Wrpr

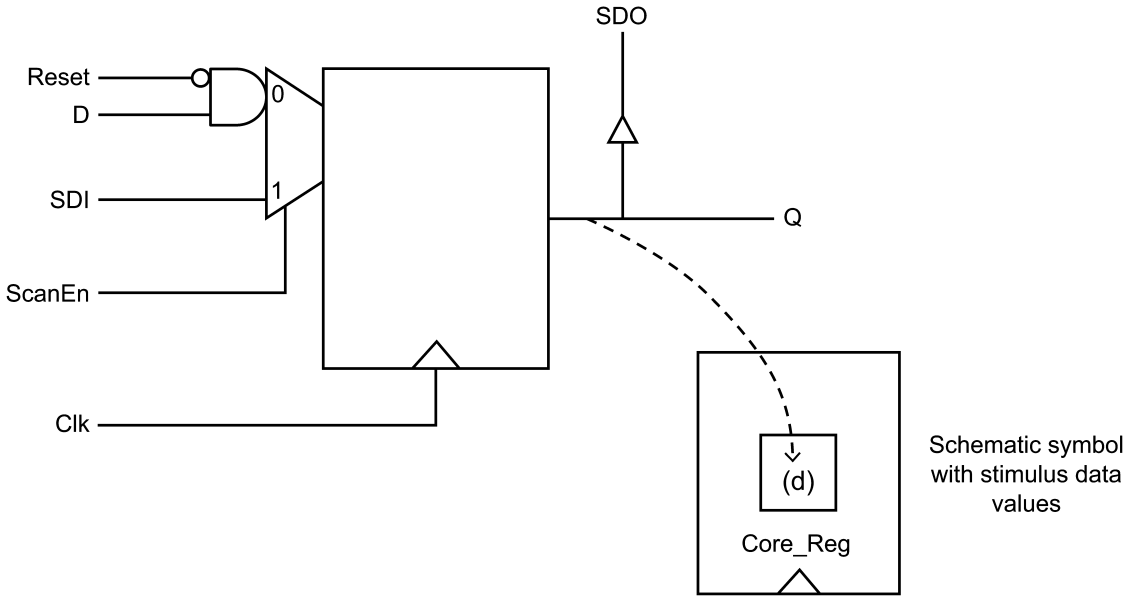


Figure 30—Core_Reg

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

High-speed clock 16xClk (AUXCK) is generated by an internal phase-locked loop (PLL). Modern cores use on-chip clock control (OCC) circuits to control such high-speed clocks to apply delay tests at-speed. Figure 31 depicts a detailed schematic of a plausible example of how an OCC can be harnessed to the WCS to apply at-speed delay tests. The signals shown as Capture_WBR and Shift_WBR are versions of CaptureWR and ShiftWR, respectively, that are qualified by WIR decode, the WS_INTEST_SCAN instruction in this case. State diagrams for the capture and shift functions are shown at the top of the figure for convenience. These use the signals shown as CaptEn and ShftEn respectively as the state-directing variables. Though their internal details are not shown, the depicted synchronizers can be understood by normally skilled readers to translate the Capture_WBR and Shift_WBR signals from the WRCK clock domain into the 16xClk clock domain. The S state machine generates a single 16xClk cycle pulse on clock enable signal S1 once for every rising edge of WRCK while Shift_WBR = 1. The C state machine sequentially generates single 16xClk cycle pulses on clock enable signals C1 and C2 once when a rising edge of Capture_WBR is clocked in by WRCK. Scan registers G1 and G2 qualify signals C1 and C2 respectively to provide vector-by-vector programming of capture clock pulses issued on Core_Clk. If all the respective constituent flip-flops were to power up in the zero state, the NOR gates in the C and S state machines ensure that the respective state machines self initialize. Other power-up states will quickly progress to states C0 and S0. Figure 32 depicts a timing diagram for applying a delay test with this example OCC presuming that G1 and G2 are programmed to ones.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

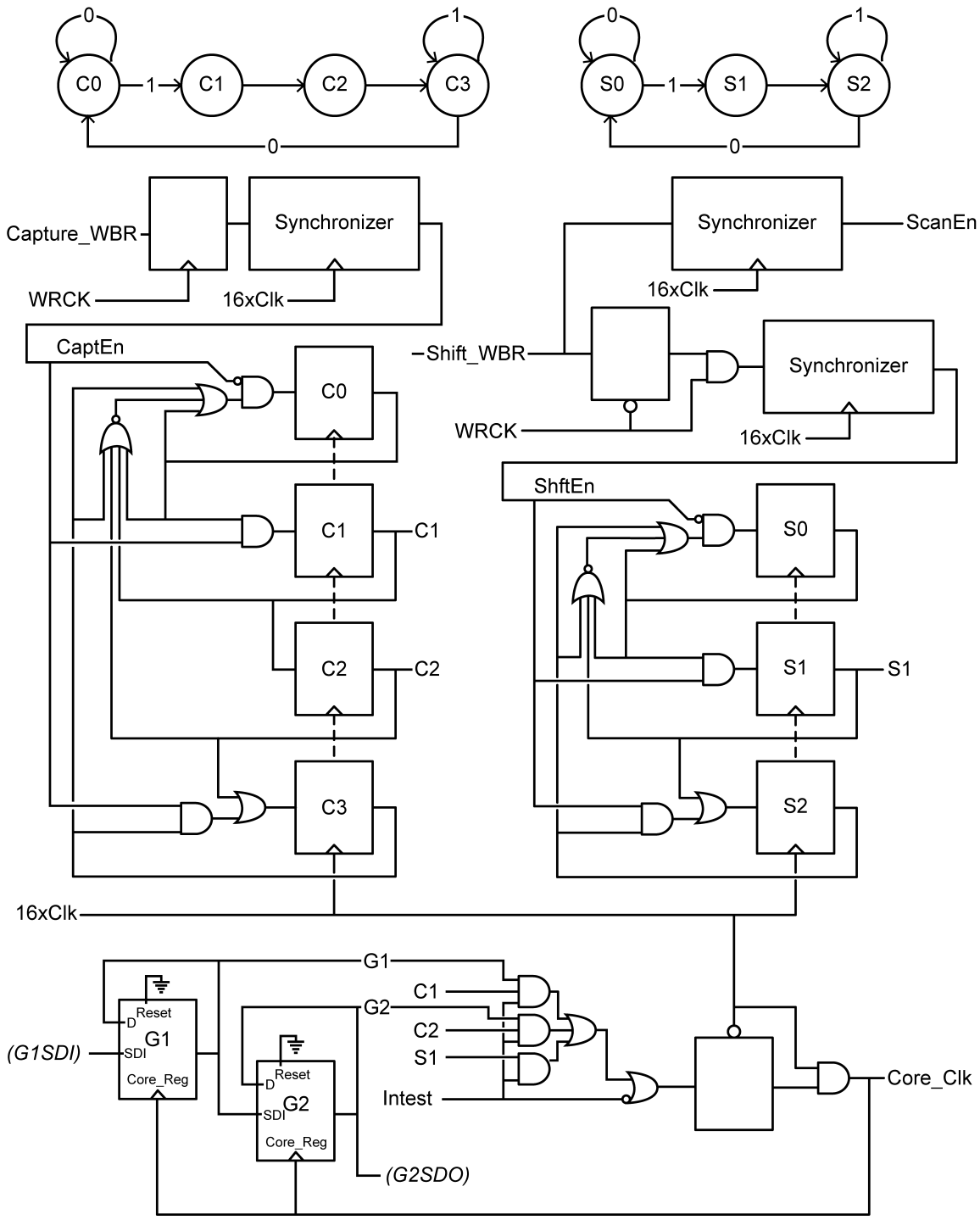


Figure 31 —Example OCC

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

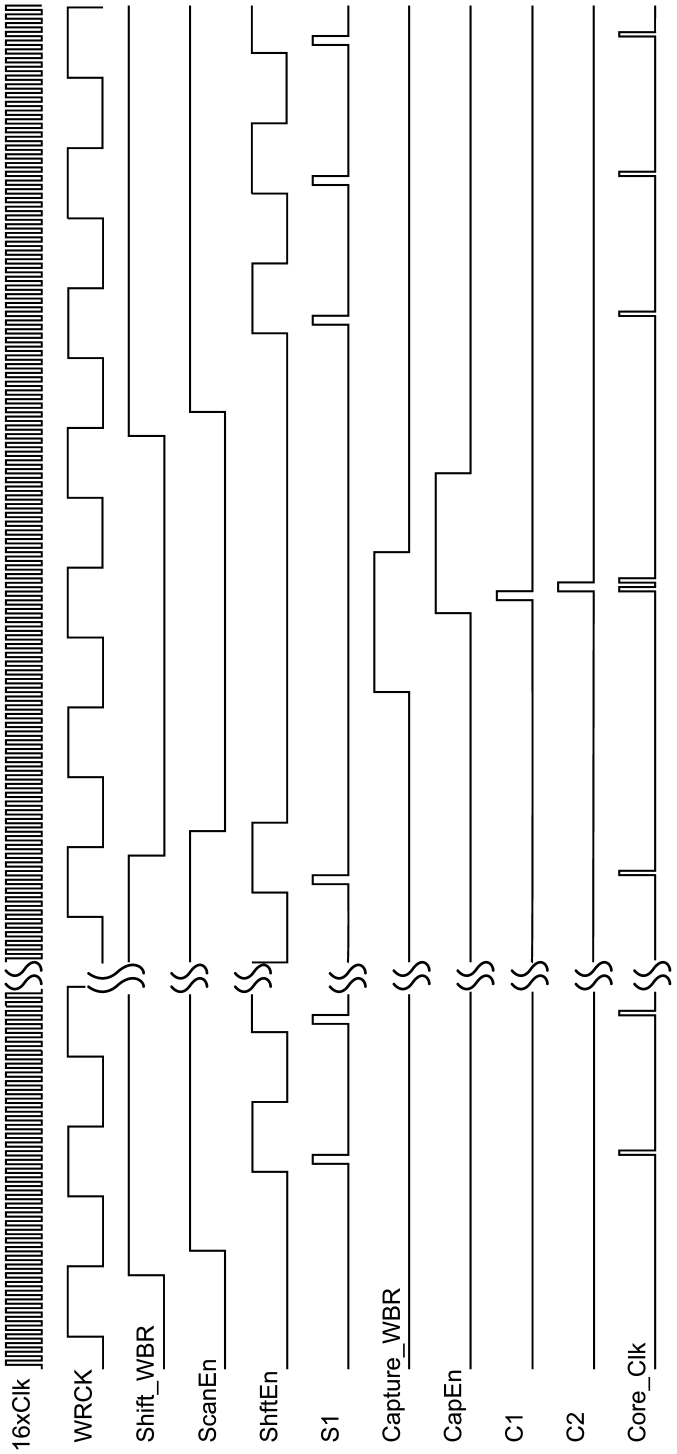


Figure 32—Example OCC timing

Figure 33 depicts how the WC_SF2_CI2 type cells are stitched into the serial path of the WBR (shown as a dashed line) and how the serial path is extended through the internal scan path (shown as a dash-dotted line). This is a simplified view that omits various means that configure and sensitize the serial path in accordance with the purpose of the WS_INTEST_SCAN instruction.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

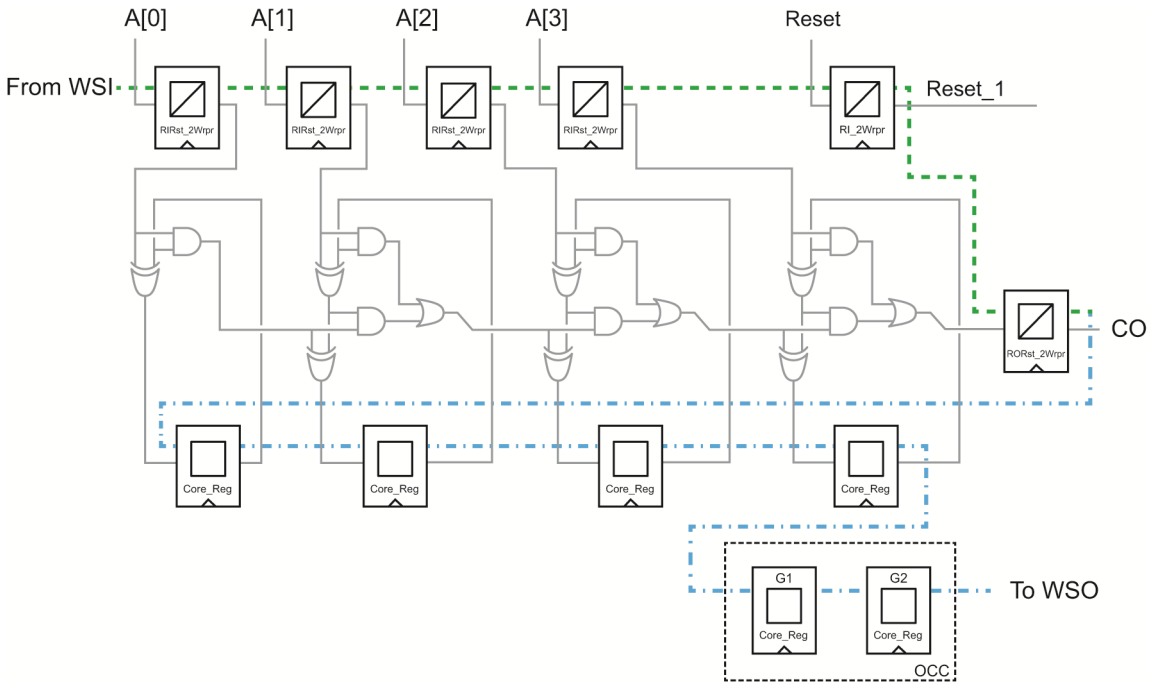


Figure 33—WBR serial scan path

Figure 34 depicts the data that is loaded into each of the cells for a single example delay test pattern. The pattern corresponds to a functional case of adding 9 to 7 to sum to 16. The values for OCC registers G1 and G2 are not shown, but they are both ones as in the situation of the timing diagram of Figure 32. The values shown for the internal wires in the adder are respectively from right to left those of the circuit after the scan load operation and after the first of the two Core_Clk pulses of the capture event. Those circuit nodes shown with values 10 and 01 are sites whose associated slow-to-rise and slow-to-fall transition delay faults respectively are detected by the 7 + 9 pattern.

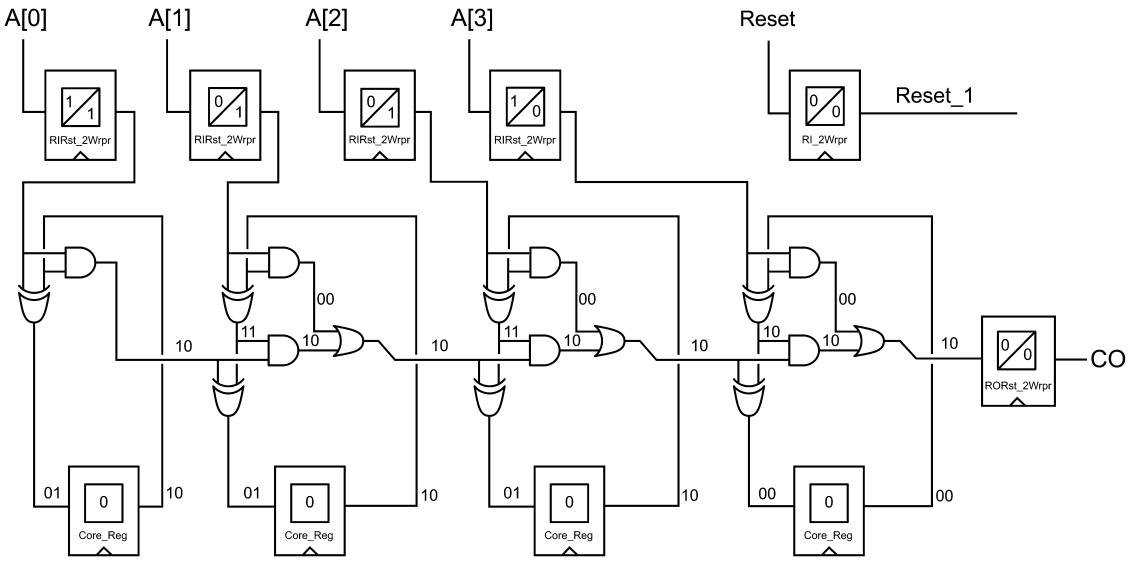


Figure 34—7 + 9 delay pattern

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Figure 35 depicts the register values that are the expected result of the pattern after the second of the two Core_Clk pulses of the capture event.

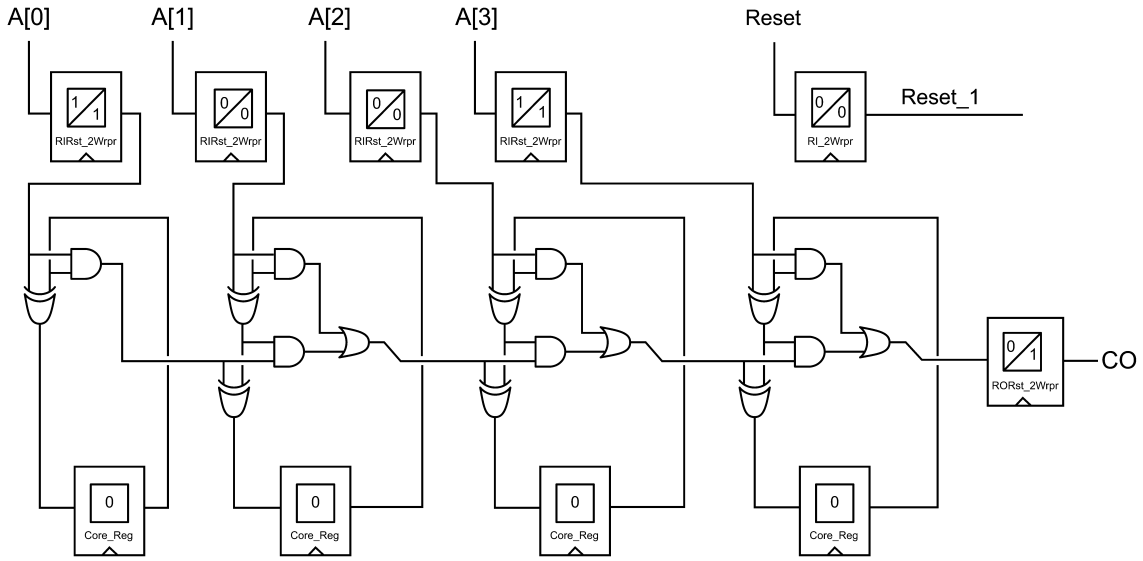


Figure 35—7 + 9 result

Figure 36, Figure 37, and Figure 38 together illustrate an alternative wrapping treatment of the core of Figure 26 in which WBR cells of type WC_SF1_CI1_G are organized to affect a WP_INTEST_SCAN implementation with four scan chains.

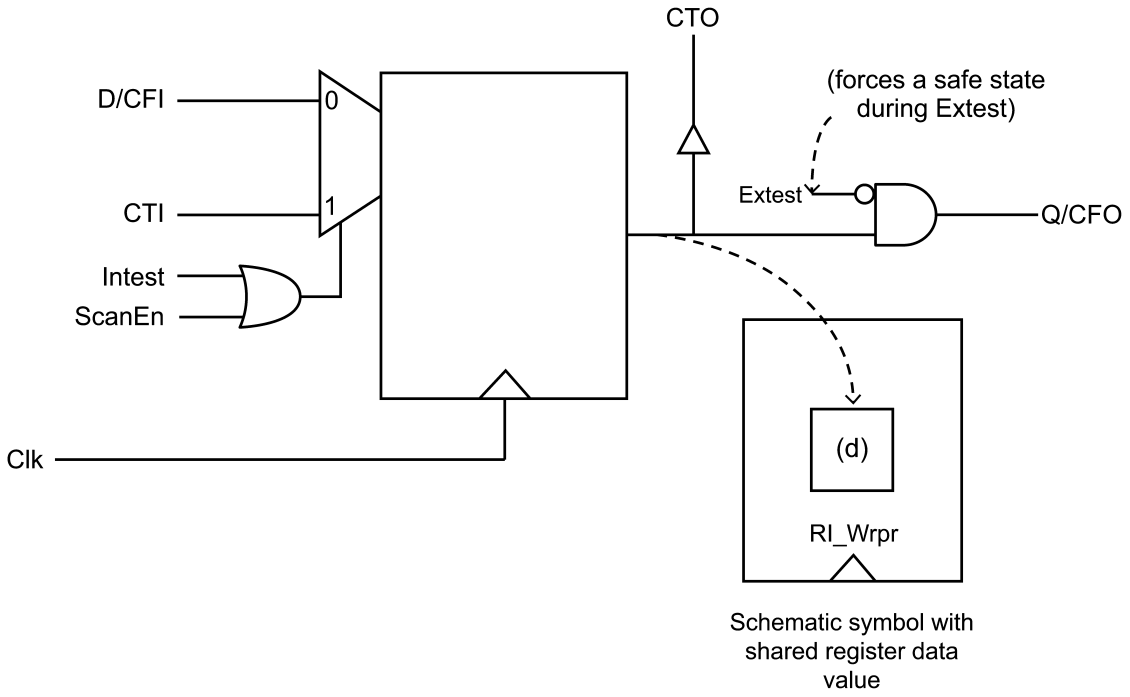


Figure 36—RI_Wrpr

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

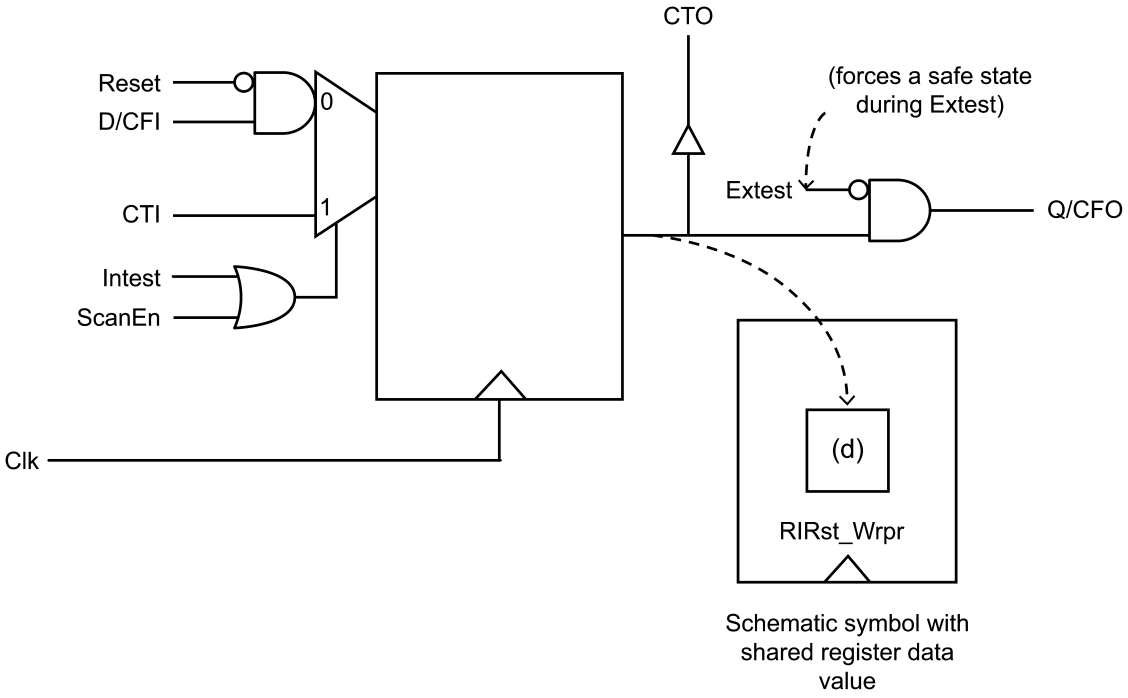


Figure 37—RIRst_Wrpr

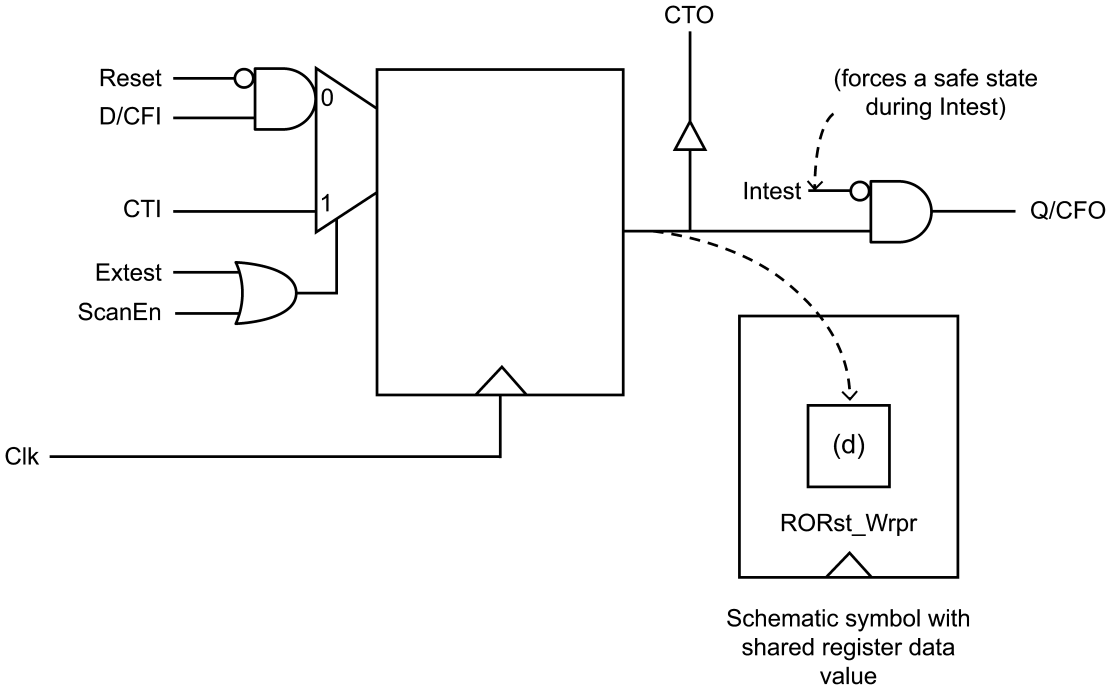


Figure 38—RORst_Wrpr

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

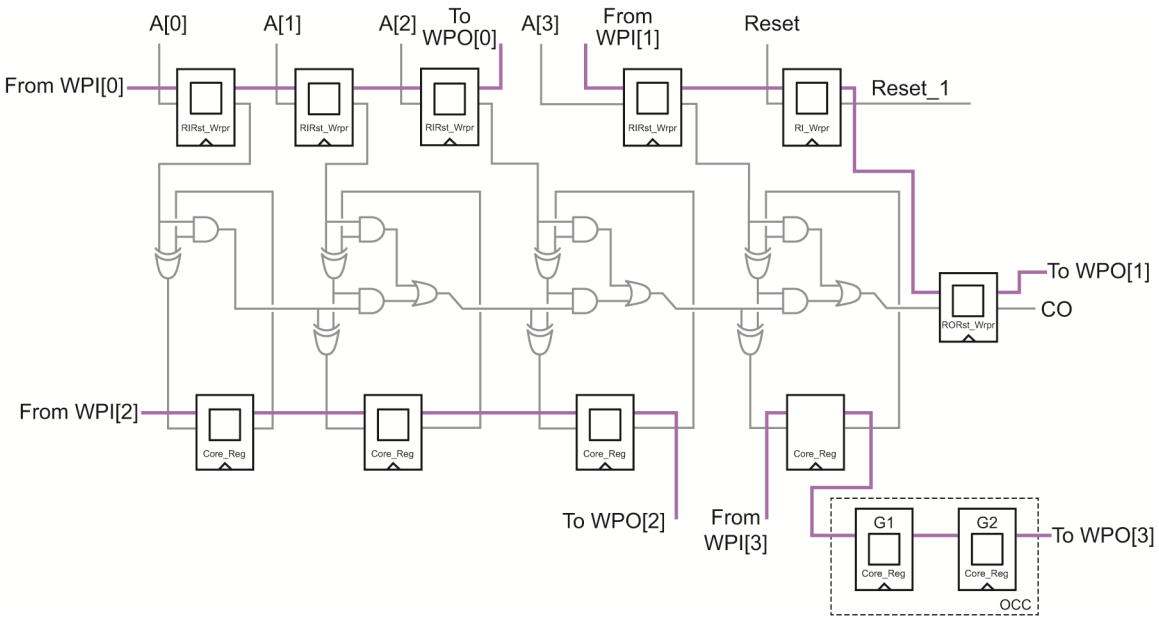


Figure 39—WBR with parallel scan chains

Figure 39 depicts WC_SF1_CI1 type cells stitched into two WBR chains between WPI[1:0] and WPO[1:0] and core internal registers are stitched into two internal scan chains between WPI[3:2] and WPO[3:2]. This is a simplified view that omits various means that configure and harness these chains to the parallel TAM in accordance with the purpose of the WP_INTEST_SCAN instruction. Note that this construct is incapable of applying the specific 7 + 9 delay test pattern of the previous example, but ATPG likely can achieve high transition fault coverage with other patterns of its derivation, nonetheless. This construct can use the same OCC as the previous example, but it could just as well use a different external protocol and a different OCC that does not include the WSC signals, since this example is for a parallel instruction.

13. Wrapper states

13.1 Wrapper Disabled and Wrapper Enabled states

13.1.1 Introduction

The WRSTN signal can be used to force the wrapper logic into a state that enables functional operation of the core. Figure 40 and Figure 41 identify two main wrapper states: Wrapper Disabled and Wrapper Enabled, respectively. Both have substates that are entered in a sequence controlled by the WSP as defined by this standard. The wrapper is forced unconditionally to a Wrapper Disabled state when the WRSTN signal of the WSP is low. In Wrapper Disabled state, the wrapper is inactive and functional operation of the core logic can continue unhindered. Test operations are performed while the wrapper is kept in Wrapper Enabled state by a high value on the WRSTN signal.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

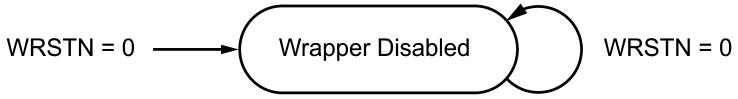


Figure 40 —Wrapper Disabled state conditions

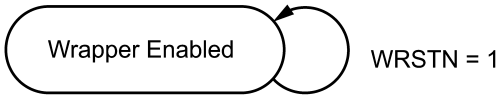


Figure 41 —Wrapper Enabled state conditions

13.1.2 Specifications

Rules

a) Asserting logic 0 on WRSTN (WRSTN = 0) shall unconditionally force the wrapper into Wrapper Disabled state; no actions of the wrapper logic (e.g., WIR, WBR) shall occur in response to WSP signals and/or WPP signals other than WRSTN and/or WRCK.

b) Asserting logic 0 on WRSTN (WRSTN = 0) shall unconditionally force the WS_BYPASS instruction to become effective.

c) Asserting logic 1 on WRSTN (WRSTN = 1) shall unconditionally enable the wrapper to respond to the WSP signals.

d) When the wrapper logic becomes enabled after being disabled, the effective instruction present in the WIR shall be the WS_BYPASS instruction.

Recommendations

e) The protocol used to control the WRSTN signal should apply at least one complete WRCK pulse while WRSTN is set to logic 0 (WRSTN = 0), before WRSTN can be set to logic 1 (WRSTN = 1).

Permissions

f) The wrapper may require an active WRCK during Wrapper Disabled state (WRSTN = 0), prior to becoming enabled by logic 1 on WRSTN (WRSTN = 1).

13.1.3 Description

Independent of the current state of a wrapper, this wrapper shall enter Wrapper Disabled state when the active low WRSTN signal is asserted. The wrapper shall remain in this state while the WRSTN signal remains low. In the Wrapper Disabled state, the wrapper is forced to its inactive state. This is achieved either by asynchronously resetting the WIR contents to WS_BYPASS or by gating the WIR outputs (with WRSTN = 0) to create the same effect. In the latter case, a synchronous reset operation is still needed to load the WS_BYPASS instruction into the WIR before the wrapper is enabled by de-asserting the WRSTN signal. For this reason, it is allowed to require an active WRCK during Wrapper Disabled state, prior to using the wrapper in Wrapper Enabled state.

If asynchronous reset of the WIR is used to implement the Wrapper Disabled state, the sequence prior to test operations consists only of deasserting WRSTN. When gating of WIR outputs is used to implement the Wrapper Disabled state and a synchronous reset of the WIR is needed to load WS_BYPASS, the sequence prior to test operations needs an active WRCK during the Wrapper Disabled state. Because this sequence

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

can also be used for wrappers with an asynchronous reset of the WIR, it is recommended for all wrappers. In order to have a reliable operation of the wrapper, WRSTN should be deasserted only on the falling edge of WRCK so that at least 1/2 WRCK period is left for signals to propagate within the wrapper. Figure 47 describes a synchronous reset timing.

All rules described in this standard apply to the Wrapper Enabled state, except rules specific to the Wrapper Disabled state of the IEEE 1500 wrapper.

14. WSP timing diagrams

14.1 Introduction

This clause defines timing relationships between the mandatory WSP terminals. The actual values, denoted TBD (to be determined), are to be supplied in the CTL by the wrapper provider. These timing specification parameters apply to any wrapper implementation. Values with a hyphen (-) indicate they are not necessary or not pertinent to the usage. Timing diagrams shown in this clause are not meant to show protocol, but rather timing relationships between wrapper signals.

14.2 Specifications

Rules

a) SelectWIR, ShiftWR, and CaptureWR shall be sampled on the rising edge of WRCK.

b) The UpdateWR signal shall be sampled on the falling edge of WRCK.

c) During a WSP Shift operation, WSI shall be sampled on the rising edge of WRCK.

d) During a WSP Shift operation, when only the WSO terminal is provided as the wrapper scan out, WSO shall change on the WRCK edge specified by the WRCK ReferenceEdge keyword (i.e., Leading or Trailing) given in CTL.

e) During a WSP Shift operation, when the optional WSOR is provided, WSOR shall change on the rising edge of WRCK and WSO shall change on the next falling edge of WRCK. Both WSOR and WSO edges shall be specified by the WRCK ReferenceEdge keyword in CTL.

f) Actions of IEEE 1500 standard components (e.g., WIR, WDRs) shall occur on either the rising or falling edge of WRCK in response to WSC signals as depicted in Figure 42, Figure 43, and Figure 44.

g) Changes on the SelectWIR signal shall not occur coincident with Shift, Capture, or Update operations.

h) Provided that permission 13.1.2(f) is exercised, the timing relationships depicted in Figure 47 shall be applicable.

Permissions

i) A second wrapper scan output terminal, WSOR, that changes on the rising edge of WRCK during WSP Shift operations, may be provided in addition to the mandatory WSO.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

14.3 Description

14.3.1 Introduction

Even though ShiftWR can change while WRCK is high, it is required that data driven from WSO(R), changing at the specified edge of WRCK, shall result from the data or instruction register that was selected immediately preceding the rising edge of WRCK.

Table 3, Figure 42, Figure 43, and Figure 44 describe timing parameters for WSP operation. See Clause 8 for details on these signals.

Table 3—WSC and WBR timing parameters for WSP operation

Timing parameter	Symbol	Min	Max
Clock pulse high	t_{ckwh}	TBD	—
Clock pulse low	t_{ckwl}	TBD	—
WRSTN Pulse Width Low	t_{rstl}	TBD	—
WRSTN negation setup time with respect to (w.r.t.) WRCK rising edge	t_{rstsu}	TBD	—
SelectWIR setup time w.r.t. WRCK rising edge	t_{swsu}	TBD	—
SelectWIR hold time w.r.t. WRCK rising edge	t_{swhd}	TBD	—
ShiftWR or CaptureWR setup time w.r.t. WRCK rising edge	t_{ctlsu}	TBD	—
ShiftWR or CaptureWR hold time w.r.t. WRCK rising edge	t_{ctlhd}	TBD	—
UpdateWR setup time w.r.t. WRCK falling edge	t_{updsu}	TBD	—
UpdateWR hold time w.r.t. WRCK falling edge	t_{updhd}	TBD	—
WSI setup time w.r.t. WRCK rising edge	t_{sisu}	TBD	—
WSI hold time w.r.t. WRCK rising edge	t_{sihd}	TBD	—
WSO output valid time w.r.t. WRCK falling edge	t_{sov}	—	TBD
WSO(R) output valid time w.r.t WRCK rising edge	t_{sorv}	—	TBD

Timing relationship of the AUXCK with respect to WRCK is discussed in Clause 16.

Figure 42 through Figure 44 provide timing diagrams for the WSP. Figure 42 shows the WSP timing where WSO is changing on the falling edge of WRCK, and Figure 43 show the WSP timing when WSO is specified to change on the rising edge of WRCK. Figure 44 shows the case where the optional WSOR terminal is provided, where WSOR changes on the rising edge of WRCK and the WSO changes on the falling edge of WRCK. For rising edge WSO (see WSO timing in Figure 43) or WSOR timing, the timing parameter for scan out valid (t_{sov}) shall be with respect to the rising edge of WRCK.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

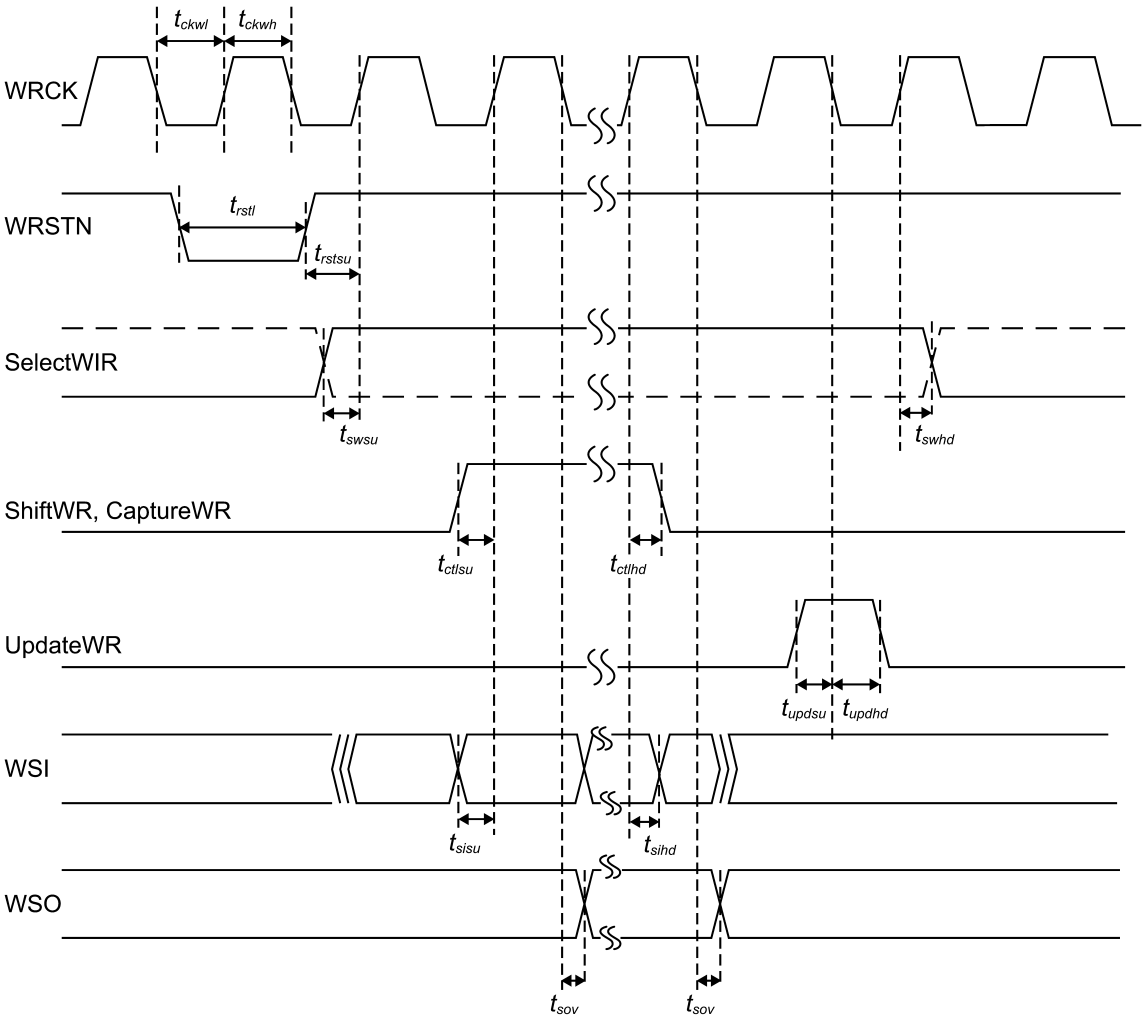


Figure 42—IEEE 1500 WSP timing, WSO falling WRCK edge

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

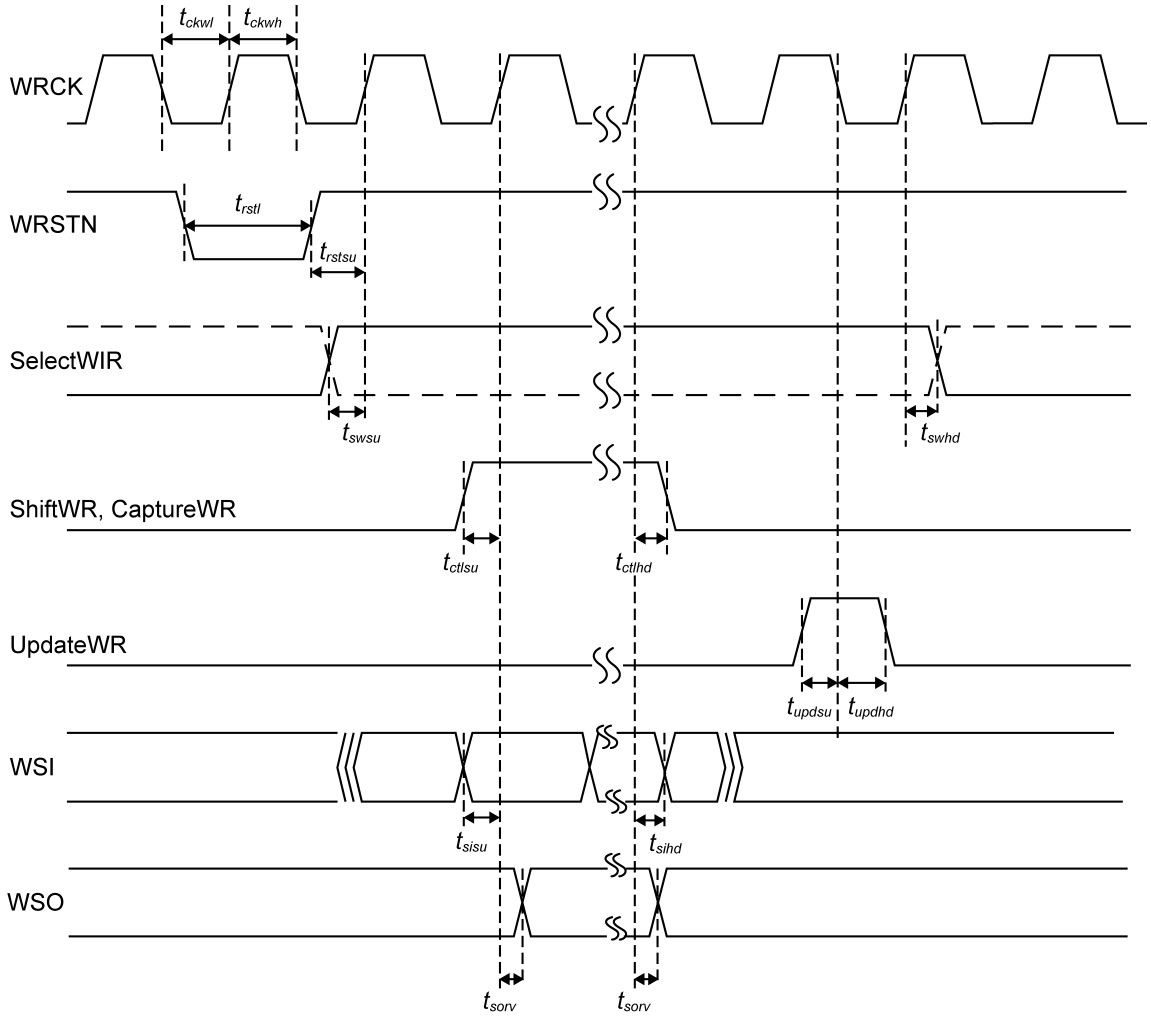


Figure 43 —IEEE 1500 WSP timing, WSO rising WRCK edge

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

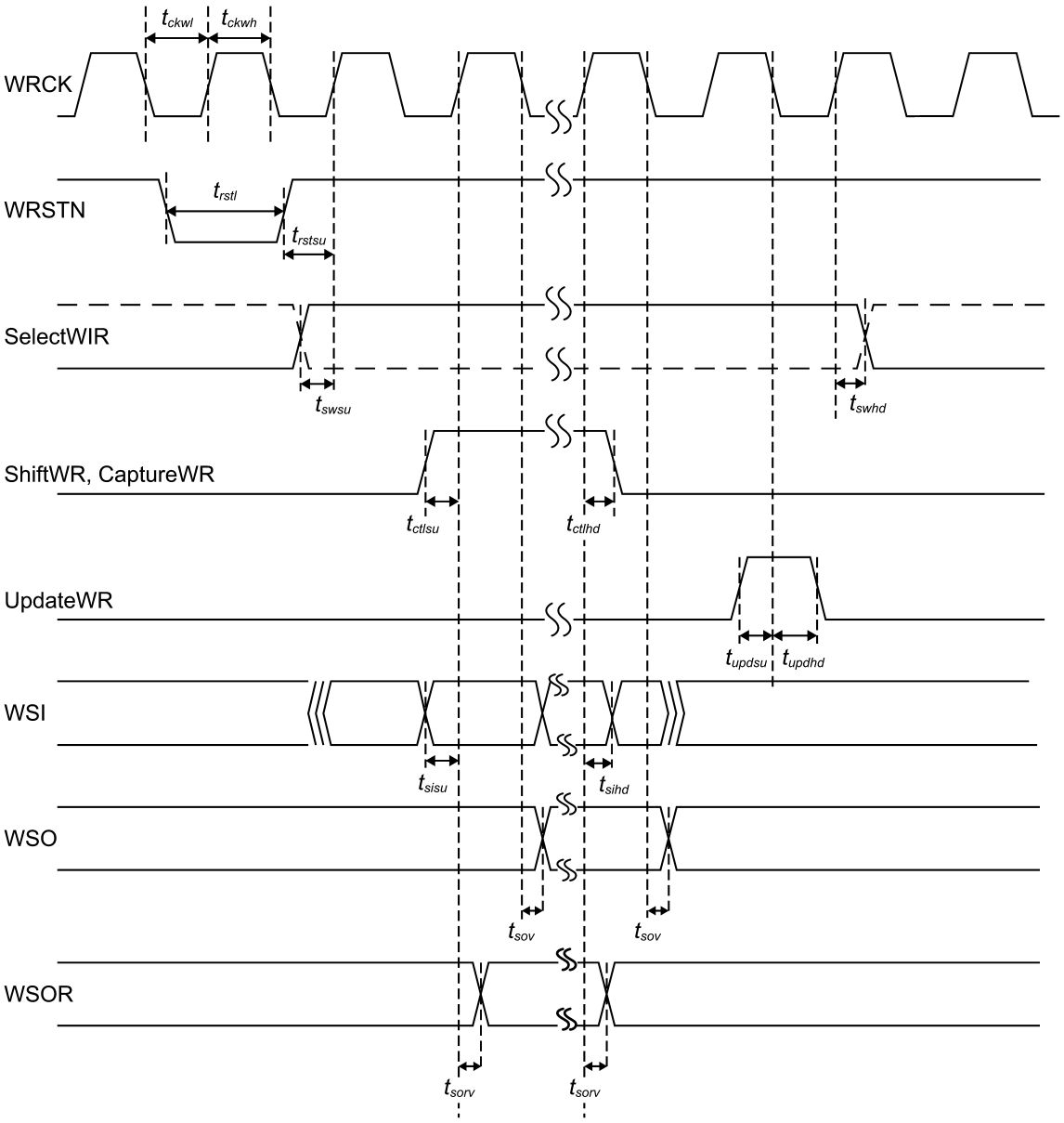


Figure 44—IEEE 1500 WSP timing, with both WSO and WSOR present

14.3.2 Timing parameters for event and functional input/output (I/O)

Table 4, Figure 45, and Figure 46 describe timing parameters regarding events and WFI/WFO. Figure 45 describes timing relationships for wrapper cells with minimally required terminals, CaptureWR, and ShiftWR. Figure 46 describes timing relationships of wrapper cells that support the Update event.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Table 4—Event and functional timing

Timing parameter	Symbol	Min	Max
CaptureWR setup time w.r.t. WRCK rising edge	t_{ctlsu}	TBD	—
CaptureWR hold time w.r.t. WRCK rising edge	t_{ctlhd}	TBD	—
UpdateWR setup time w.r.t. WRCK falling edge	t_{updsu}	TBD	—
UpdateWR hold time w.r.t. WRCK falling edge	t_{updhd}	TBD	—
WFO output valid time w.r.t. WRCK rising edge without update	t_{rov}	—	TBD
WFO output valid time w.r.t. WRCK falling edge with update	t_{fov}	—	TBD
WFI setup time w.r.t. WRCK rising edge	t_{fisu}	TBD	—
WFI hold time w.r.t. WRCK rising edge	t_{fihd}	TBD	—

The minimal configuration timing specification shown in Figure 45 can apply, as an example, to wrapper cell WC_SF1_CI1 from Figure 21, where CFI is to be connected to WFI and CFO to be connected to WFO as shown in Figure 17.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

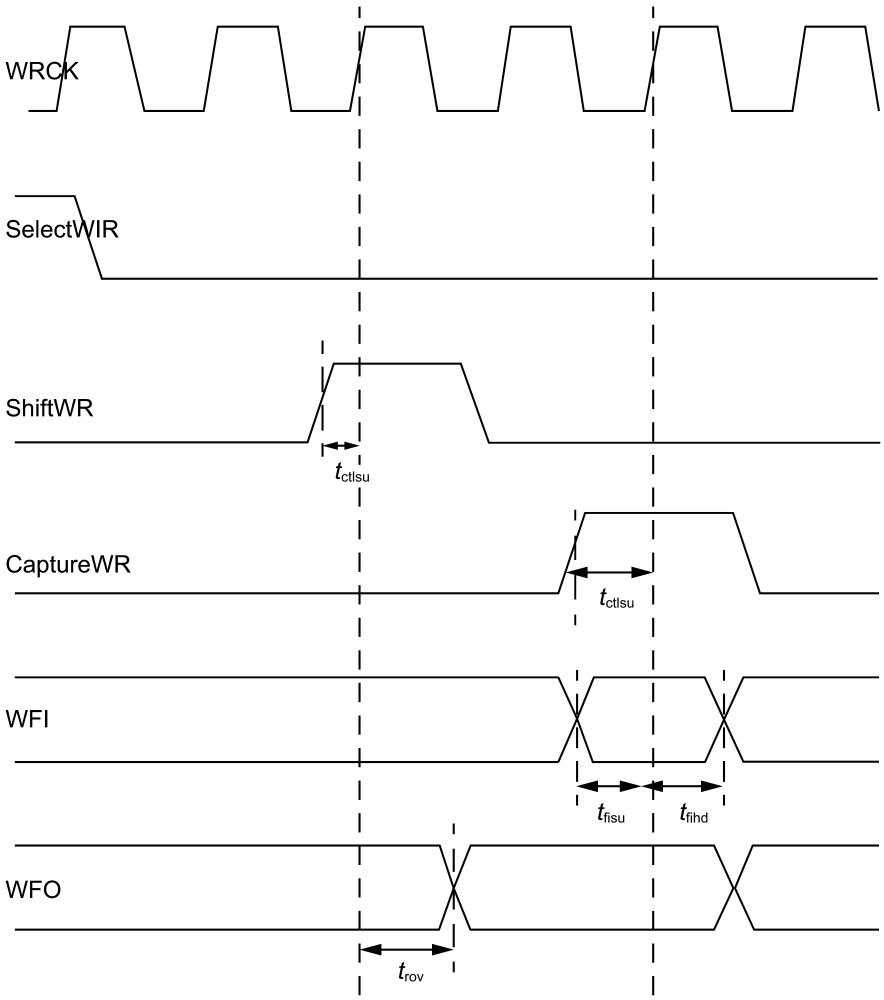


Figure 45—IEEE 1500 minimal event and functional I/O timing

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

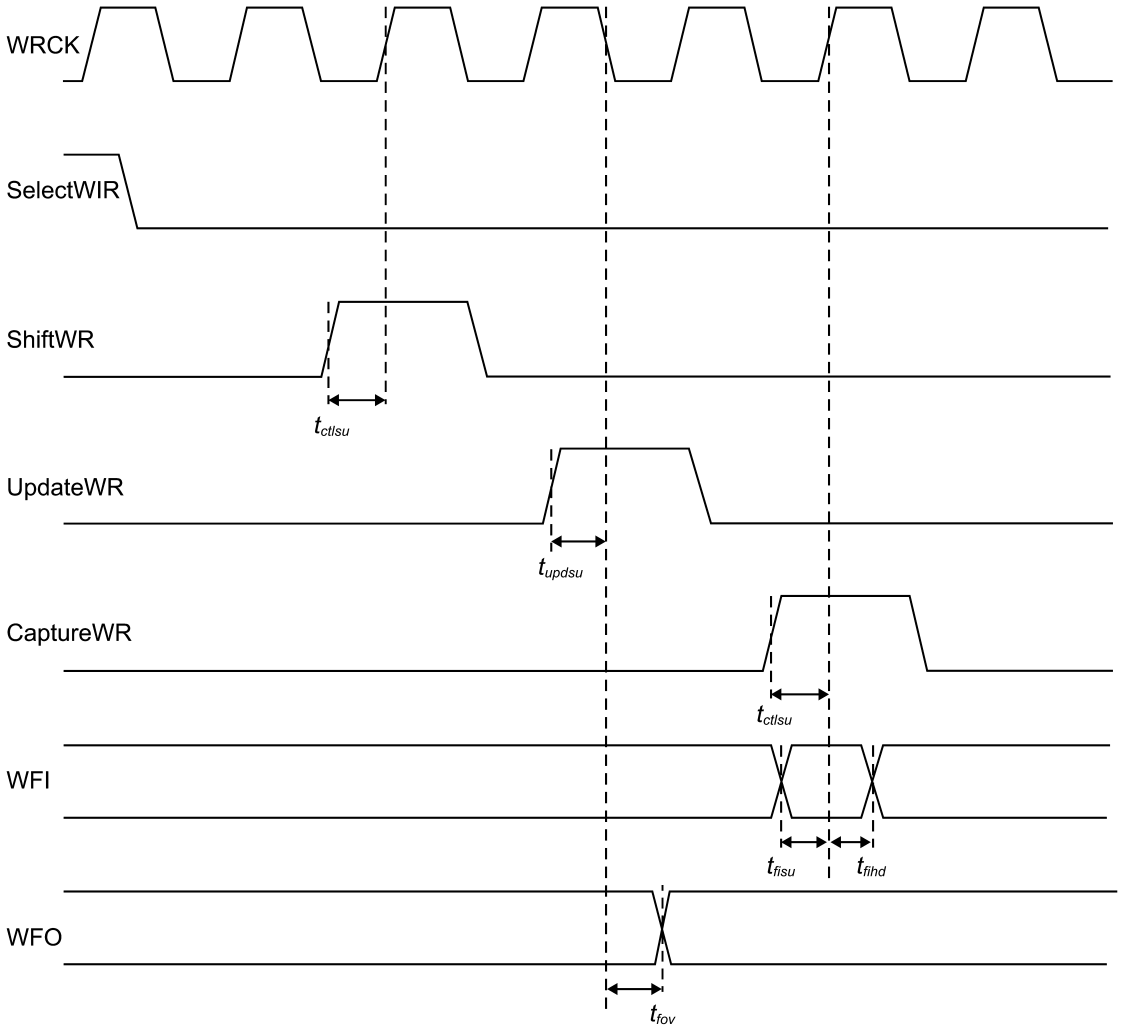


Figure 46—IEEE 1500 update event and functional I/O timing

14.4 Synchronous reset timing

Table 5 and Figure 47 show the timing parameters for an optional synchronous reset.

Table 5—Synchronous reset timing

Timing parameter	Symbol	Min	Max
Clock pulse high	t_{ckwh}	TBD	—
Clock pulse low	t_{ckwl}	TBD	—
Synchronous WRSTN assertion setup time w.r.t. WRCK rising edge	t_{srstsu}	TBD	—
Synchronous WRSTN assertion hold time w.r.t. WRCK falling edge	t_{srsthd}	TBD	—

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

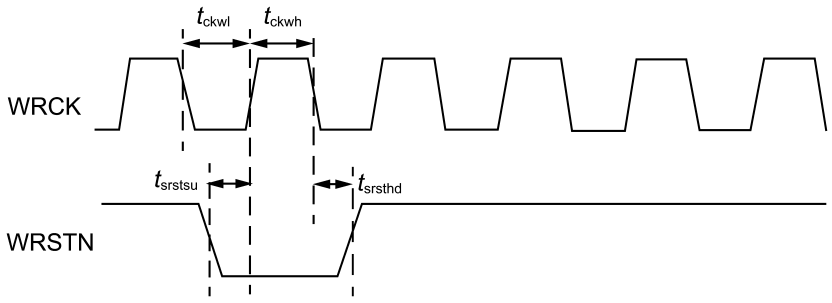


Figure 47 —IEEE 1500 synchronous reset timing

14.5 Timing for fast pipelined shift

Figure 48 shows an example of WSP timing that will allow for higher speed shifting when the WSP signals have long routes across the SoC, from the source of the WSP signals to the IEEE 1500 wrapper of the core. In such cases, these long routes will affect the timing of the WSP signals and the performance of shifting WSP instructions and data.

The solution often requires pipelining the WSP signals in the SoC to meet a specific timing performance. However, pipelining the control signals (i.e., CaptureWR, ShiftWR, and UpdateWR) is not recommended, as it will affect the WSP protocol as it interoperates between IEEE 1500 cores and other test logic in the SoC, such as the IEEE 1149.1 TAP controller and test logic. Whereas pipelining of the serial data path, i.e., the WSI and WSO(R), between cores and test logic in the SoC is acceptable, as it simply adds additional shift elements in the path that are easily accounted for during shift operations.

The WSP timing in Figure 48 shows a fast shift methodology where the WSI-WSO(R) data path can be pipelined in the SoC, but the WSC signals, SelectWIR, CaptureWR, ShiftWR, and UpdateWR are not pipelined. Instead, CaptureWR, ShiftWR, and UpdateWR are timed as multi-cycle paths. This is done by delaying the WRCK pulses for the Capture, Shift, and Update events and then operating WRCK at its full frequency during the shift operation.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

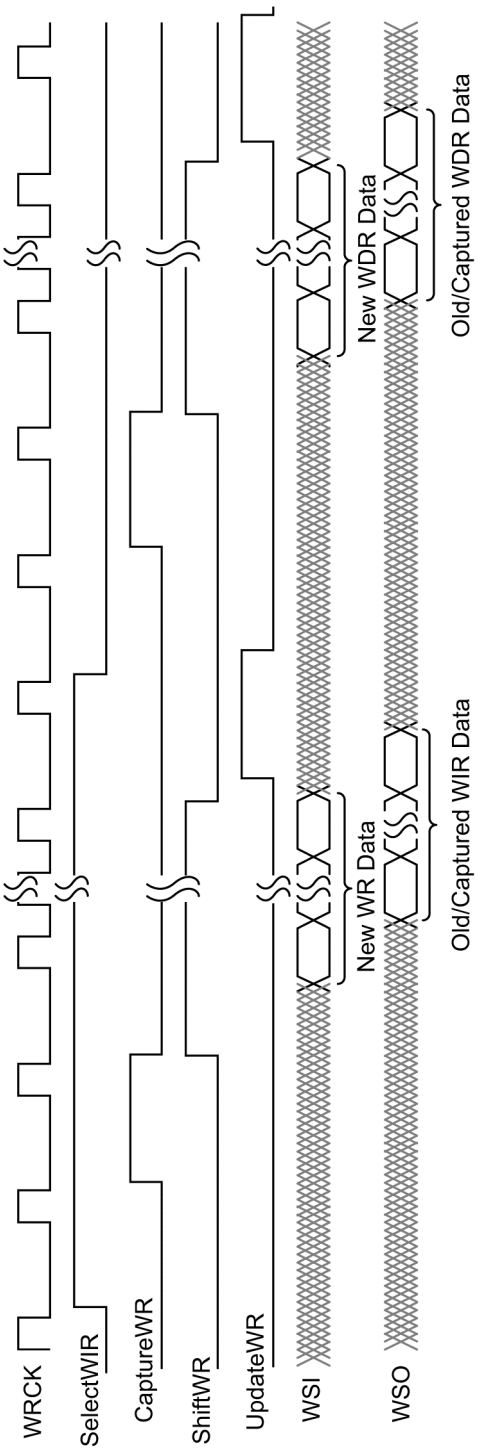


Figure 48 —Fast shift timing

As shown in Figure 48, SelectWIR is asserted and then there is a cycle delay before the next WRCK pulse. This delay allows the SelectWIR signal an extra cycle to propagate. Similarly, WRCK is delayed for the assertion of CaptureWR and then the assertion of ShiftWR. Once the ShiftWR has had time to propagate, a WIR shift operation occurs. As the ShiftWR is stable during the shift operation, it can occur at the full WRCK frequency, without delaying the WRCK pulses. Following the shift operation, UpdateWR is

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

asserted and the WRCK pulse is delayed. The same capture, shift, update operations are shown in the other half of the timing diagram for the selected WDR. Although there is a single WRCK cycle delay shown in the example timing diagram, this delay can be multiple cycles, as dictated by the SoC timing requirements for the control signals and the full-speed shift frequency.

Support for this timing method requires proper timing analysis, for the multi-cycle control signals and the full-speed WRCK shift. In addition, test pattern generation and pattern formatting shall follow the timing method. Figure 48 shows the timing for the WSP, but this can be extended to the IEEE 1149.1 TAP controller, when the IEEE 1500 cores are accessed using the TAP controller's protocol. In this case, the test pattern generation and formatting would skip TCK pulses and extend TMS in order to control the CaptureIR/DR, ShiftIR/DR, and UpdateIR/DR states to create multi-cycle propagation delays for the IEEE 1500 WSC signals. Note that this methodology does not require any changes to the TAP controller or IEEE 1500 hardware implementation, only formatting of the patterns, and thus can be done post-silicon if required to meet timing.

15. WSP configurations for IEEE 1500 system chips

15.1 Introduction

This clause provides specifications and description for configuring multiple IEEE 1500 WSPs at the system chip level.

15.2 Connecting multiple WSPs

15.2.1 Introduction

This subclause provides example system chip configurations when connecting the WSPs of multiple IEEE 1500 wrappers. Figure 49 through Figure 52 show various configurations of IEEE 1500 connections for the WSP. In the figures, the system chip configuration is defined by the way the WSPs and data register scan paths are connected at the system chip, core, and subcore levels and how the WSPs are controlled. IEEE Std 1500 allows single or multiple WSPs to be configured, depending on the application needs of the system chip, cores, and subcores.

In Figure 49, the WSP scan path order is Core1-Core2…CoreN, and the signals connected to the WSC terminals are globally bussed. Therefore, all wrappers are controlled together. This allows different instructions to be loaded into different WIRs, and the WSP protocol of the cores is controlled in a lock-step fashion. Per rule 15.2.2(a), the Capture, Shift, and Update operations shall occur concurrently. Per recommendation 15.2.2(b), the SelectWIR to all WSPs should be controlled to the same value. Thus, all N WIRs would be selected simultaneously.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

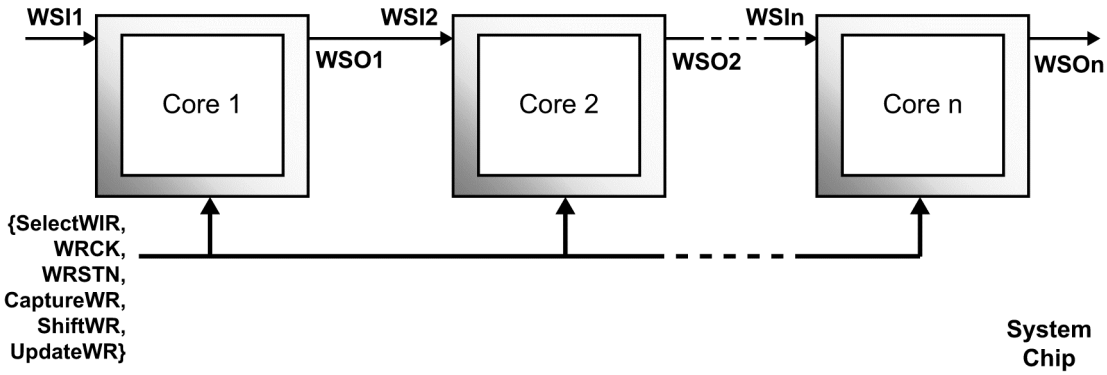


Figure 49—Serial connection of WSPs

The multiple parallel configuration shown in Figure 50 shows how the WSC signals can be bussed, as in the serial WSP configuration of Figure 49, yet the WSI-WSO(R) scan paths of the three cores are separate parallel scan paths, one scan path for each core. This configuration supports parallel access to the WIR and WRs of the three cores, while still following rule 15.2.2(a) and recommendation 15.2.2(b).

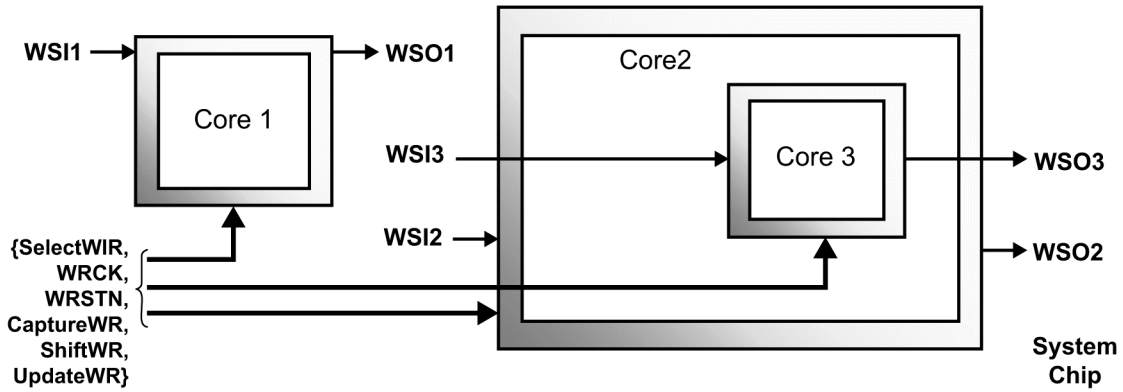


Figure 50—Multiple parallel WSP configuration

Figure 51 and Figure 52 show WSP configurations with a core, Core 1 (which has two subcores), Core 2, and Core 3. These two configurations show subcores that are accessed hierarchically by SIBs.

In Figure 51, subcores Core2 and Core3 are accessed using two inline SIBs, SIB2 and SIB3. An example of an inline SIB design is shown in Figure 53. When SIB2 and SIB3 are reset, the two subcores are bypassed, and the scan path through the hierarchy will be from WSI1→ Core1 → SIB2 → SIB3 → WSO1. By shifting the SIBs and updating them, the cores can be selected as needed for testing. When both cores are selected, the scan path through the hierarchy will be from WSI1→ Core1 → Core2 → SIB2 → Core3 → SIB3 → WSO1.

Note that the inline SIB shown in Figure 53 has two update storage elements, the second one delays the SIB select by one test clock cycle per rule 7.2.1(f).

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

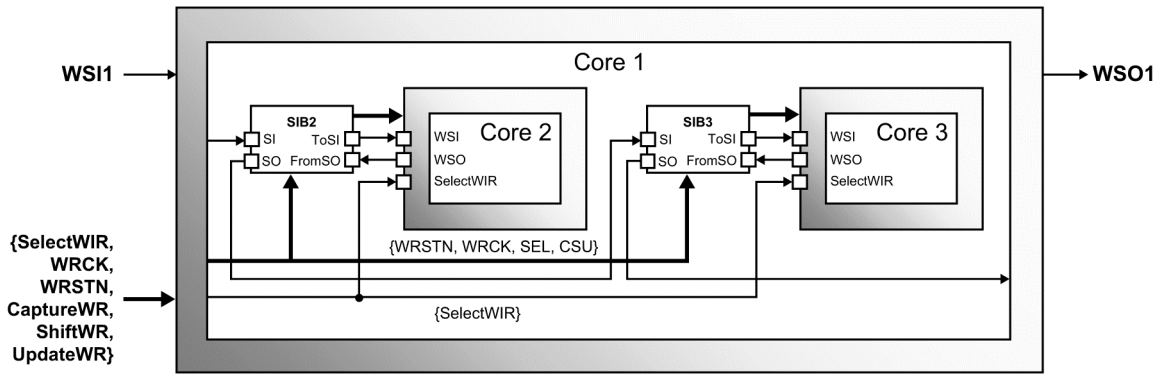


Figure 51—Hierarchical subcores with inline SIBs

In Figure 52, subcores Core2 and Core3 are accessed using two remote SIBs, RSIB2 and RSIB3. An example of a remote SIB design is shown in Figure 54. The remote SIBs have two scan paths, one for accessing the SIBs themselves, and a second remote path for accessing the selected subcore(s). The SIB path is labeled SI-SO in Figure 52, and ScanIn-ScanOut in Figure 54. The remote path is labeled RSI-RSO in Figure 52, and RScanIn-RScanOut in Figure 54. The two paths are accessed by two separate IEEE 1500 instructions in Core1. One instruction is used to control the SIB select values, through the SI-SO daisy chain path of RSIB2 and RSIB3. The data register scan path for the instruction used to access the remote SIBs will be WSI1→ RSIB2 → RSIB3 → WSO1.

After the remote SIB select values are updated, another separate IEEE 1500 instruction in Core1 is used to access the scan path configuration through the subcores selected by the remote SIBs. When both cores in Figure 52 are selected, the scan path through the hierarchy will be from WSI1→ Core2 → Core3 → WSO1. If a subcore is not selected, there is a zero-bit bypass through the associated RSIB for that subcore. A bypass storage element may be added in the remote SIB if a registered bypass is needed.

Note that the RSIB does not have a storage element to delay the SIB select. This is because it satisfies rule 7.2.1(f) by using two separate instructions.

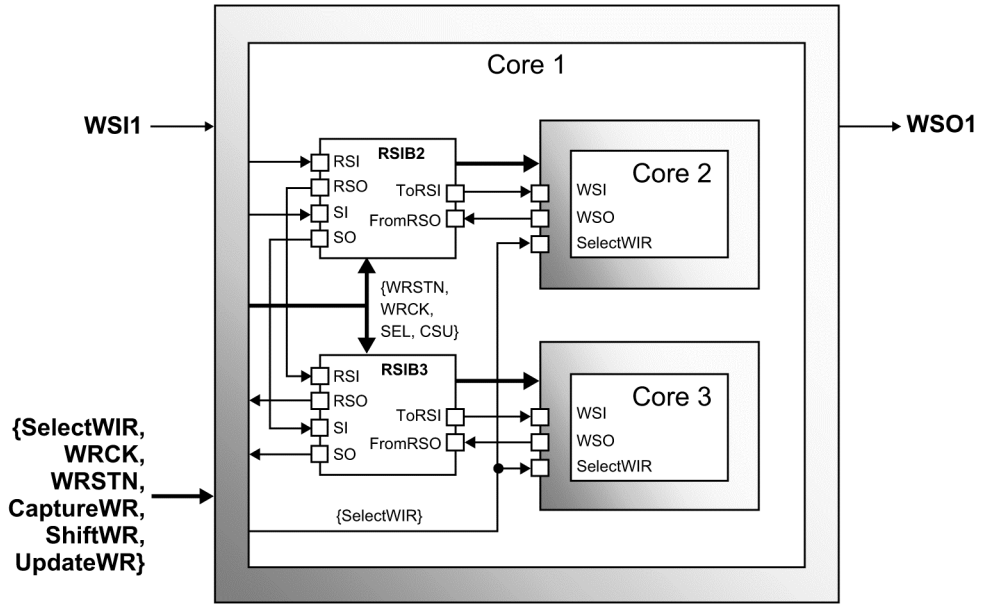


Figure 52—Hierarchical subcores with remote SIBs

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

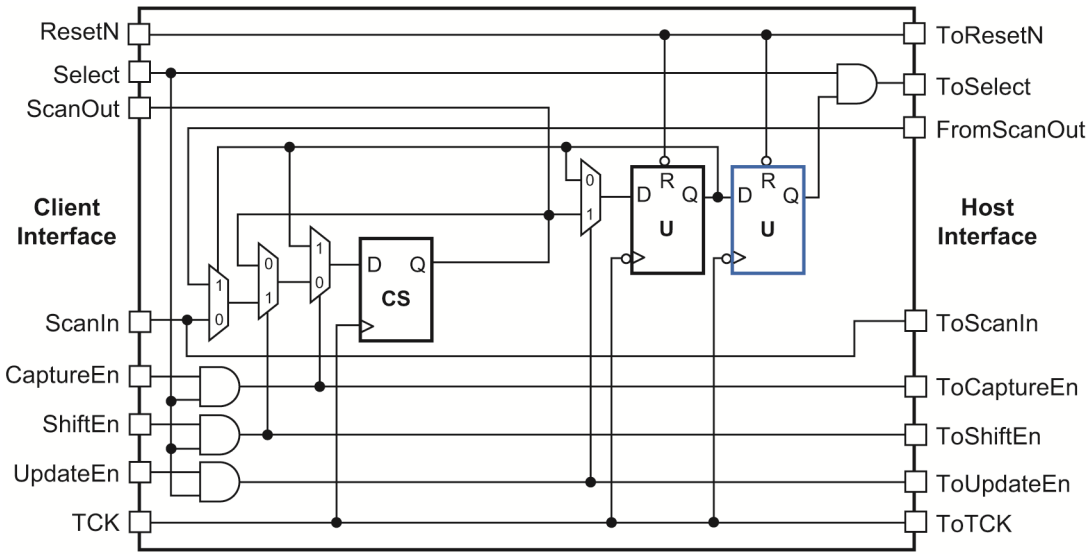


Figure 53—Example of IEEE 1687 inline SIB

Figure 53 is an example of an IEEE 1687 inline SIB. It has two interfaces: a client interface, and a host interface. The client interface connects to the interface or scan path segment that selects and controls the SIB, and the host interface connects to the interface or scan path segment that is selected and controlled by the SIB. The client interface can be selected and controlled by asserting the Select input of the SIB with the decode of an IEEE 1500 instruction that selects the SIB scan path. The TCK can be clocked by WRCK or an AUXCK shift clock, and ResetN can be connected to WRSTN. When ResetN is asserted to a logic 0, the SIB is reset, and the host interface is deselected. After reset, the scan path through the SIB includes only the CS storage element, from ScanIn to ScanOut.

When selected at the client interface, the SIB can then be used to select or deselect the host interface by shifting in a SIB value and updating it. When it is selected by the SIB update value, it enables the host scan path. Note that when selected, the host scan path is inserted into the scan path before the SIB. The scan path will be from the ScanIn input, to the ToScanIn output, and then through the selected host scan path and back to the FromScanOut input, followed by the SIB. The CaptureEn, ShiftEn, UpdateEn inputs, and ToSelect output are gated by the SIB Select input. This gating is done so that the ToCaptureEn, ToShiftEn, and ToUpdateEn outputs are not active when the SIB is not selected. When the host interface is connected to an IEEE 1500 WSP, the ToSelect can be used to gate the SIB ToCaptureEn, ToShiftEn, and ToUpdateEn signals in order to generate the WSP CaptureWR, ShiftWR, and UpdateWR signals for the selected subcore.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

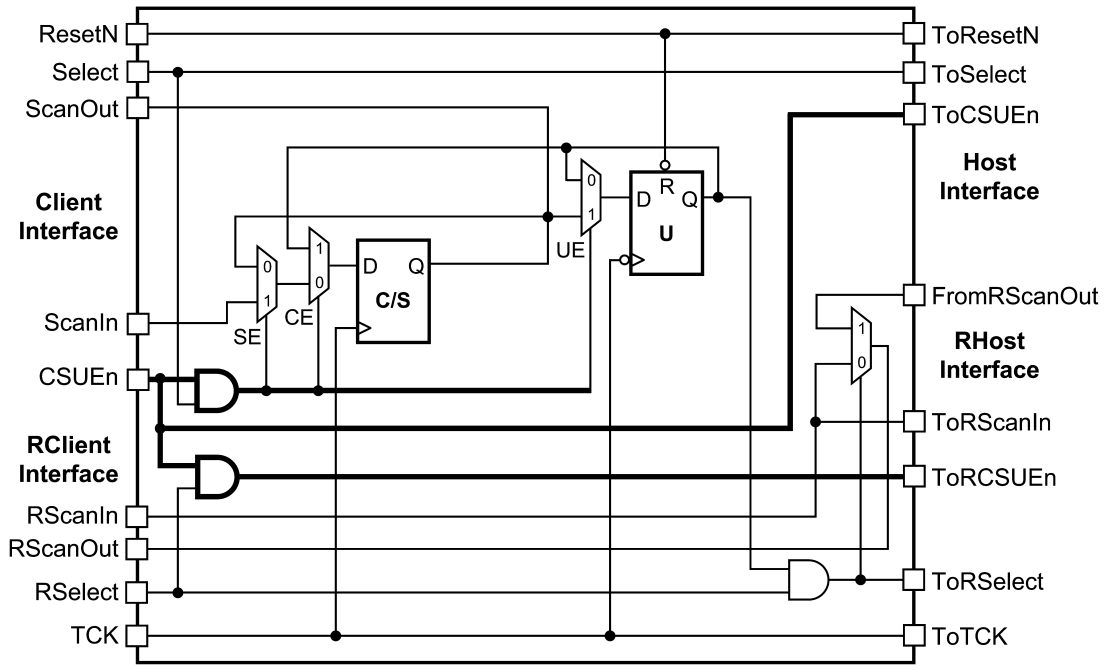


Figure 54 —Example of IEEE 1687 remote SIB

Figure 54 is an example of an IEEE 1687 remote SIB. It has four interfaces: a client interface, a host interface, a remote client interface, and a remote host interface. This operates similar to the inline SIB. The difference is that it has a second remote scan path, through the remote client-host interfaces. The client-host scan path is similar to the inline SIB in Figure 53, using the Select and ToSelect group of signals. The second, or remote scan path, uses the RSelect and ToRSelect group of signals. The CSUEn, ResetN, and TCK are shared for both the SIB path and remote path. The remote scan path is selected by a separate IEEE 1500 instruction from the instruction that is used to select the SIB scan path. Note that the remote SIB does not have a storage element to delay the SIB select. This is because it satisfies rule 7.2.1(e) by using the two separate instructions.

The bolded CSUEn signals in Figure 54 are the respective Capture, Shift, and Update inputs and outputs of the clients and host interfaces. The remote scan path, between RScanIn to RScanOut, can be selected by first updating the SIB to select the remote scan path connected to ToRScanIn and FromRScanout. Then the RSelect input from the decoded remote scan path instruction is asserted to operate the remote path. When the remote scan path is accessed but not selected, RSelect is asserted and the SIB value deselects the remote path, the RScanOut loops back from the RScanIn to bypass the remote scan path. When the remote interface is connected to an IEEE 1500 WSP, the ToRSelect can be used to gate the SIB ToRCaptureEn, ToRShiftEn, and ToRUpdateEn signals in order to generate the WSP CaptureWR, ShiftWR, and UpdateWR signals for the selected subcore.

15.2.2 Specifications

Rules

a) When operating the WSPs of two or more cores selected together in a scan path, the operations shall occur such that the Capture, Shift, and Update operations shall occur concurrently for all WSPs.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Recommendation

b) When operating the WSPs of two or more cores selected together in a scan path, the operations should occur such that the value of the SelectWIR to all WSPs shall be controlled to the same value.

15.2.3 Description

Four examples of WSP configurations are shown in Figure 49, Figure 50, Figure 51, and Figure 52. Multiple WSPs connected in series, multiple parallel WSPs, subcores with inline SIBs, and subcores with remote SIBs are shown in the respective figures. Other WSP configurations are possible.

Recommendation 15.2.2(b) recommends that when cores and subcores are accessed together in a scan path, all selected cores are accessing their WIRs, or their associated data register together. This prevents a selected subcore or core from accessing its WIR, while another other core/subcore is accessing its data registers. Accordingly, data and instruction will not be mixed for the chained register path of the selected cores and subcores.

16. Plug-and-play (PnP)

16.1 Introduction

This clause provides specifications and descriptions for enabling interoperation between multiple IEEE 1500 wrappers at the system chip level.

16.2 Background and definition

PnP is the term used to describe a level of interoperability between different cores' wrappers on the same SoC design. This level of interoperability was settled upon as a compromise between these somewhat differing objectives:

— The architecture should provide for all cores having a uniform test capability utilizing the WSC and standard instructions.

— The architecture should be amenable to a variety of best practices of design and test of cores and SoCs. Core designers and SoC integrators should have flexibility to adopt a variety of implementation styles to meet their system and test design objectives.

— It should be permissible that the resources of the WBR be shared with normal system functionality (shared use).

As IEEE Std 1500 entirely owns the WIR and the WBY, it is a simple matter for this standard to specify those registers so that they can easily interoperate between multiple wrappers. However, specification of the interoperability of the WBRs is complicated by the notion that WBR might be shared with system functionality. This shared use of WBR resources is the primary constraint on achieving PnP. In the case that a core designer exercises the liberty to co-mingle WBR functionality with system functionality, it is recognized that core system clocks might operate the sequential elements in which WBR features reside. In the case that system functionality is separated from wrapper functionality, PnP is assured by design. At the same time, it is recognized that the WBR shall be responsive to WRCK and the other WSC signals to support a common test capability. To resolve such WBR clocking issues and still mandate a uniform level

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

of interoperability, the compromise accepts a limited decoupling of the cause-and-effect relationship between WRCK and events in the WBR.

The governing principle of IEEE 1500 PnP is as follows:

A user of this standard in applying tests on an SoC comprising multiple IEEE 1500 cores via the WSC is assured that the multiple cores' wrappers can interoperate robustly at or below a specified maximum WRCK frequency, provided that WSO is output on the falling edge of WRCK. Robust PnP means that timing margins would improve with decreasing frequency, so below some maximum frequency, interoperability is assured. Note that this allows SoC testing to be accomplished without requiring careful timing of any IEEE 1500 test signals and without requiring knowledge of wrapper timing characteristics.

However, this standard allows WSO(R) to be output on the rising edge of WRCK as well. If the liberty provided by 14.2 rule (d) and permission (i) is exercised, careful timing analysis (e.g., hold time violations) of IEEE 1500 test signals and wrapper timing characteristics is required for interoperability.

Permission 8.2.2(d) is repeated here for convenience:

> The WSP may include AUXCKn terminal(s) in addition to the dedicated WRCK for operation of registers other than the WBY and WIR. Such clocks may be shared with other system clocks and may be used to operate other core features.

Core designers may use AUXCKs of their choice to actuate WSP-directed WBR events. Such use does not change the relationship between any of the other WSC terminals and WRCK, nor does it alter any of the rules governing those relationships. However, it might impose additional operational limitations. For example, one design implementation style that is accommodated by these rules is where WBR sequential elements do not receive WRCK exactly as a clock, but rather as an enable on the data port of the sequential elements. With such an implementation style, WBR operation would depend on adherence to setup and hold time specifications of WRCK with respect to the underlying AUXCK as depicted in Figure 55. The timing parameters shown in this figure are sample parameters representing setup and hold timing of WRCK with respect to the AUXCK for some hypothetical wrapper implementation employing an AUXCK.

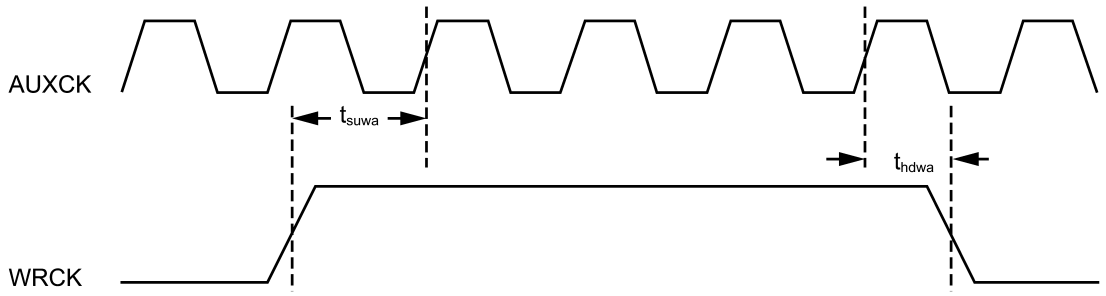


Figure 55—Example of timing relationship specification of WRCK w.r.t. an AUXCK

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

16.3 PnP aspects of standard instructions

16.3.1 Specifications

Rules

a) All standard serial instructions (WS_BYPASS, WS_SAFE, WS_CLAMP, WS_INTEST, WS_INTEST_SCAN, WS_INTEST_RING, WS_PRELOAD, and WS_EXTEST) shall be robustly PnP with respect to the Shift operation of their selected registers. The respective registers of these instructions shall be responsive to ShiftWR, SelectWIR, and WRCK.

b) All wrappers shall be PnP with respect to Capture and Update events of the WSC while executing the WS_EXTEST instruction.

c) Provided that permission 8.2.2(d) is exercised, the timing relationships required between WRCK and the AUXCK(s) shall be specified.

16.3.2 Description

All wrappers shall shift interoperatively when arranged on an SoC with their WSI and WSO(R) terminals serially concatenated. All wrappers interact during Shift operation.

WS_EXTEST is the only standard instruction provided for testing of SoC circuitry that connects to core terminals. It is assumed that the wrappers of multiple cores are simultaneously involved in testing this circuitry. Because of this extensive interaction, WS_EXTEST uniquely has the requirement to be fully PnP for all WSC operations.

WRCK and auxiliary timing relationships required for PnP operation are expected to be provided in CTL per rule 17.4.1(b). A sample representation of such timing relationship is shown in Figure 55.

16.4 PnP limitations on protocols

16.4.1 Specifications

Recommendation

a) PnP WS_EXTEST operation using the WBR should use protocols in which the intended Apply event and the corresponding Capture events do not occur simultaneously.

16.4.2 Description

For a WBR cell to reliably capture a new test response, the PnP definition requires that there be sufficient timing margin. If the Apply event providing a test stimulus and the Capture event for observing a response are triggered by the same edge of the same pulse of WRCK, there might exist a timing race. This can be ameliorated by applying the various events sequentially rather than simultaneously.

16.5 Non-PnP in IEEE Std 1500

Although this standard provides a basic PnP testing capability, there is no intention to inhibit non-PnP uses of this standard. For example, timing-related testing is inherently timing critical, and this testing is

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

facilitated by aspects of this standard such as the allowance for AUXCKs. The user-defined parallel port and user-defined instructions and events are all opportunities for innovation within IEEE Std 1500 that are nonetheless non-PnP.

17. Compliance definitions common to wrapped and unwrapped cores

17.1 Introduction

This clause lists and describes rules that shall be followed to claim compliance with this standard. When used without any qualification, the word *core* applies to both wrapped and unwrapped cores in these rules. Words in **bold** represent CTL (IEEE 1450.6) keywords.

17.2 General rules

17.2.1 Specifications

The rules in this subclause are general rules, not specific to individual terminals, core wrappers, or test patterns.

Rules

a) All hardware-related test information pertinent to core integration shall be provided in CTL, without relying on IEEE 1450 (standard test interface language [STIL]) language extensions or functions.

b) All test data and test protocols pertinent to core integration shall be provided in either CTL or another IEEE TTSC (or successor) sponsored standard test data language.

c) The IEEE 1500 compliance level (wrapped or unwrapped) and version (2022 or 2005) to which the core and associated information apply shall be specified using the **Compliancy** statement: **Compliancy** IEEE1500 <2022 | 2005> <**Wrapped** | **Unwrapped**>.

d) The information related to the length of an existing WBR shall be supplied in CTL using **ScanStructures** {**Length** integer;}. When SIBs are present in the WBR the length specified is the length of the WBR with all segments and all SIBs are open.

e) The information related to the order of the wrapper boundary cells within the register shall be supplied in CTL using **ScanStructures** {**Cells** cell-list;}. When SIBs are present in the WBR, the scan cells listed include all segments of the WBR when all SIBs are open.

f) The scan terminals of the WBR shall be specified using the **ScanDataType** statement: signame { **DataType** ScanDataIn {**ScanDataType** Boundary;} }

g) All state elements of the WBR shall be described in CTL as part of a scan chain using the **ScanStructures** construct of CTL.

Permissions

h) Other IEEE TTSC (or successor) sponsored standard test data languages may be used to express test pattern data and test protocols for any of the test modes provided that CTL is still used to describe all hardware-related test information pertinent to core integration.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

17.2.2 Description

Here is the reasoning behind the general rules:

a) For automation needs, the information delivered by the core provider to the system integrator is provided in CTL (IEEE Std 1450.6) or a combination of CTL and another IEEE TTSC (or successor) sponsored standard test data language. Any use of user-defined mechanisms in the language that are provided for extensibility will limit interoperability. Furthermore, user-defined mechanisms should not be used when keywords already exist to convey the same information. When another test data language is used in combination with CTL, then CTL is used to describe the physical core boundary and its properties. Only the test data for test modes and for executing certain tests within a given test mode can be provided by the other test data language. Test modes can still be described in CTL or in the other test data language.

   As an example, a core provider wants to communicate to the system integrator that there are three test modes in the design: two for testing the internal logic of the design and one for providing access to the wrapper scan chain for testing the external logic of the design. Also, the core provider wants to communicate the fact that the two internal test modes are different configurations that perform the same task. In other words, only one of the patterns in each of the internal test modes needs to be executed to complete the testing of the core. This text would have been sufficient to describe the information; however, it is not easy to automate processing of such text. The following CTL describes this information in an automatable form:

```
Environment {
    CTL config1 {
        TestMode InternalTest {AlternateTestMode config2;}

    }
    CTL config2 {
        TestMode InternalTest {AlternateTestMode config1;}

    }
    CTL config3 {
         TestMode ExternalTest;

    }
}
```

   While the semantics are relevant to the understanding of the example, CTL's syntax is easily readable because of the verboseness of its keywords. Configurations of the design are test modes that are defined in CTL blocks of statements (CTL {}). Each test mode is assigned a type or purpose through the **TestMode** statement. Alternate test modes are identifiable through the **AlternateTestMode** statement. The reader should observe that the information defined in the syntax shown reflects the text that was intended to be described.

b) Test data shall be provided in either CTL or other IEEE TTSC (or successor) sponsored standard test data languages that work well with the integration tools that process such data. Certain test

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

architectures for Intest modes are better expressed in other TTSC test data languages such as IEEE 1687 ICL and PDL.

c) For ease of reference, the CTL shall specify which type of compliance is being provided (IEEE 1500 wrapped, or IEEE 1500 unwrapped) and also an identification of which version of the IEEE Std 1500 was used to define this compliance. This is needed to handle any future, enhanced versions of this standard.

d) The intention is for the high-level information model provided by CTL to allow such pattern generation to be possible without need for access to the core's or even the wrapper's netlist definition. For an unwrapped core, providing full information about an existing WBR (if included) enables integration with other wrapper components. Any core terminals that have IEEE 1500 WBR cells provided with the core shall be correctly identified so that the final integrated WBR can include those cells as well as any additional WBR cells. Also, knowing the correct association between WBR cells within the built-in WBR's bit positions and their corresponding core interface terminals is necessary so that the completed boundary chains can be correctly described for the fully wrapped core.

e) See explanation for rule (d) above.

f) The WBR chains are differentiated from other scan chains of the core using this statement.

g) This information aids in automatically recognizing and operating the existing WBR.

h) This permission allows for the use of other IEEE TTSC (or successor) sponsored standard test data languages while physical attributes of the core boundary should still be described in CTL. See (b).

17.3 Per terminal rules

17.3.1 Specifications

The rules in this subclause are specific to the core's external interface terminals.

Rules

a) All interface terminals of the core shall be identified using the **Signals** block of statements in CTL.

b) All nondigital interface terminals identified for the core shall be classified according to their electrical characteristics using the following statement:

signame {

ElectricalProperty property_type;

}

c) All digital interface terminals of the core shall be categorized according to their test function for all test modes using the following CTL statement:

signame {

DataType (data_type)+;

}

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

d) All digital terminals of IEEE 1500 wrapped or unwrapped cores with corresponding wrapper cells shall be identified in CTL using the following statement:

Internal {

signame {

IsConnected {

Wrapper IEEE1500 CellID cell_type;

}

}

}

e) All digital terminals of IEEE 1500 wrapped or unwrapped cores with no WBR cells or intended to have no WBR cells shall be identified in CTL using the following statement:

signame {

Wrapper None;

}

f) Active states of test terminals needed for the validity of test information of the core shall be specified using the **ActiveState** statement associated with the **DataType** statement as follows:

signame {

DataType data_type {**ActiveState** active_state;}

}

g) Certain terminals such as clock, test mode, and set/reset terminals are assumed to be at a certain state at the beginning of every test protocol for sequences to be valid. That assumption shall be specified using the **AssumedInitialState** statement associated with the **DataType** statement as follows:

signame {

DataType data_type {**AssumedInitialState** assumed_state;}

}

17.3.2 Description

Here is the reasoning behind the per-terminal rules:

a) There are many requirements to accurately refer to the core interface terminals. The test patterns will refer to them, and the core integrator will need to understand which terminals of the core have certain chip-level test interface connection requirements. Labelling the terminals is the first step required to pass along the important information from the core provider through to the core integrator.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

b) CTL allows for the description of property types that include digital, analog, power, and ground terminals. Unless specified, the property type of a terminal is assumed to be digital since IEEE Std 1500 is targeted for digital terminals. Use of this property exempts some terminals of the core from other compliance rules.

c) A core comes with many types of test terminals that are used in special ways. For example, scan-related terminals serve a special function in the operation of a scan chain. These are to be identified using the **DataType** statement in CTL for all the test modes required for the associated compliance level.

d) The correspondence between the WBR cells and the core terminals is identified within the CTL that accompanies the wrapped core.

e) Digital terminals that do not have wrapper cells shall be explicitly stated.

f) While more data types can have active states associated with them, the ones listed in this rule are required to be specified.

g) It is important to know if certain terminals need to be at a specific value at the start of every migrated test, and this is the mechanism used to provide this information.

17.4 Test pattern information rules

17.4.1 Specifications

The rules in this subclause pertain to requirements placed on the test patterns for the core.

Rules

a) All test patterns shall be supplied using the IEEE 1450.6 (CTL) format or another IEEE TTSC (or successor) sponsored standard test data language, and the **Foreign** pattern construct shall be used to reference these TTSC test data languages.

b) All WSC-related critical timing relationships and other design-specific critical timing relationships shall be specified in the CTL **DriveRequirements** and **StrobeRequirements** blocks.

c) When using CTL as the test data language, test patterns shall contain test protocols (**Macro** or **Procedure** blocks) in CTL so that the protocol does not assume that consecutive test patterns are (scan) overlapped.

d) When using CTL as the test data language, patterns used to initialize a test mode shall be separated from the test data patterns and identified as such using the **EstablishMode** keyword.

e) Any terminal that is not included in the supplied test pattern shall not affect the test result.

f) Values on terminals that are exempted from having WBR cells by IEEE Std 1500, with the exception of clocks and test signals, shall not affect the test results.

g) There shall be a sufficient set of patterns supplied with the core to validate every test protocol (**Macro** and **Procedure** blocks) provided in the CTL.

h) Provided that the full internal test pattern set is not supplied with the core (to the core integrator), the complete pattern set shall be made available to the device/chip manufacturer or testing company to use during production test.

Recommendations

i) Fault grading results for all internal patterns and/or pattern-bursts should be provided in CTL including the total number of faults, the number of faults detected, the number of redundant faults,

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

the number of automatic test pattern generation (ATPG) untestable faults, and the number of possibly tested faults.

j) Timing information in the CTL **DriveRequirements** and **StrobeRequirements** blocks should be specified with acceptable margin to allow maximum flexibility for event placement.

17.4.2 Description

Here is the reasoning behind the test pattern information rules:

a) The only acceptable format for describing a core, its physical boundary, and properties is the format defined by IEEE Std 1450.6 (CTL). Other IEEE TTSC (or successor) sponsored standard test data languages can be used to describe test data. This allows for consistent hardware and property description of a core while allowing for other languages used by other standards and flows to provide the generated test data that will need to be integrated into the SOC test data when the core is integrated into the SOC.

b) **DriveRequirements** and **StrobeRequirements** are CTL constructs for specifying timing relationships between the pins of wrapped or unwrapped cores. These shall be provided to help facilitate proper timing operation by the core integrator. Subclause 20.3 shows usage examples of the **DriveRequirements** and **StrobeRequirements** constructs.

c) CTL requires that the data and protocol be separated when used as the test data language. Any test pattern shall use a protocol that is defined in a **Macro** or **Procedure** blocks and identified with a purpose of **DoTest** or **DoTestOverlapped**. CTL allows for test protocols to come in both "flavors." However, when unwrapped cores are provided with **DoTestOverlapped** patterns, the test pattern data that use the protocol need to have redundant parameters to allow for serialization of the parallel values in a **DoTestOverlapped** environment. Although this solution works in CTL, this is not a very streamlined approach.

The problem of overlapped tests might not be fully apparent, so the following example is provided to illustrate why overlapping of test is undesirable.

Suppose an unwrapped core has three patterns that are overlapped. The core contains one scan chain and has three parallel inputs (a, b, and c) and three parallel outputs (x, y, and z). The CTL pattern data might look as follows when the patterns are overlapped:

```
//Overlapped patterns for unwrapped core

//Note: the data values encode the pattern number for convenience in this example.

Pattern overlapped_3 {

P { si=001; so=xxx; abc=001; xyz=001; }

P { si=010; so=001; abc=010; xyz=010; }

P { si=011; so=010; abc=011; xyz=011; }

P { si=000; so=011; abc=000; xyz=xxx; }

}
```

In the above example, parallel stimulus and response data appear together—within the **Macro** application that is associated with that test. Note that the stimulus data are not significant on the final overlapped test, but the STIL syntax does not allow for stimulus of x.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

When the patterns are modified to account for the wrapping of the core, the parallel data will have to be converted to scan data since these data are now applied through the wrapper chain. With this, data that were expected on xyz during the load of pattern number n in the unwrapped core patterns would appear during the load of pattern number n + 1. This requires the unwrapped core patterns to be modified after wrapping and to look as follows:

//Overlapped patterns for wrapped core

Pattern overlapped_3 {

P { si=001; so=xxx; abc=001; xyz=xxx; }

P { si=010; so=001; abc=010; xyz=001; }

P { si=011; so=010; abc=011; xyz=010; }

P { si=000; so=011; abc=000; xyz=011; }

}

In order to avoid this manipulation of the patterns and provide more flexibility during translation, patterns are required without overlapping.

d) Separating out the test setup patterns allows the core integrator to merge the setup patterns for multiple cores to help ensure that all such cores being tested in parallel will enter their respective test modes prior to the application of any tests to any of the cores.

e) If the test pattern associated with a wrapped or unwrapped core does not include some core terminals, the values on these terminals shall not change the test result.

f) If there is one or more core terminals that do not have wrapper cells (perhaps because these terminals are analog inputs) and do not serve a test function as do clocks or scan enable terminals, the test patterns shall not presume any specific values on these pins because it is not possible to ensure that such values can be applied on these pins. Without the control provided by the IEEE 1500 wrapper mechanism, an unwrapped I/O shall be assumed to be at an unknown state.

g) It is important for verification purposes that every migrated pattern works. Since test migration is effectively done by migrating/modifying the protocols without modifying the pattern data, the main tasks needing verification are the updated/migrated protocols.

h) Full pattern sets shall eventually be accessible for production test.

i) Core users want to be able to determine the overall chip test coverage. To be able to compute this they will need to combine the fault counts for all cores and the surrounding application-specific integrated circuit (ASIC) logic. Also, if they cannot apply all test patterns for whatever reason, it is important that they be able to know the effect on test coverage if certain tests are dropped.

j) Because of the generally unknown amount of delay in paths between the chip I/O pins and the internal core pins, it might be difficult to meet very strict timing requirements on test patterns for the core when they are migrated out to the chip pins. Cores can be provided with information that selects a certain working timing for the operations possible on it. An integrator can only try to achieve all the timing specification as depicted by the instance of the timing that is selected for the events. In reality, the core might have some rigid requirements and some flexibility in the timing. This should be shown in the information for the system integrator to successfully incorporate the core for the purposes of test.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

18. Compliance definitions specific to unwrapped cores

18.1 General rules

18.1.1 Specifications

The rules in this subclause pertain to any test logic within the unwrapped cores that will need to be understood prior to attempting to add or complete the wrapping of the core. Words in **bold** represent CTL (IEEE 1450.6) keywords.

Rules

a) All core terminals assigned to share functional registers with the IEEE 1500 wrapper shall be provided with a complete WBR cell, which includes the functional registers.

b) All core terminals that cannot be wrapped without logical modification to the unwrapped core shall have a WBR cell built into the core.

c) Full control of any embedded WBR(s) shall be provided from core terminals.

d) If the state of the core relies on the stability of certain core input terminals during the scan Shift operation of the embedded environment, this information shall be specified using the **InputProperty** statement as follows:

signame {

InputProperty ScanStable;

}

e) If the core provider determines the state of the core environment relies on stability of certain core output terminals during the internal test operation of the core, this information shall be specified in CTL.

f) Every core shall come with at least one definition of an internal test mode for the core in CTL.

18.1.2 Description

Here is the reasoning behind the test logic information rules:

a) A functional I/O register can be reused as part of a WBR cell provided that the full WBR cell is implemented within the core.

b) Since it is not feasible to change the logic already built into a hard core or a black-box core, its unwrapped version shall have WBR cells built in to be compliant with this standard. For example, a core output driven by a tristate driver should have wrapper cells on both the data and enable inputs to the tristate driver. A wrapper cell cannot be added to the output of a tristate driver.

c) The existing WBR shall be fully controllable from the core's terminals so that it can be included as part of the core WBR when the core is wrapped with an IEEE 1500 wrapper.

d) If the test patterns for the core depend on certain terminals remaining stable during scan operations for the wrapper and in any logic surrounding the core, the core integrator needs to know this because it has implications on how the embedded core is connected to the TAM. The input property **ScanStable** specifies this requirement.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

e) Any signal made stable to prevent damaging effect on other signals shall be described using this statement. This would be identified by the following:

signame {

OutputProperty ScanStable;

DisableState ExpectOff;

IsDisabledBy Macro scan_macro_name

}

f) At least one core internal test mode shall be fully defined in CTL so that test patterns for testing the core itself can be migrated to the SoC for that test mode.

18.2 Per terminal rules

18.2.1 Specifications

The rules in this subclause pertain to core interface terminals that require wrapping or that are already wrapped.

Rules

a) The preferred wrapper cell type for every core terminal that is to be wrapped shall be described in CTL using the **ConnectTo** wrapper statement.

18.2.2 Description

Here is the reasoning behind the per-terminal rules:

a) For a wrapper to be automatically created for an unwrapped core, information shall be provided to help the wrapper generator understand which type of wrapper cell should be used for each pin of the core. The **External** block in the CTL construct used to specify this information as follows:

signal_name {

ConnectTo{

Wrapper IEEE1500 **CellID** cell_enum**;**

}}

The term *cell_enum* represents a WBR cell name as defined in 12.7.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

18.3 Additional test information rules

18.3.1 Specifications

Rules

a) Any existing quiet or powered-down mode shall be defined in CTL using the **TestMode** keyword.

Recommendations

b) If a safe mode exists, its associated safe values shall be described in CTL using the following construct: **DataType** data_type {**ActiveState** active_state;}.

18.3.2 Description

Here is the reasoning behind the additional test information rules:

a) To perform I_{DDQ} testing for the SoC, it is important to avoid high-current conditions within the design. Each embedded core should be placed into a quiet mode to reduce current drain during such testing. By specifying a quiet test mode, this simplifies the I_{DDQ} test generation process for the core integrator. Not providing a quiet mode definition might result in the core being placed into a state that consumes more current than would otherwise be necessary.

b) It is important that no damage be done to a core while testing of other cores or surrounding logic is being performed. To help ensure this damage cannot happen, a core safe state should be defined (even for cores that have no unsafe states).

19. Compliance definitions specific to wrapped cores

19.1 General rules

19.1.1 Specifications

The rules in this subclause apply to all IEEE 1500 wrapped cores and are not specific to individual terminals. Words in **bold** represent CTL (IEEE 1450.6) keywords.

Rules

a) IEEE 1500 wrapper architecture and operation shall be compliant to all IEEE 1500 hardware rules described in this standard.

b) The information related to the length of the WIR, implemented public instructions, and their corresponding opcodes shall be provided in CTL.

   1) The WIR is to be defined using the **ScanStructure** construct.

   2) The scan cells composing the WIR are to be identified using the **ScanDataType** construct.

   3) The protocol operating the WIR is to be defined as **Instruction** using the **Purpose** construct.

   4) Instructions and their opcodes are to be defined using the **TestModeForWrapper** construct.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

c) All IEEE 1500 wrapper terminals shall be described in CTL.

d) Available information about the safe state of input and output terminals shall be provided for all possible test operating modes.

Recommendations

e) The number of (tester) cycles needed to execute the test patterns should be specified for all the patterns of the core.

19.1.2 Description

Here is the reasoning behind the general rules for wrapped cores:

a) This rule reiterates that all the wrapper logic shall conform to the requirements and behaviors of the IEEE 1500 wrapper architecture as described in other clauses of this standard.

b) All wrapper-supported public instructions shall be defined and described using CTL to help ensure they can be utilized by standard software. Because instructions are considered test modes in CTL, the **TestModeForWrapper** construct is to be used to define instructions.

c) All wrapper pins shall be described accurately in CTL so standard software can understand how to integrate the core into surrounding logic. Since the netlist for the wrapper might not be provided, accurate descriptions of all wrapped pins are necessary for software to understand how to test the logic connected to each wrapped pin.

d) In order for the core integrator to know how to prevent unwanted conflicts from occurring during test, it is important to note the safe state values required on core/wrapper terminals.

e) The ability to schedule pattern application tasks on the SoC relies on this information being provided. Without this information, optimized scheduling of the tests would be difficult.

19.2 Per terminal rules

19.2.1 Specifications

The rules in this subclause pertain to specific external interface terminals of the wrapped core.

Rules

a) All IEEE 1500 terminals that are used to operate the wrapper shall be identified with the following statement:

signame {

Wrapper IEEE1500 PinID pin_type;

}

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

19.2.2 Description

Here is the reasoning behind the per-terminal rules:

a) By explicitly specifying the function of special IEEE 1500 wrapper control terminals, there is no requirement that these terminals need to have specific pin names at the wrapped core boundary. The pin_type field is expected to be a member of the WSP terminal list described in 8.1.2.

19.3 Wrapper protocol information rules

19.3.1 Specifications

Rules

a) The protocol that operates the WIR through a scan operation to establish the test mode shall shift values into all state elements of the WIR.

Recommendations

b) The Capture, Shift, and Update events of the protocol that operates the WIR should be identified in CTL for the wrapped core using the **Identifiers** syntax or as a purpose of the protocol.

19.3.2 Description

Here is the reasoning behind the wrapper protocol information rules:

a) Partial loads of the WIR are disallowed as they cause problems when multiple cores are integrated. A typical integration step would daisy-chain all the WIRs into a single scan chain. If the protocol that operates the WIRs does not shift the complete daisy-chained set of WIRs, the synchronized scan operation to establish the modes of the top-level scan chain of the SoC might not work.

b) A typical integration of wrapped cores would daisy-chain the WIRs of all the wrapped cores. To allow for this form of connection, the activities of the WIRs need to be synchronized. Since there is only one possible protocol at this time, this is a recommended practice.

20. IEEE 1500 application

20.1 Introduction

This clause is informative only. Words in **bold** represent CTL (IEEE 1450.6) keywords.

20.2 CTL (IEEE 1450.6) overview

IEEE Std 1450.6 defines the CTL. A CTL description of an IEEE 1500 wrapper is required, so a brief overview of CTL is included in IEEE Std 1500 to explain the pertinent connection between the two standards.

CTL is a language for capturing and expressing test-related information for reusable cores, which is meant to co-exist with and complement information expressed as a netlist. It is utilized to describe IEEE 1500

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

wrappers. Test data information about a core can be captured in CTL or provided in another IEEE TTSC (or successor) sponsored standard test data language so that the SoC integrator or automation tools can successfully create a complete test for the SoC. Using the CTL description of a core, a wrapper can be constructed, and the appropriate TAM can be determined based on the test constraints in the CTL description. Once all the structures are in place, the test patterns can be translated from the core boundary to the SoC boundary. CTL is the language to support all the information that the core provider needs to give the system integrator so that the latter can successfully test the embedded core and any UDL around the core. This language is broad enough to describe cores on which IEEE 1500 wrappers are to be implemented, IEEE 1500 wrapped cores, their different test methodologies, and the different ways in which they are integrated in the SoC.

When CTL is also used for the test pattern data, one of the most important requirements for CTL is that the patterns, which contain the bulk of the data, are reusable without any modification whatsoever. This is accomplished by creating patterns using protocol statements (P statements) as opposed to vector statements (V statements) as used in traditional STIL (IEEE Std 1450). This allows the vector application protocol to be modified by the core integrator in an expeditious manner. Each pattern is identified by its intended purpose and associated test mode, so that a test synthesis tool can select and reorder as desired, again, without having to adjust the bulk pattern data.

CTL is both human and computer readable, as is STIL. Hence, it can be utilized for documentation purposes, as well as for driving chip test integration tools.

Every mode of the core can have a mechanism to initialize the mode. The initialization sequences are described in CTL on a per-mode basis. Some modes of the core contain test patterns with their associated timing information, constraints, and statistics. Some modes contain structural information so that the structure can be used to create patterns at another level of integration of the design. CTL is designed to describe all these needs of different modes of the core. For any given test mode of a core, typically only a subset of the full CTL syntax should be required to adequately describe the attributes of the mode.

At the basic level, **Signals** and **SignalGroups** are defined with their attributes. The following text explains CTL keywords. For complete definitions, refer to IEEE Std 1450.6.

Signals: This block defines each of the signal names of the core. Attributes can be attached to signals for additional information, e.g., **ScanIn** and **ScanOut** length, indicating the length of the scan chain connected to the signal.

SignalGroups: Signals can be clustered into signal groups. Attributes can also be attached here. A signal can occur in multiple signal groups, each time having different attributes.

Timing: Each parallel or scan vector in a pattern macro has an associated timing block to define the waveform and corresponding timing on each signal.

Pattern: This block contains the parallel and scan data for testing the core. Since pattern data are typically the bulk of the CTL file, they are often split off into a separate file.

MacroDefs: This block contains the protocol for applying test pattern data to the core or chip. Protocols for patterns provided with the core are written at the core boundary. Once integrated, this protocol is expanded to the chip level. The actual patterns are not changed in this process, so they still reference the same macro name, but in fact a new, expanded macro definition has replaced the core-level definition.

PatternBurst: This collection of patterns represents a schedule, and they are to be executed on the core. Patterns can be run serially or in parallel.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

ScanStructures: This block can be used to describe in detail the internal scan chains of a core. In addition to the scan-in and scan-out port, a list of scan cells and clocking information is specified using the **ScanStructures** block.

Environment: This block is the top-level construct in CTL and is used to define the test modes for the core. Information in each test mode is categorized in subblocks such as the **Internal**, **External**, and **PatternInformation** blocks.

Internal: This block is used to describe the internal characteristics of the core signals. This information is provided to allow the core integrator to know the pertinent test information for each terminal of the core without needing full access to the design information. Examples of this information include wrapper type, timing accuracy required, and electrical characteristics such as analog or digital.

External: This block is used to describe the external characteristics that are expected from the perspective of the core boundary. Examples include connect to chip pins (input, output, bidirectional), connect to another named core, and connect to TAM, and connect to UDL.

PatternInformation: This block defines the purpose of each of the test patterns provided and the test mode necessary for the execution of each pattern. Other information like the fault model used and achieved coverage can also be given.

20.3 IEEE 1500 examples

20.3.1 Introduction

A core and wrapper are depicted in Figure 56. Note that, whereas this figure shows some implementation details, IEEE Std 1500 defines only the behavior of the wrapper and leaves the implementation open. In this particular example, the wrapper can connect Core A both to a serial TAM, via WSI and WSO(R), as well as to a 3-bit parallel TAM, via WPI[0:2] and WPO[0:2]. There are six instructions that are described by the figures in this subclause, supporting normal mode and test modes with connections to either serial or parallel TAMs. Mode setting through the WIR means setting the controls of the wrapper's multiplexers. In Figure 60, the multiplexers that do occur in the wrapper are named m1, m2, ... m12. The dark circles in the multiplexers show which path is enabled if the multiplexer control signal is set to 1.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

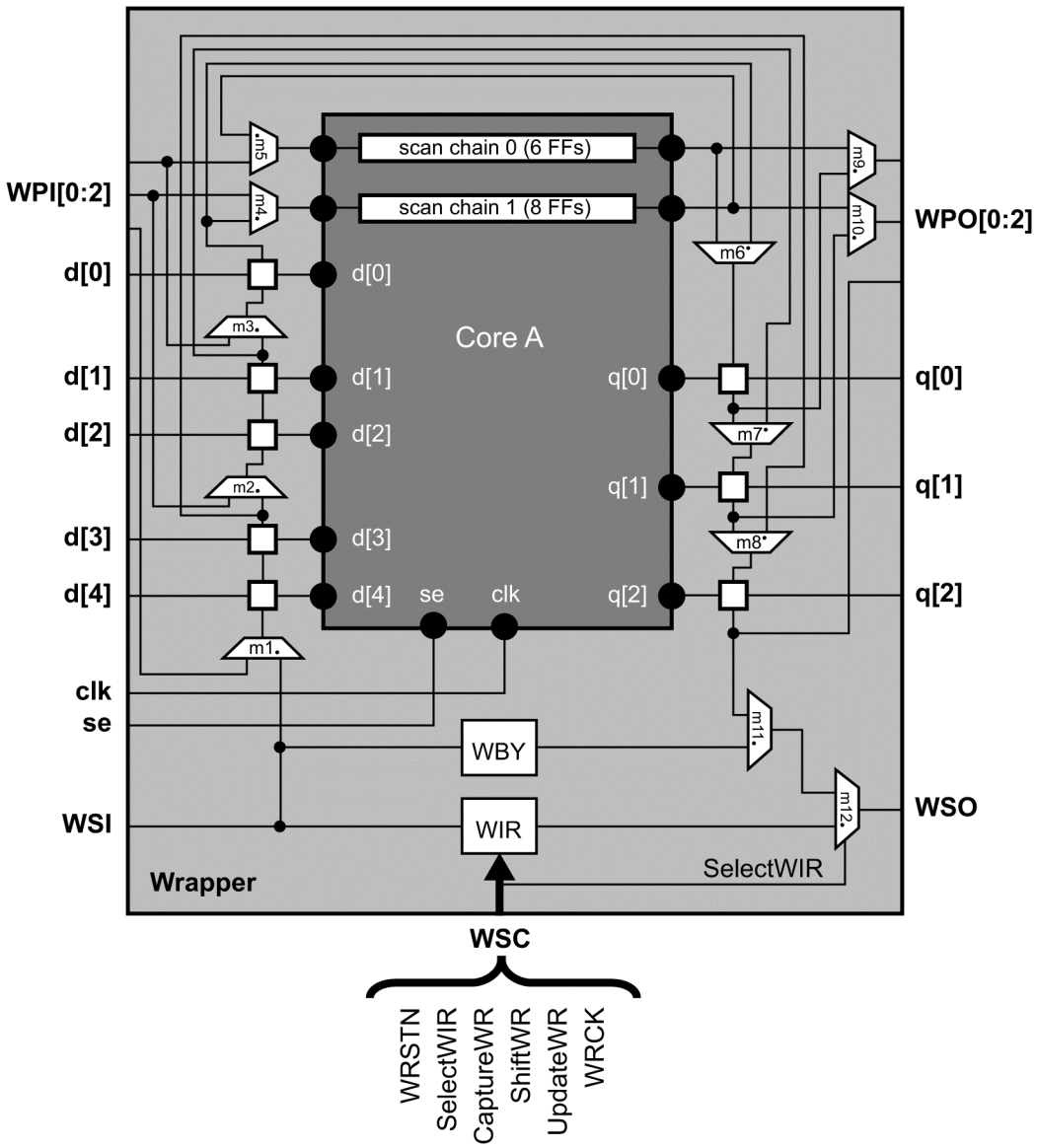


Figure 56—Example Core A and its IEEE 1500 wrapper

Table 6 lists the six instructions and the corresponding multiplexer settings they cause.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Table 6—Instruction decoding for multiplexer for the wrapper of Core A

Mode	Instruction	Opcode	Multiplexer settings											
			m1	m2	m3	m4	m5	m6	m7	m8	m9	m10	m11	m12
Normal	WS_BYPASS	0000	X	X	X	X	X	X	X	X	X	X	1	0
Serial bypass	WS_BYPASS	0000	X	X	X	X	X	X	X	X	X	X	1	0
Serial intest	WS_INTEST_ SCAN	1010	1	1	1	1	1	0	0	0	X	X	0	0
Serial extest	WS_EXTEST	1001	1	1	1	X	X	1	0	0	X	X	0	0
Parallel intest	WP_INTEST	1110	0	1	1	0	0	1	0	0	0	0	X	X
Parallel extest	WP_EXTEST	1101	0	0	0	X	X	1	1	1	1	1	X	X

In Figure 57 through Figure 60, the six modes are shown by bold highlighting of the active wires in a particular mode. The CTL description is included for three of the instructions: WS_BYPASS, WS_INTEST_SCAN, and WS_EXTEST.

20.3.2 WS_INTEST_SCAN

Figure 57 depicts the wrapper in its serial internal test mode in which the wrapper and internal scan chains are concatenated into one long scan chain. Following Figure 57 is the CTL code that describes the wrapper configuration and test for this. It is anticipated that the user should refer to IEEE Std 1450.6 for details on the CTL constructs shown in the code. This example is the full CTL, showing the test modes and test patterns for this core.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

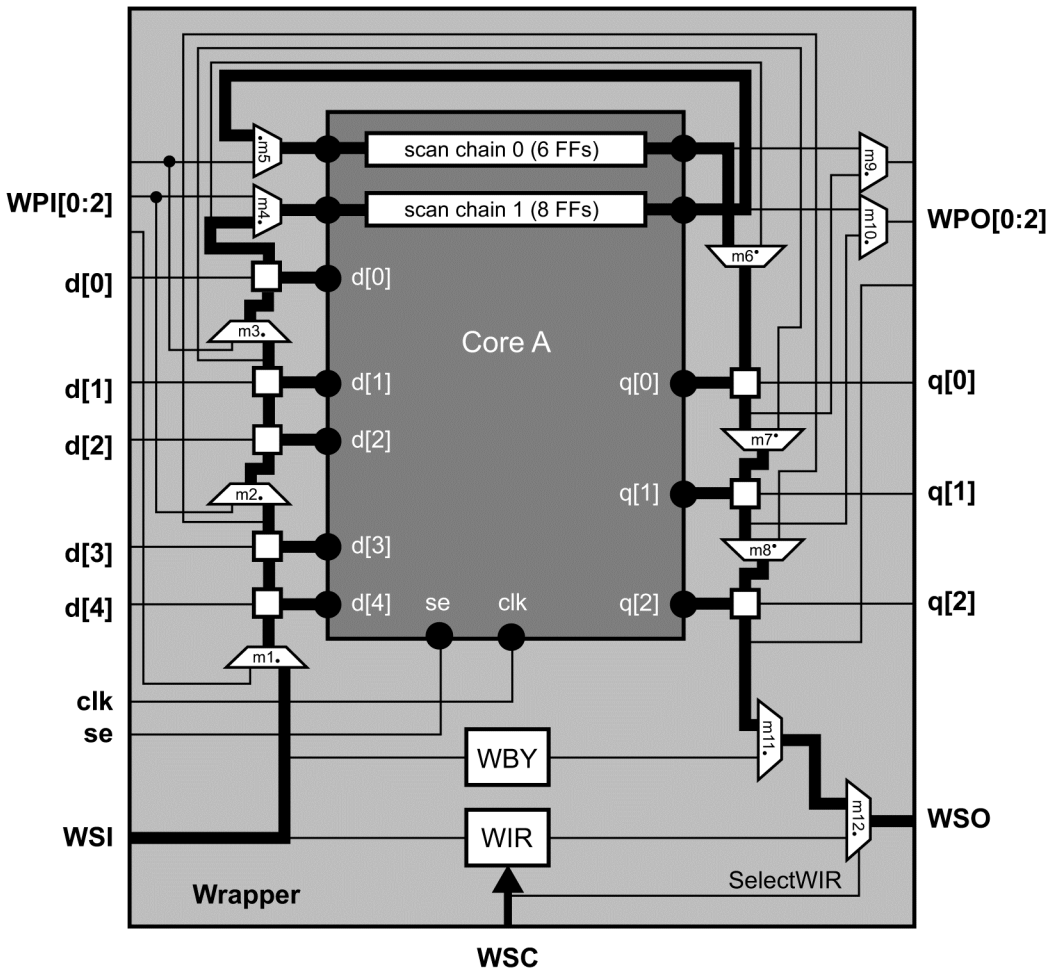


Figure 57—Serial intest mode using WS_INTEST_SCAN

```
STIL 1.0 {
      Design 2005;2
      CTL 2005;
}
Header {
      Title "Serial Intest";
      Date "2005";
}
Signals {
      WSI In;
      se In {DefaultState D;}
      clk In {DefaultState D;}
      d[0..4] In;
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
      WPI[0..2] In;
      WPO[0..2] Out;
      q[0..2] Out;
      WSO Out;
      WRSTN In {DefaultState U;}
      SelectWIR In {DefaultState D;} C
      aptureWR In {DefaultState D;}
      ShiftWR In {DefaultState D;}
      UpdateWR In {DefaultState D;}
      WRCK In {DefaultState D;}
}
SignalGroups {
      si_m123 = 'WSI' {ScanIn 22;}
      si_wir = 'WSI' {ScanIn 4;}
      so_wir = 'WSO' {ScanOut 4;}
      so_m123 = 'WSO' {ScanOut 22;}
      wsp[0..5] = 'WRSTN+SelectWIR+CaptureWR+ShiftWR+UpdateWR+WRCK';
}
Variables {
      SignalVariable si[0..1];
      SignalVariable so[0..1];
      SignalVariable instruction [0..3];
}

ScanStructures {
      ScanChain wir_chain {
            ScanLength 4;
            ScanIn WSI;
            ScanOut WSO;
            ScanCells wcell[0..3];
            ScanMasterClock WRCK;
      }
}


Timing {
      WaveformTable default_WFT {
             Period '100ns';
             Waveforms {
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                'wsp[0..5]+WPI[0..2]+d[0..4]+clk+se+WSI' {
                      01X {'5ns' D/U/N;}
                }
                'clk+WRCK' {
                      P {'0ns' D; '45ns' U; '55ns' D;}
                }
                'WPO[0..2]+q[0..2]+WSO' {
                      01X {'0ns' X; '95ns' L/H/X;}
                }
          }
    }
}

MacroDefs {
    setup_intest {
          W default_WFT;
          V {WRSTN=0;}
          V {wsp[0..5] = 110100;}
          C {instruction=#;}
          Shift {V {si_wir= \W instruction[0..3]; WRCK=P;}}
          V {si_wir=X; WRCK=0; ShiftWR=0; UpdateWR=1;}
          V {WRCK=P;}
          V {WRCK = 0; SelectWIR=0; UpdateWR = 0;}
    }
    do_intest {
          W default_WFT;
          C {se=1; ShiftWR=1; si=#; so=#;}
          Shift {V {si_m123= \W si[1] \W si[0] \W d[0..4];
                clk=P; WRCK=P;}}
          V {WRCK=P; ShiftWR=0; UpdateWR=1;}

          V {clk=0; WRCK=0; se=0; CaptureWR=1; ShiftWR=0;}
          V {clk=P; WRCK=P;}
          V {CaptureWR=0; clk=0; WRCK=0;}
          C {se=1; ShiftWR=1;}
          Shift {V {so_m123= \W q[2..0] \W so[1] \W so[0]; clk=P;
                WRCK=P;}}
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
      }
}
PatternBurst all_pats {
       PatList {
            initSerialInternalTestMode {Protocol Macro setup_intest;}
            pat1 {Protocol Macro do_intest};
      }
}
PatternExec run_all { PatternBurst all_pats;
}
Environment wrapped_core {
      CTL {
           Internal {
                 WRSTN {
                       Wrapper IEEE1500 PinID WRSTN;
                       DataType Asynchronous TestControl Reset {
                             ActiveState D;
                                   AssumedInitialState U;
                       }
                 }
                 SelectWIR {
                       Wrapper IEEE1500 PinID SelectWIR;
                       DataType TestControl TestWrapperControl;
                 }
                 CaptureWR {
                       Wrapper IEEE1500 PinID CaptureWR;
                       DataType TestControl {
                             ActiveState U;
                       }
                 }
                 ShiftWR {
                       Wrapper IEEE1500 PinID ShiftWR;
                       DataType TestControl ScanEnable {
                             ActiveState U;
                       }
                 }
                 UpdateWR {
                       Wrapper IEEE1500 PinID UpdateWR;
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                DataType TestControl {
                      ActiveState U;
                }
          }
          WRCK {
                Wrapper IEEE1500 PinID WRCK;
                DataType TestControl TestClock ScanMasterClock {
                      AssumedInitialState D;
                }
                DriveRequirements {
                      TimingSensitive {
                            Period Min '50ns';
                            Pulse High Min '10ns';
                                  }
                }
          }
     }
      External {
          'clk+se+wsp[0..5]' {ConnectTo {DataType In;}}
           WSI {ConnectTo {DataType In;}
                ConnectTo {Wrapper IEEE1500 PinID WSO;}}
           WSO {ConnectTo {DataType Out;}
                ConnectTo {Wrapper IEEE1500 PinID WSI;}}
      }

CTL serial_internal_test_mode {
      TestMode InternalTest;
      Compliancy IEEE1500 2022 Wrapped;
      TestModeForWrapper WS_INTEST_SCAN 1010;
      Internal {
            'd[0..4]+q[0..2]+WPI[0..2]+WPO[0..2]' {
                  DataType Unused;}
       clk {


DataType TestControl
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
CaptureClock ScanMasterClock
      {AssumedInitialState D;}
DriveRequirements {
      TimingSensitive {
            Period Min '50ns';
            Pulse High Min '10ns';
                        }
            }
            }

            se {DataType TestControl ScanEnable {ActiveState U;}
                  DriveRequirements {
                        TimingSensitive {
                              Reference clk {
                                    ReferenceEdge Leading;
                                    Setup '5ns'; Hold '5ns';
                              }

                        }
                  }
            }

            si_m123 {DataType TestData ScanDataIn
                        {ScanDataType Boundary Internal;}
                  DriveRequirements {
                        TimingSensitive {
                              Reference WRCK {
                                    ReferenceEdge Leading;
                                    Setup '5ns'; Hold '5ns';
                              }
                        }
                  }

            }

            so_m123 {DataType TestData ScanDataOut
                        {ScanDataType Boundary Internal;}
                  StrobeRequirements {
                        TimingSensitive {
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                                    Reference WRCK {
                                          ReferenceEdge Trailing;
                                          EarliestTimeValid '5ns';
                                    }
                                    }
                              }
                              LaunchClock WRCK {TrailingEdge;}
                        }
                  }
                  PatternInformation {
                        PatternExec run_all {
                              Purpose Production;
                              PatternBurst all_pats;
                              Fault {
                                    Type StuckAt;
                                    FaultCount 1092;
                                    FaultsDetected 999;
                              }
                        }
                        Pattern initSerialInternalTestMode {
                               Purpose EstablishMode;
                        }
                        Macro setup_intest {
                              Purpose ModeControl;
                              UseByPattern EstablishMode;
                        }
                        Macro do_intest {
                              Purpose DoTest;
                        }
                  }
            }
      }

      Pattern initSerialInternalTestMode {
            P {instruction[0..3]=1010;}
      }
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
Pattern pat1 {
      P {d[0..4]=00000; si[0]=111000; si[1]=11110000;
            so[0]=001X11; so[1]=111100X1; q[0..2]=001;}
      P {d[0..4]=01101; si[0]=011010; si[1]=01011101;
            so[0]=1100X1; so[1]=10110000; q[0..2]=110;}
      P {d[0..4]=11001; si[0]=110010; si[1]=00011100;
            so[0]=010001; so[1]=1X110100; q[0..2]=00X;}
      P {d[0..4]=01010; si[0]=001101; si[1]=10011101;
            so[0]=11X000; so[1]=101110XX; q[0..2]=011;}
      P {d[0..4]=01111; si[0]=010001; si[1]=10101101;
            so[0]=0X1111; so[1]=00001X10; q[0..2]=000;}
}
```

The following example is the minimal CTL just used to describe this core in this test mode, without test protocols or test patterns. Test protocols and test data is provided in a separate pattern file.

```
STIL 1.0 {
      Design 2005;
      CTL 2005;
}
Header {
      Title "Serial Intest";
      Date "2005";
}
Signals {
      WSI In;
      se In {DefaultState D;}
      clk In {DefaultState D;}
      d[0..4] In;
      WPI[0..2] In;
      WPO[0..2] Out;
      q[0..2] Out;
      WSO Out;
      WRSTN In {DefaultState U;}
      SelectWIR In {DefaultState D;}
      CaptureWR In {DefaultState D;}
      ShiftWR In {DefaultState D;}
      UpdateWR In {DefaultState D;}
      WRCK In {DefaultState D;}
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
}
SignalGroups {
      si_m123 = 'WSI' {ScanIn 22;}
      si_wir = 'WSI' {ScanIn 4;}
      so_wir = 'WSO' {ScanOut 4;}
      so_m123 = 'WSO' {ScanOut 22;}
      wsp[0..5] = 'WRSTN+SelectWIR+CaptureWR+ShiftWR+UpdateWR+WRCK';
}
Variables {
      SignalVariable si[0..1];
      SignalVariable so[0..1];
      SignalVariable instruction [0..3];
}

ScanStructures {
      ScanChain wir_chain {
            ScanLength 4;
            ScanIn WSI;
            ScanOut WSO;
            ScanCells wcell[0..3];
            ScanMasterClock WRCK;
      }
}


Timing {
      WaveformTable default_WFT {
            Period '100ns';
            Waveforms {
                  'wsp[0..5]+WPI[0..2]+d[0..4]+clk+se+WSI' {
                        01X {'5ns' D/U/N;}
                  }
                  'clk+WRCK' {
                        P {'0ns' D; '45ns' U; '55ns' D;}
                  }
                  'WPO[0..2]+q[0..2]+WSO' {
                        01X {'0ns' X; '95ns' L/H/X;}
                  }
            }
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
      }
}

Environment wrapped_core {
      CTL {
           Internal {
                 WRSTN {
                       Wrapper IEEE1500 PinID WRSTN;
                       DataType Asynchronous TestControl Reset {
                             ActiveState D;
                             AssumedInitialState U;
                       }
                 }
                 SelectWIR {
                       Wrapper IEEE1500 PinID SelectWIR;
                       DataType TestControl TestWrapperControl;
                 }
                 CaptureWR {
                       Wrapper IEEE1500 PinID CaptureWR;
                       DataType TestControl {
                           ActiveState U;
                       }
                 }
                 ShiftWR {
                       Wrapper IEEE1500 PinID ShiftWR;
                       DataType TestControl ScanEnable {
                          ActiveState U;
                       }
                 }
                 UpdateWR {
                       Wrapper IEEE1500 PinID UpdateWR;
                       DataType TestControl {
                           ActiveState U;
                        }
                 }
                 WRCK {
                      Wrapper IEEE1500 PinID WRCK;
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                DataType TestControl TestClock ScanMasterClock {
                      AssumedInitialState D;
                }
          }
      }
      External {
          'clk+se+wsp[0..5]' {ConnectTo {DataType In;}}
          WSI {ConnectTo {DataType In;}
              ConnectTo {Wrapper IEEE1500 PinID WSO;}}
          WSO {ConnectTo {DataType Out;}
              ConnectTo {Wrapper IEEE1500 PinID WSI;}}
      }

CTL serial_internal_test_mode {
      TestMode InternalTest;
      Compliancy IEEE1500 2022 Wrapped;
      TestModeForWrapper WS_INTEST_SCAN 1010;
      Internal {
            'd[0..4]+q[0..2]+WPI[0..2]+WPO[0..2]' {
                      DataType Unused;}
            clk {
                DataType TestControl
                CaptureClock ScanMasterClock
                {AssumedInitialState D;}
                DriveRequirements {
                      TimingSensitive {
                            Period Min '50ns';
                            Pulse High Min '10ns';
                                                }
                 }
            }

            se {DataType TestControl ScanEnable {ActiveState U;}
                  DriveRequirements {
                        TimingSensitive {
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                        Reference clk {
                              ReferenceEdge Leading;
                              Setup '5ns'; Hold '5ns';
                        }
                  }
            }
      }

      si_m123 {DataType TestData ScanDataIn
            {ScanDataType Boundary Internal;}
            DriveRequirements {
                  TimingSensitive {
                        Reference WRCK {
                              ReferenceEdge Leading;
                              Setup '5ns'; Hold '5ns';
                        }
                  }
            }

      }

      so_m123 {DataType TestData ScanDataOut
            {ScanDataType Boundary Internal;}
            StrobeRequirements {
                  TimingSensitive {
                        Reference WRCK {
                              ReferenceEdge Trailing;
                              EarliestTimeValid '5ns';
                        }
                   }
            }
            LaunchClock WRCK {TrailingEdge;}

      }
}
External {
      'clk+se+wsp[0..5]' {ConnectTo {DataType In;}}
      WSI {ConnectTo {DataType In;}
            ConnectTo {Wrapper IEEE1500 PinID WSO;}}
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                WSO {ConnectTo {DataType Out;}
                      ConnectTo {Wrapper IEEE1500 PinID WSI;}}
            }
            PatternInformation {
                FileName "other_pattern_file_name";
                ForeignPatterns {
                    BlockName TestName;
                }
        }
        FileReference "other_pattern_file_name";
}
```

20.3.3 WS_BYPASS

Note that both the multiplexer settings in Table 6 AND the highlighted active wires in Figure 58 show that the functional and serial bypass modes are nonconflicting and hence can be established by one combined instruction, WS_BYPASS. Following Figure 58 is the CTL code that describes the wrapper configuration and test.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

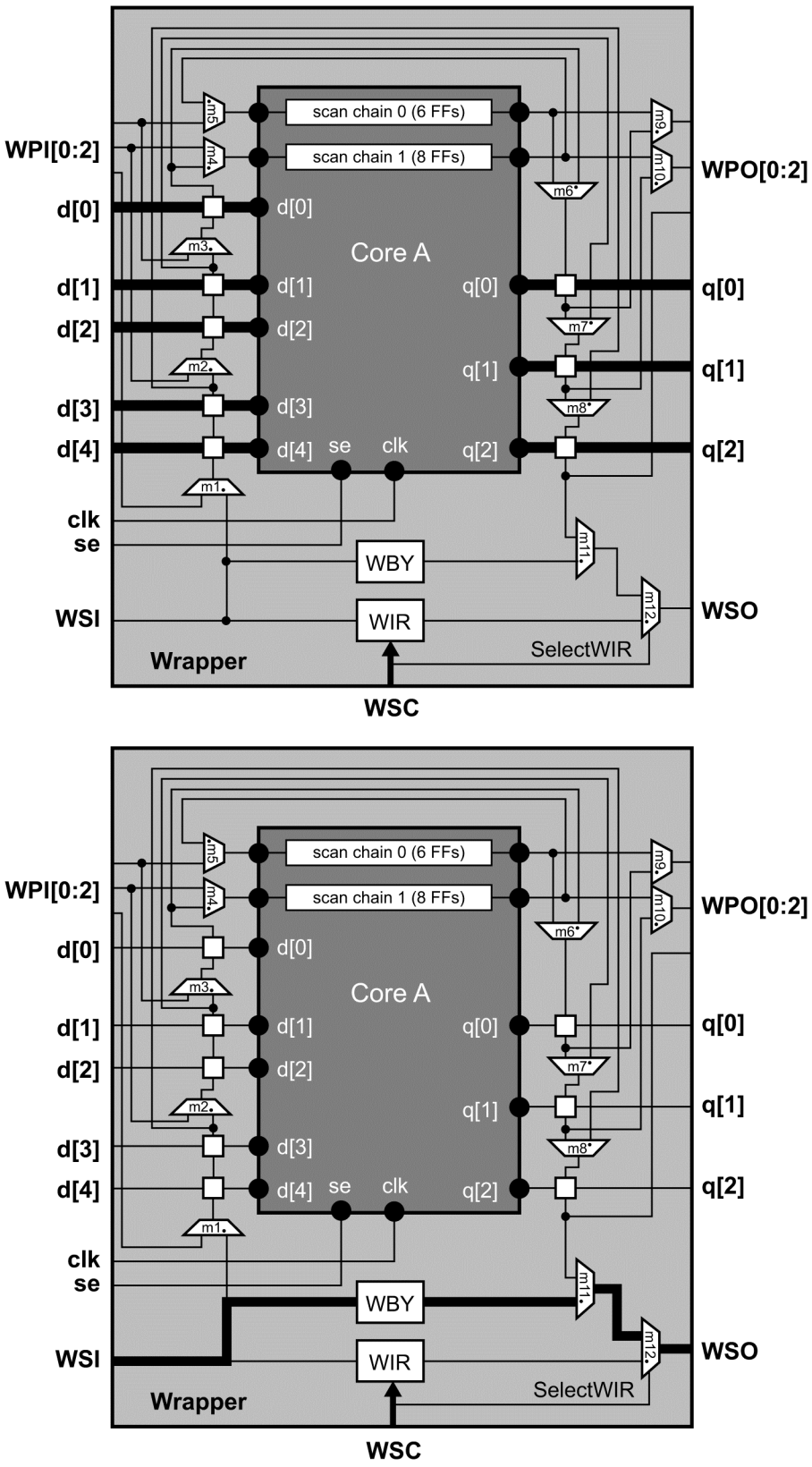


Figure 58 —WS_BYPASS configuration

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
STIL 1.0 {
      Design 2005;
      CTL 2005;
}
Header {
      Title "Bypass Mode - WS_BYPASS";
      Date "2005";
}
Signals {
      WSI In;
      se In {DefaultState D;}
      clk In {DefaultState D;}
      d[0..4] In;
      WPI[0..2] In;
      WPO[0..2] Out;
      q[0..2] Out;
      WSO Out;
      WRSTN In {DefaultState U;}
      SelectWIR In {DefaultState D;}
      CaptureWR In {DefaultState D;}
      ShiftWR In {DefaultState D;}
      UpdateWR In {DefaultState D;}
      WRCK In {DefaultState D;}
}
SignalGroups {
      si_byp = 'WSI' {ScanIn 1;}
      si_wir = 'WSI' {ScanIn 4;}
      so_byp = 'WSO' {ScanOut 1}
      wsp[0..5] = 'WRSTN+SelectWIR+CaptureWR+ShiftWR+UpdateWR+WRCK';
}
Variables {
            SignalVariable instruction[0..3];
}
MacroDefs bypass_mode_macros {
      operateScanChain {
            W default_WFT;
            C {ShiftWR=1; WRCK=0;}
            Shift {V {WSI=#; WSO=#; WRCK=P;}}
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
      }
}
ScanStructures bypass_mode_chains {
      ScanChain bypassChain {
            ScanIn WSI;
            ScanOut WSO;
            ScanLength 1;
            ScanCells c0;
            ScanMasterClock WRCK;
      }
}
ScanStructures {
      ScanChain wir_chain {
      ScanLength 4;
      ScanIn WSI;
      ScanOut WSO;
      ScanCells wcell[0..3];
      ScanMasterClock WRCK;
      }
}

Timing {
      WaveformTable default_WFT {
            Period '100ns';

            Waveforms {
                  'wsp[0..5]+WPI[0..2]+d[0..4]+clk+se+WSI' {
                        01X {'5ns' D/U/N;}
                  }
                  'clk+WRCK' {
                        P {'0ns' D; '45ns' U; '55ns' D;}
                  }
                  'WPO[0..2]+q[0..2]+WSO' {
                        01X {'0ns' X; '95ns' L/H/X;}
                  }
            }
      }
}
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
MacroDefs {
      setup_testmode {
            W default_WFT;
            V {WRSTN=0;}
            V {wsp[0..5] = 110100;}
            Shift {V {si_wir= \W instruction [0..3]; WRCK=P;}} V
            {si_wir=X; WRCK=0; ShiftWR=0; UpdateWR=1;}
            V {WRCK=P;}
            V {WRCK = 0; SelectWIR=0; UpdateWR = 0;}
            }
}
Environment wrapped_core {
      CTL {
                  WRSTN {
                        Wrapper IEEE1500 PinID WRSTN;
                        DataType Asynchronous TestControl Reset {
                              ActiveState D;
                              AssumedInitialState U;
                        }
                  }
                  SelectWIR {
                        Wrapper IEEE1500 PinID SelectWIR;
                        DataType TestControl TestWrapperControl;
                  }
                  CaptureWR {
                        Wrapper IEEE1500 PinID CaptureWR;
                        DataType TestControl {
                              ActiveState U;
                        }
                  }
                  ShiftWR {
                        Wrapper IEEE1500 PinID ShiftWR;
                        DataType TestControl ScanEnable {
                              ActiveState U;
                        }
                  }
                  UpdateWR {
                        Wrapper IEEE1500 PinID UpdateWR;
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                DataType TestControl {
                    ActiveState U;
                }
            }
            WRCK {
                Wrapper IEEE1500 PinID WRCK;
                DataType TestControl TestClock ScanMasterClock {
                    AssumedInitialState D;
                }
            }
        }
        External {
        'clk+se+wsp[0..5]' {ConnectTo {DataType In;}}

        WSI {ConnectTo {DataType In;}
            ConnectTo {Wrapper IEEE1500 PinID WSO;}}
        WSO {ConnectTo {DataType Out;}
            ConnectTo {Wrapper IEEE1500 PinID WSI;}}
        }
}
CTL bypass_test_mode {
        TestMode Bypass Normal;
        Compliancy IEEE1500 2022 Wrapped;
        TestModeForWrapper WS_BYPASS 0000;
        DomainReferences {
            ScanStructures bypass_mode_chains;
            MacroDefs bypass_mode_macros;
        }
        Internal {
            'd[0..4]+q[0..2]+se' {DataType Functional;}
            WPI[0..2]+WPO[0..2]' {DataType TestData Unused;}
            clk {
                DataType TestControl MasterClock
                    {AssumedInitialState D;}
                DriveRequirements { TimingSensitive {
                    Period Min '50ns';
                    Period Max '100ns';
                }}
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
            }
            WSI {DataType TestData ScanDataIn
                  {ScanDataType ByPass;}}
            WSO {DataType TestData ScanDataOut
                  {ScanDataType ByPass;}}
      }
      PatternInformation {
            Pattern initByPass TestMode{
                  Purpose EstablishMode;
                  Protocol Macro setup_testmode;
            }
            Macro setup_testmode{
                  Purpose ModeControl;
                  UseByPattern EstablishMode;
            }
            Macro operateScanChain{
                  Purpose ControlObserve;
                  ScanChain bypassChain;
            }
      }
  }
}
Pattern initByPass_TestMode {
      P {instruction[0..3]=0000;}
}
```

20.3.4 WS_EXTEST

Figure 59 depicts the wrapper in its serial external test mode. Following Figure 59 is the CTL code that describes the wrapper configuration and test for this.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

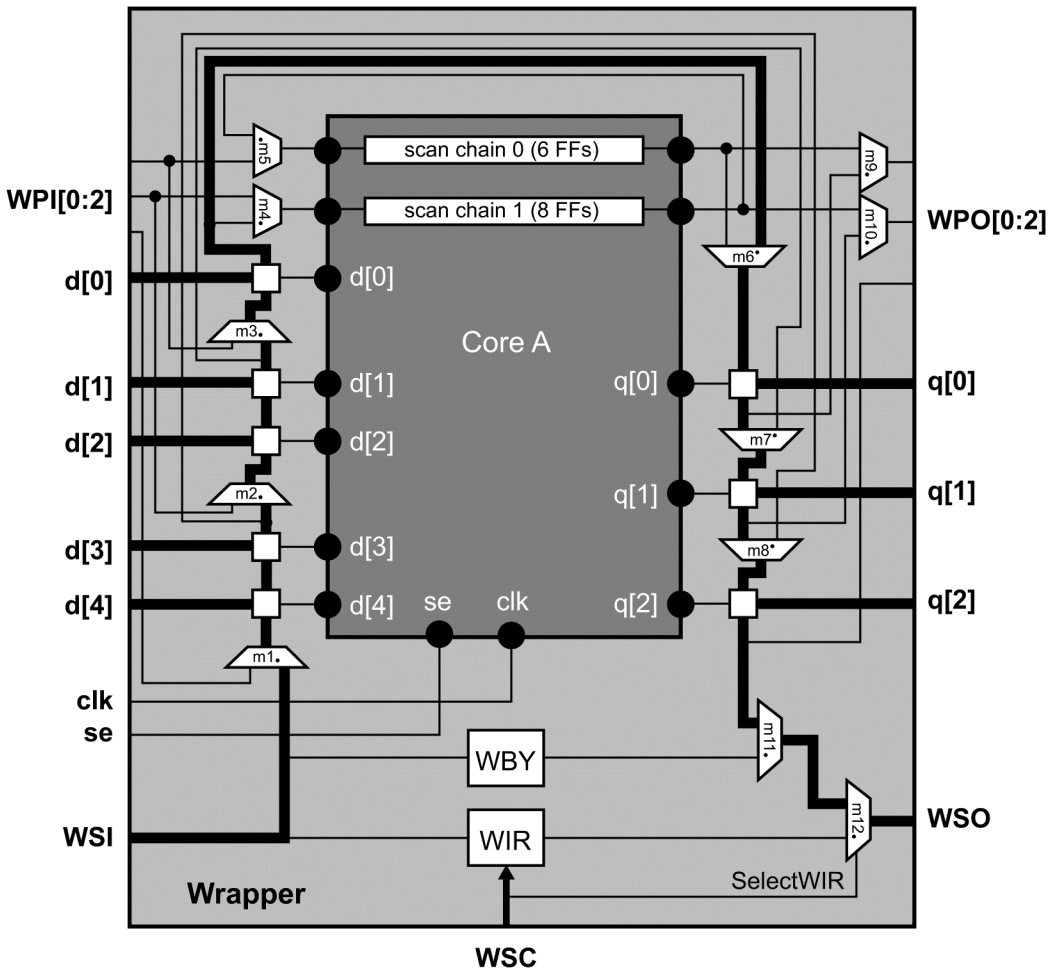


Figure 59—WS_EXTEST configuration

```
STIL 1.0 {
      Design 2005;
      CTL 2005;
}
Header {
      Title "Extest Mode - WS_EXTEST";
      Date "2005";
}
Signals {
      WSI In;
      se In {DefaultState D;}
      clk In {DefaultState D;}
      d[0..4] In;
      WPI[0..2] In;
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
      WPO[0..2] Out;
      q[0..2] Out;
      WSO Out;
      WRSTN In {DefaultState U;}
      SelectWIR In {DefaultState D;}
      CaptureWR In {DefaultState D;}
      ShiftWR In {DefaultState D;}
      UpdateWR In {DefaultState D;}

      WRCK In {DefaultState D;}
}
SignalGroups {
      si_ext = 'WSI';
      si_wir = 'WSI';
      so_ext = 'WSO';
      wsp[0..5] = 'WRSTN+SelectWIR+CaptureWR+ShiftWR+UpdateWR+WRCK';
}
Variables {
           SignalVariable instruction[0..3];
}
MacroDefs extest_mode_macros {
      operateScanChain {
            W default_WFT;
            C {ShiftWR=1; WRCK=0;}
            Shift {V {WSI=#; WSO=#; WRCK=P;}}
      }
}
ScanStructures extest_mode_chains {
      ScanChain wrapperChain {
            ScanIn WSI;
            ScanOut WSO;
            ScanLength 8;
            ScanCells c[0..7];
            ScanMasterClock WRCK;
      }
}
Timing {
      WaveformTable default_WFT {
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
            Period '100ns';
            Waveforms {
                  'wsp[0..5]+WPI[0..2]+d[0..4]+clk+se+WSI' {
                        01X {'5ns' D/U/N;}
                  }
                  'clk+WRCK' {
                        P {'0ns' D; '45ns' U; '55ns' D;}
                  }
                  'WPO[0..2]+q[0..2]+WSO' {
                        01X {'0ns' X; '95ns' L/H/X;}
                  }
            }
      }
}
MacroDefs {
      setup_testmode{
            W default_WFT;
            V {WRSTN=0;}
            V {wsp[0..5] = 110100;}
            Shift {V {si_wir= \W instruction[0..3]; WRCK=P;}}
            V {si_wir=X; WRCK=0; ShiftWR=0; UpdateWR=1;}
            V {WRCK=P;}
            V {WRCK = 0; SelectWIR=0; UpdateWR = 0;}
            }
}
Environment wrapped_core {
      CTL {
             Internal {
                  WRSTN {
                        Wrapper IEEE1500 PinID WRSTN;
                        DataType Asynchronous TestControl Reset {
                            ActiveState D;
                            AssumedInitialState U;
                        }
                  }

                  SelectWIR {
                        Wrapper IEEE1500 PinID SelectWIR;
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
                DataType TestControl TestWrapperControl;
            }
            CaptureWR {
                Wrapper IEEE1500 PinID CaptureWR;
                DataType TestControl {
                    ActiveState U;
                }
            }
            ShiftWR {
                Wrapper IEEE1500 PinID ShiftWR;
                DataType TestControl ScanEnable {
                    ActiveState U;
                }
            }
            UpdateWR {
                Wrapper IEEE1500 PinID UpdateWR;
                DataType TestControl {
                    ActiveState U;
                }

            }
            WRCK {
                Wrapper IEEE1500 PinID WRCK;
                DataType TestControl TestClock ScanMasterClock {
                    AssumedInitialState D;
                }
            }
        }
        External {
            'clk+se+wsp[0..5]' {ConnectTo {DataType In;}}
            WSI {ConnectTo {DataType In;}
                ConnectTo {Wrapper IEEE1500 PinID WSO;}}
            WSO {ConnectTo {DataType Out;}
                ConnectTo {Wrapper IEEE1500 PinID WSI;}}
        }
}
CTL extest_test_mode {
        TestMode ExternalTest;
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
Compliancy IEEE1500 2022 Wrapped;
TestModeForWrapper WS_EXTEST 1001;
DomainReferences {
      ScanStructures extest_mode_chains;
      MacroDefs extest_mode_macros;
}
Internal {
      'd[0..4]' { DataType Functional TestData;
            IsConnected In {
                  StateElement Scan 'c[4..0]';
                  CaptureClock WRCK {
                        LeadingEdge;
                        StateAfterClock Connection;
                       }
                  }
      }
      'q[0..2]' {DataType Functional TestData;
            IsConnected Out {
                  StateElement Scan 'c[5..7]';
                  LaunchClock WRCK {
                         LeadingEdge;
                        StateAfterClock ExpectUnknown;
                  }
            }
      }


      se {DataType Functional Unused;}
      'WPI[0..2]+WPO[0..2]' {DataType
      TestData Unused;} clk {
            DataType TestControl MasterClock
            UseByPattern EstablishMode;
      }
      Macro operateScanChain{
            Purpose ControlObserve;
            ScanChain wrapperChain;
      }
}
```


Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

```
        }
}
Pattern initExtest_TestMode {
        P {instruction[0..3]=1001;}
}
```

20.3.5 WP_EXTEST and WP_INTEST

Figure 60 illustrates the configuration of the wrapper for the parallel extest and parallel intest modes.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

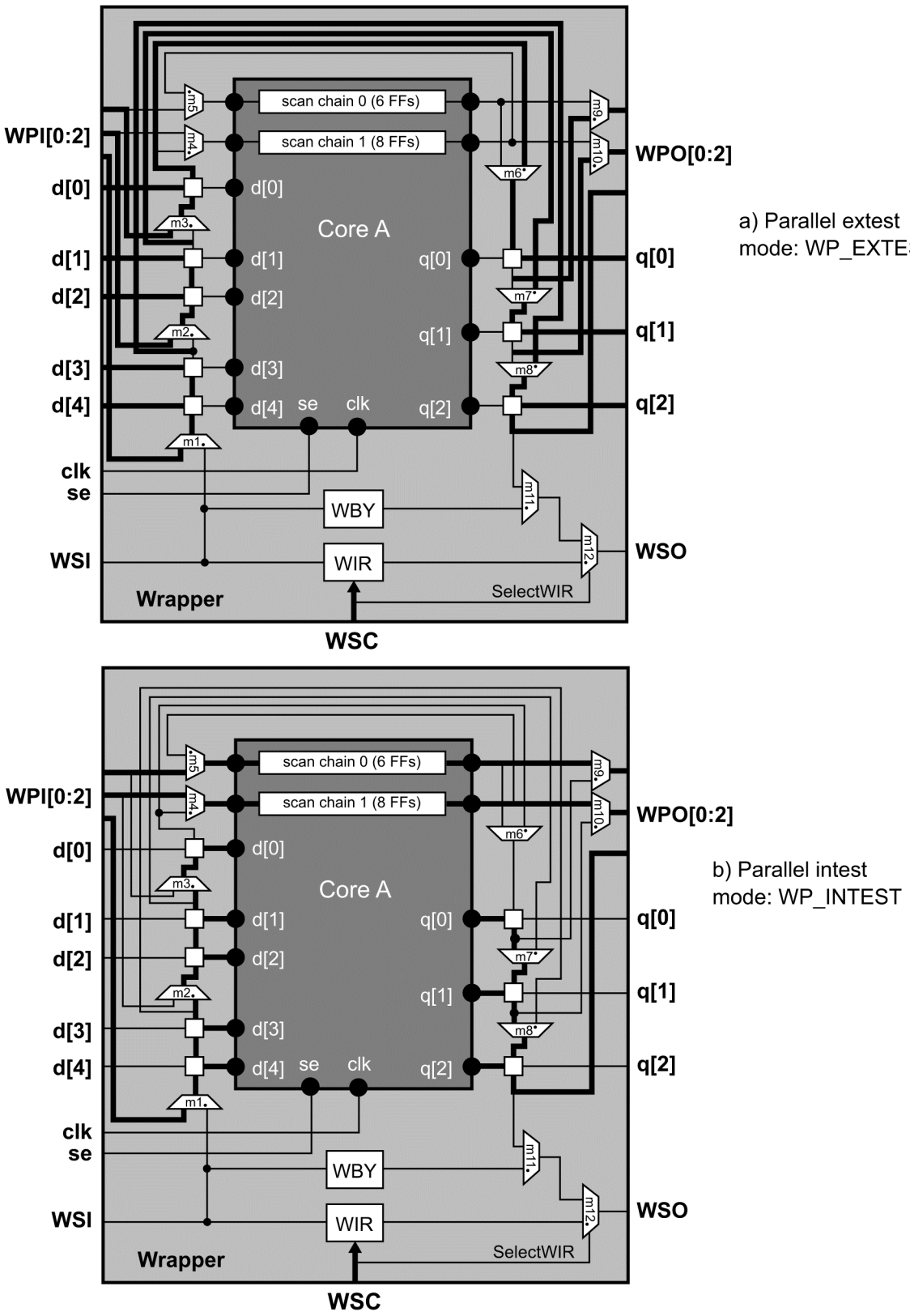


Figure 60 —Parallel extest and parallel intest modes

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Annex A

(informative)

Bubble diagram definition

In IEEE Std 1500, an attempt has been made to focus on architecture and remain neutral to implementation styles. In this spirit, a new graphical vocabulary has been implemented to describe hardware behavior visually without describing the physical implementation of the hardware. Informally, diagrams drawn using these symbols have been called *bubble diagrams* because the much-used symbol for a storage element is a circle.

The symbols to describe behavior are few. In fact, there are only five as shown in Figure A.1. The first is the symbol for a storage element, which is a circle. The second is a data path, which is represented as a line with an arrowhead indicating data flow direction. The third is a dataflow decision point, which is represented as a vertical line with no arrowhead, wherein the decision is determined by the contents of the wrapper instruction register (WIR). The fourth symbol is a data flow decision point, which is only used in segment insertion bit (SIB) cells and represented by a chevron of approximately 120 degrees, wherein the decision is determined by the value stored in the SIB's update storage element. The fifth symbol is a connection point, represented as a small, filled circle located at the junction of two datapaths.

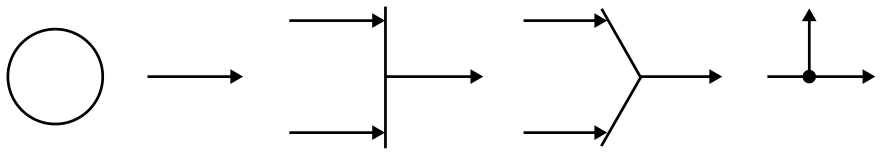


Figure A.1—The five symbols used in bubble diagrams

The storage element has a characteristic associated with it that indicates the events to which it responds. This characteristic is displayed as the presence of one or more characters inside the circle from the set S, C, U, and F indicating the Shift, Capture, Update, and any functional event respectively. Figure A.2 shows various storage elements. A double U (UU) indicates an update storage element coupled with a delayed version used in some SIBs as per rule 7.2.1(f).

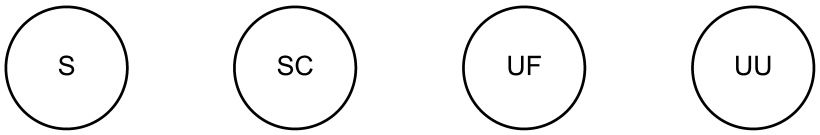


Figure A.2—Various storage elements

Shift data flows from cell test input (CTI) to cell test output (CTO) and can go through multiple storage elements represented by the bubbles. Capture data come into a storage element from the CFI terminal or the CFO terminal. Update data come from the shift path storage element closest to CTO. Functional data come from CFI.

An IEEE 1149.1 BC_1 cell using the IEEE 1500 cell notation is shown in Figure A.3. The BC_1 cell supports the Shift, Capture, Update, and Apply events. Note that the Apply event is a virtual event, composed of other events or modes, and, therefore, is not specifically represented in these diagrams.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

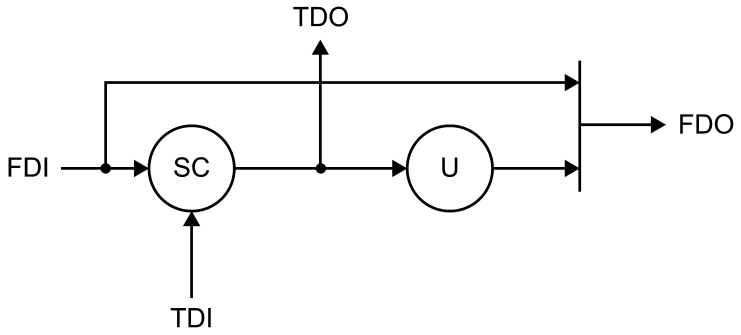


Figure A.3—Example of an IEEE 1149.1 BC_1 cell

An IEEE 1687 SIB_mux_pre cell using the IEEE 1500 bubble diagram scheme is shown in Figure A.4. The An IEEE 1687 SIB_mux_pre cell supports Shift, Capture, Update, and Apply events as well as a delayed Update in compliance with rule 7.2.1(f).

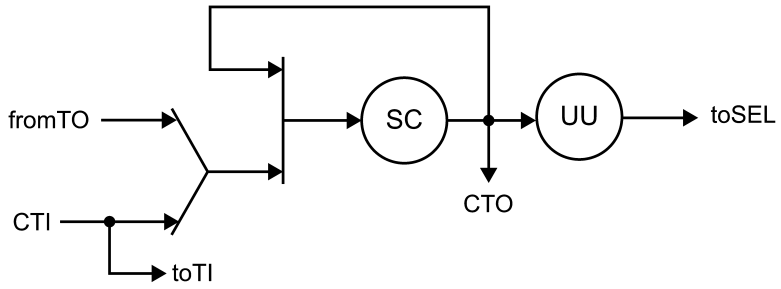


Figure A.4—Example of an IEEE 1687 SIB_mux_pre cell

For further examples of usage, refer to Annex B where various bubble diagrams are reduced to implementation examples.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Annex B

(informative)

Wrapper boundary register (WBR) cell examples

Table B.1 describes IEEE 1500 WBR cell example names, and a gallery of bubble diagrams depicting each example follows the table. The bubble diagrams used in these examples are defined in Annex A. It is understood that the means of data path selection shown in these bubble diagram figures is to be configured by the content of the WIR.

Table B.1—WBR cell examples

Cell description	Name	Figure number
One storage element in shift path dedicated to wrapper function	WC_SD1_CI1	Figure B.1
Two storage elements in the shift path with one shared with functional operation	WC_SF2_CI2	Figure B.2
Two dedicated storage elements in the shift path	WC_SD2_CO1	Figure B.3
One dedicated storage element in the shift path and a dedicated update storage element	WC_SD1_CI1_UD	Figure B.4
Two dedicated storage elements in the shift path and a shared update storage element	WC_SD2_CB1_UF	Figure B.5
n dedicated storage elements in the shift path and a dedicated update storage element	WC_SDn_CI1_UD	Figure B.6
A reduced-functionality cell performing control-only function with one dedicated storage element in the shift path	WC_SD1_CN	Figure B.7
One dedicated storage element in the shift path, selectively capturing from cell functional input (CFI) or cell functional output (CFO), and a dedicated update storage element	WC_SD1_CBI_UD	Figure B.8

WC_SD1_CI1 has the advantage of a dedicated shift path, however, depending on the instruction, the CFO terminal might toggle during certain events, e.g., Shift or Capture. The connection between CFI and CFO could be difficult to test. WC_SD1_CI1_G can be forced to a safe value in addition to supporting the functionality described for WC_SD1_CI1. See Figure B.1.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

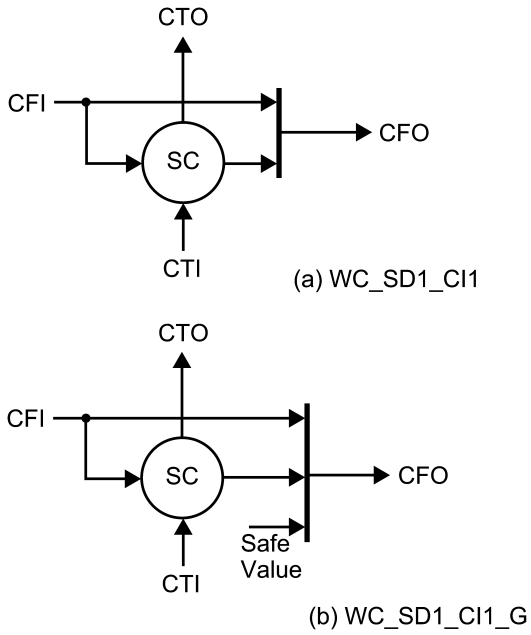


Figure B.1—WC_SD1_CI1 WBR cells

WC_SF2_CI2 is similar to WC_SD2_CI2 except that functional operation and test operation share the second sequential element from cell test input (CTI). This cell can be used to meet a test objective in which the capture and apply paths are identical. WC_SF2_CI2_G can be forced to a safe value in addition to supporting the functionality described for WC_SF2_CI2. See Figure B.2.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

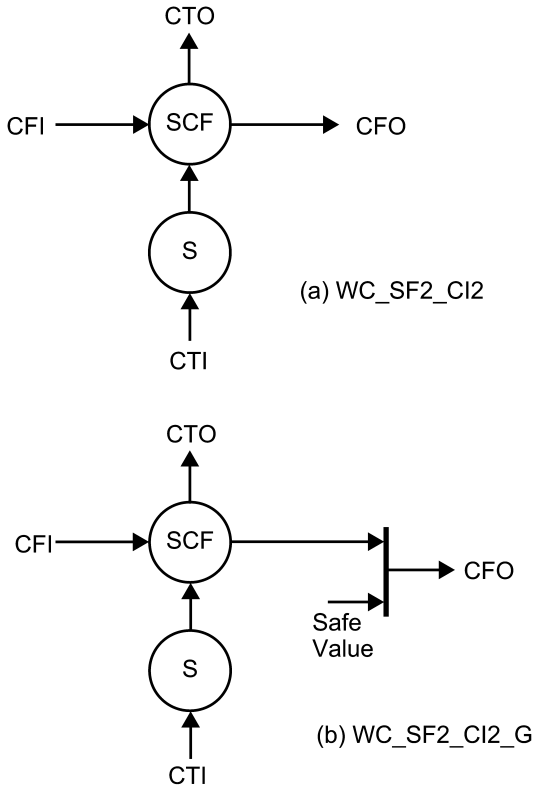


Figure B.2—WC_SF2_CI2 WBR cells

WC_SD2_CO1 and WC_SD2_CO1_G capture from the CFO and additionally support a transition test application. See Figure B.3.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

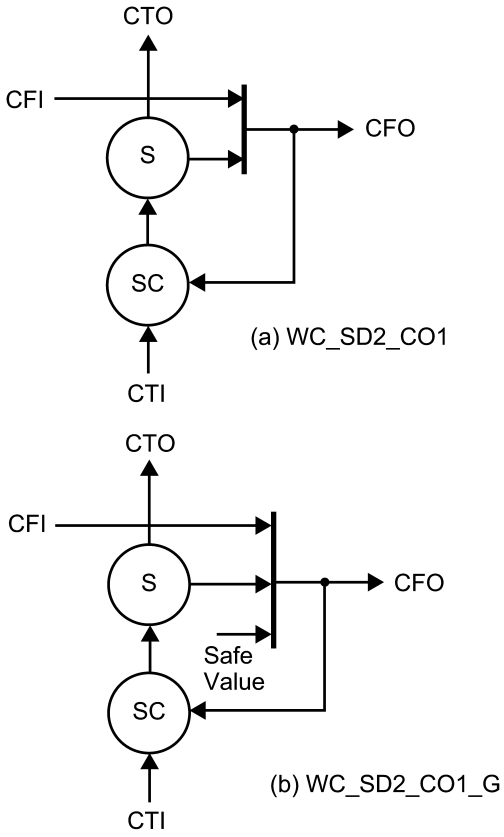


Figure B.3—WC_SD2_CO1 WBR cells

WC_SD1_CI1_UD corresponds to an IEEE 1149.1 BC_1 cell. It has dedicated shift path and a dedicated update storage element. WC_SD1_CI1_UD_G can be forced to a safe value in addition to supporting the functionality described for WC_SD1_CI1_UD. See Figure B.4.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

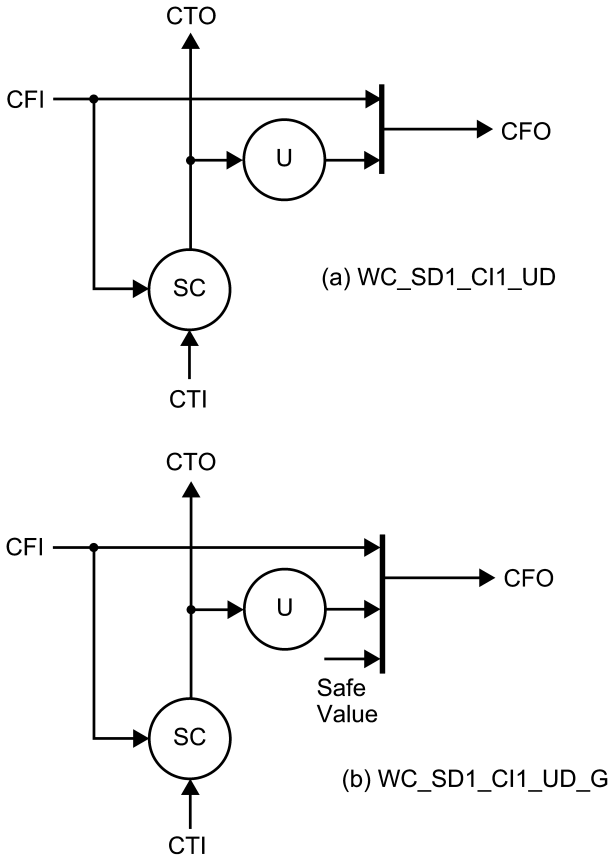


Figure B.4—WC_SD1_CI1_UD WBR cells

WC_SD2_CB1_UF and WC_SD2_CB1_UF_G support the same functionality as the WC_SD1_CB1_UF and WC_SD1_CB1_UF_G, respectively, except that additionally they have two shift path storage elements. See Figure B.5. WC_SD1_CB1_UF and WC_SD1_CB1_UF_G differ in that they have only one dedicated storage element in the shift path with the shared update storage element.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

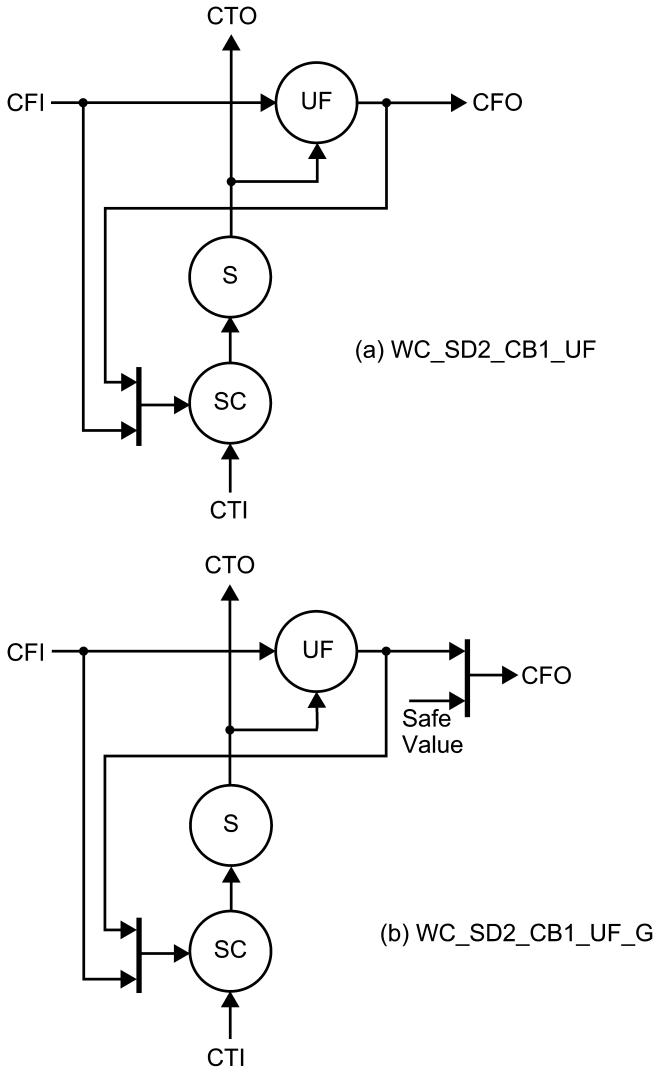


Figure B.5—WC_SD2_CB1_UF WBR cells

WC_SDn_CI1_UD and WC_SDn_CI1_UD_G support the same functionality as the WC_SD1_CI1_UD and WC_SD1_CI1_UD_G, respectively, except that additionally they have two or more shift path storage elements and, therefore, can be used for transition test applications. Similar to the BC_1, the capture sites are the storage elements closest to CTI. See Figure B.6.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

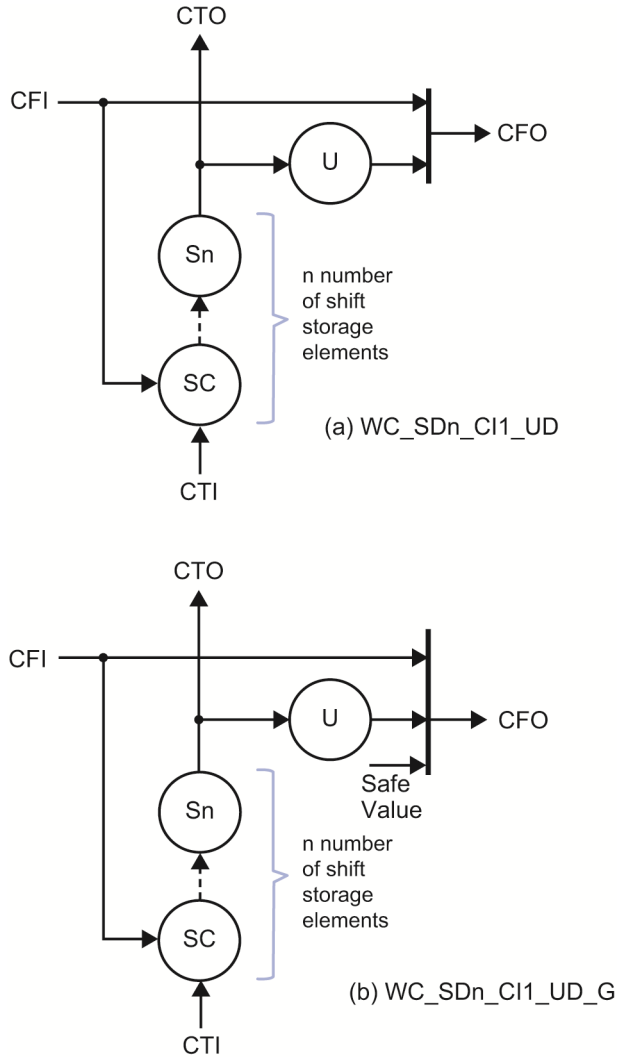


Figure B.6—WC_SDn_CI1_UD WBR cells

WC_SD1_CN is a control-only harnessing cell. This cell is to be used only in accordance with permission 12.3.2(g). See Figure B.7.

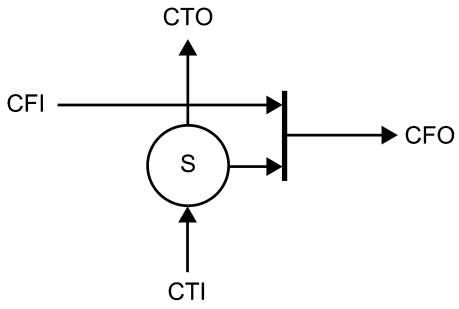


Figure B.7—WC_SD1_CN WBR cell

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

WC_SD1_CB1_UD and WC_SD1_CB1_UD_G support the same functionality as the WC_SD1_CI1_UD and the WC_SD1_CI1_UD_G cells, respectively, except additionally the origin of the capture data is selectable between CFI and CFO. See Figure B.8.

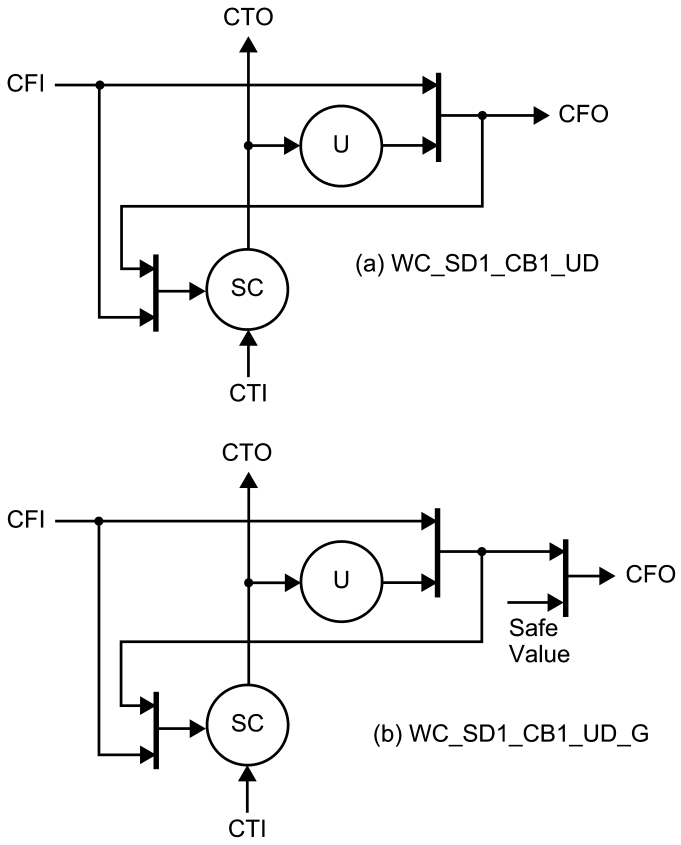


Figure B.8—WC_SD1_CBI_UD WBR cells

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Annex C

(informative)

Relationship of IEEE Std 1500 to IEEE Std 1149.1 and IEEE Std 1687

C.1 Introduction

This standard is related to other IEEE test standards, most notably IEEE Std 1149.1 and IEEE Std 1687.

IEEE Std 1149.1 has proven to be very valuable to the electronics industry and has been widely adopted. While the objectives of both IEEE Std 1149.1 and IEEE Std 1500 are to provide improvements in testing, their purpose and use differs significantly. IEEE Std 1149.1 has primarily focused on board testing and providing serial test access at the system on chip (SoC) level, for internal integrated circuit (IC) testing and debug. IEEE Std 1500 is primarily concerned with the access and control to enable testing of embedded cores and user-defined logic (UDL) within an IC. While IEEE Std 1687 is focused on access to test data registers for embedded instruments and providing an SoC level serial access network.

The IEEE 1500 architecture was designed to allow interface compatibility with the IEEE 1149.1 test access port (TAP) controller. Indeed, the wrapper's wrapper serial control (WSC) interface can correspond to the control outputs from the IEEE 1149.1 TAP controller. For example, the wrapper serial port (WSP) timing diagrams in Figure 42 through Figure 44 (see 14.3) are compatible with the Capture, Shift, Update protocol control issued from a TAP controller. Thus, while IEEE Std 1500 does not require or suggest it, the WSC interface can be operated by an IEEE 1149.1 TAP controller, via the dedicated TAP pins on the SoC. This would advantageously allow IEEE Std 1500 to be accessible at any point in the SoC life cycle.

IEEE Std 1687 was designed to use this same Capture, Shift, Update protocol of the IEEE 1500 WSP. IEEE Std 1687 does not focus on a prescriptive hardware architecture, so for example it does not specify a wrapper boundary register (WBR), as does IEEE Std 1500. IEEE Std 1687 rather focuses on describing the hardware architecture, for example how the test data registers and serial access network of the SoC are connected and accessed. Thus, IEEE Std 1687 is a complement to IEEE Std 1500 and IEEE Std 1149.1, in that it can be used to describe access to IEEE 1500 cores integrated within the SoC, and hierarchical access to data registers and subcores within an IEEE 1500 core and IEEE 1149.1 SoC.

C.2 Example of integrating IEEE 1500 cores within an SoC

The following shows an example of how IEEE 1500 cores can be integrated within an SoC. The example uses an IEEE 1149.1 TAP controller for serial access at the SoC level, and two test access mechanisms (TAMs) to integrate several cores within the SoC hierarchy. The TAMs are compliant with IEEE Std 1500 and utilize IEEE Std 1687 for the control logic in the TAMs.

Figure C.1 illustrates the SoC integration of several cores using TAM1 and TAM2. TAM1 is connected to the SoC TAP controller and is a first level of hierarchy in the SoC integration. TAM1 controls access to three cores: Core1.1, Core1.2, and Core1.3. It also controls access to TAM2, which is a second level of hierarchy in the SoC integration. TAM2 controls access to n cores, Core2.1 through Core2.n, that are daisy chained together as in Figure 49. Note that the cores shown here can also have subcores, for example Core1.1 can have hierarchical subcores, as shown in Figure 51 and Figure 52.

In Figure C.1 the TAP controller TRSTN (test reset) and IEEE 1149.1 test clock (TCK) are the source for all TCK and RSTN TAM inputs, and all core WRCK and WRSTN WSP inputs. These signals are not controlled or gated by the TAP controller or TAM logic. The ToSelect output from the TAP control to

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

TAM1 is an IEEE 1149.1 instruction decode, that selects TAM1 register bits as the TAP DR when this instruction is updated in the TAP controller's instruction register (IR). The ToCSUEn TAP outputs to TAM1 are the Capture, Shift, and Update Enables from the TAP controller's data register (DR) states, the TAP controller's IR states are not used to operate the TAMs and cores. With this method of integration, the IEEE 1500 wrapper instruction registers (WIRs) are accessed as TAP controller data registers, in the TAP controller's DR states, not in the TAP controller's IR states.

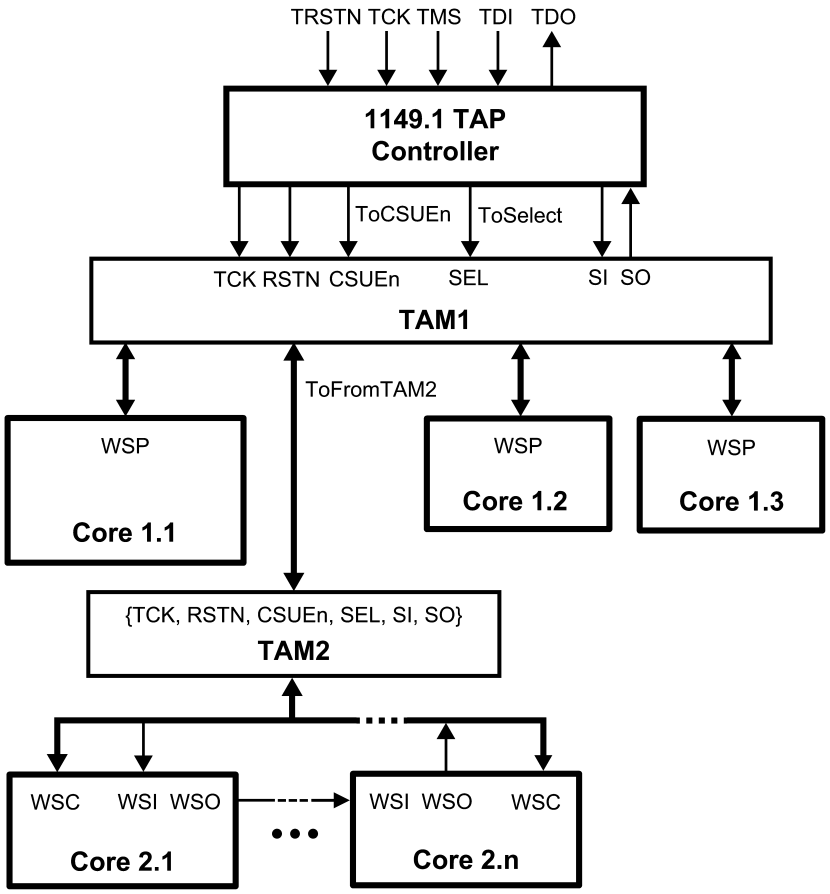


Figure C.1—SoC IEEE 1500 TAMs and core hierarchy

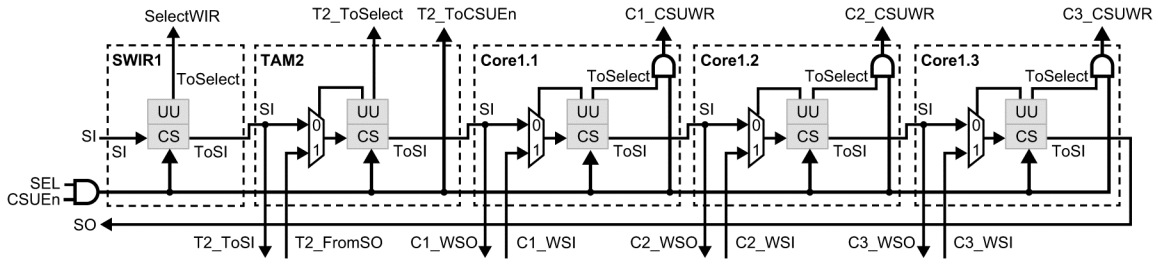


Figure C.2—TAM1 logic

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

Figure C.2 shows an implementation of the TAM1 logic. TAM1 has five register bits, one for the WSP SelectWIR to the cores (SWIR1 bit), one for TAM2 access, and one for each of the three cores accessed through TAM1. The SEL (select) input to TAM1 in Figure C.2 is the TAM1 instruction decode from the TAP controller. It is sourced by the TAP controller ToSelect output, as shown in Figure C.1. SEL gates the CSUEn signals for the TAM logic. The TAM1 CSUEn inputs are sourced from the TAP controller ToCSUEn outputs. Accordingly, when TAM1 is enabled by a TAP controller instruction, and consequently the SEL select is asserted, the CSUEn signals will operate the register bits of the TAM.

The SWIR1 register bit is similar to a SIB, but without the scan path mux. It provides a common SelectWIR input for the WSPs of Core1.1, Core1.2, and Core1.3. The ToSelect of the SWIR1 bit is used as this SelectWIR. It is controlled by shifting in a logic 1 or 0 value into the SWIR1 bit and updating it. This will then provide the SelectWIR value for the subsequent Capture, Shift, and Update operations of the three cores.

The TAM1 register bits that select TAM2, and the three cores, are inline SIBs similar to that of Figure 53. The TAM2 bit provides ToSelect and ToCSUEn outputs for the TAM2 register logic. The signals of the TAM2 bit connect to, and control, the interface of TAM2, as shown in Figure C.1. The ToSelect of the three cores' SIBs are used to gate the CSUEn signals for the TAM bits and generate the three CSUWR (i.e., CaptureWR, ShiftWR, and UpdateWR) signals for the WSPs of the associated core. The CSUWR signals for each of the three cores are enabled by shifting in a logic 1 or 0 value into the associated SIB and updating it. This will then enable or disable the CSUWR value for the subsequent Capture, Shift, and Update operations of the associated core.

When TAM2, or one or more of the cores are selected by their SIB, their scan path is inserted into the TAM1 SI-SO scan path, as shown by the scan path multiplexors in Figure C.2. Any combination of TAM2 and the cores can be selected, and each of the selected paths are daisy chained together in the order shown in the figure.

C.2.1 Steps to access Core1.2 and Core1.3

Below are the steps to access Core1.2 and Core1.3 from the TAP controller.

1. **Reset:** Reset the TAP controller and IEEE 1500 TAM network by asserting TRSTN to logic 0, and then deasserting to logic 1
   - This will reset the TAP, TAM1, TAM2, and all of the IEEE 1500 cores
   - The TAMs and cores will be unselected
   - The TAP controller DR will be either BYPASS, or IDCODE if it is implemented
2. **Select TAM network:** Update the TAP controller IR with the instruction used to select the IEEE 1500 TAM network
   - The ToCSUEn outputs of the TAP controller do not operate yet, as the TAP FSM is moving through the IR states
   - On the falling edge of TCK in the UpdateIR state, now the ToSelect to TAM1 is asserted, selecting TAM1
3. **Select TAM1 as DR:** Moving the TAP controller from UpdateIR to the SelectDR state will select the five bits of TAM1 as the DR selected between TDI and TDO
4. **Enable access to TAM1 cores:** Shift and update the TAM1 bits with 5'b10011; this will enable Core1.2 and Core1.3 to be inserted into the scan path and their WIRs will be selected, as SWIR1=1

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

5. **Insert Core WIRs into scan path:** Move the TAP controller from UpdateDR back to the SelectDR state, the DR scan order inserted between TDI and TDO is now:
   — TDI→SWIR1→TAM2 SIB→Core1.1 SIB→Core1.2 WIR→Core1.2 SIB→Core1.3 WIR→Core1.3 SIB→TDO
6. **Load Core1.2 and Core1.3 instructions:** Shift and update the TAM1, Core1.2, and Core1.3 scan path with: {3'b000, Core1.2 WIR, 1'b1, Core1.3 WIR, 1'b1}
   — This will update an instruction in Core1.2 WIR and Core1.3 WIR and select the associated WDRs in these cores for the next shift of the network, as SWIR1=0 is updated here
   — TAM2 and Core1.1 remain deselected, while Core1.2 and Core1.3 remain selected for the next shift
7. **Insert Core WDRs into the scan path:** Move the TAP controller from UpdateDR back to the SelectDR state, the DR scan order selected between TDI and TDO is now:
   — TDI→SWIR1→TAM2 SIB→Core1.1 SIB→Core1.2 WDR→Core1.2 SIB→Core1.3 WDR→Core1.3 SIB→TDO

In Figure C.1, the implementation of TAM2 is similar to that of TAM1. TAM2 will have only two register bits, one register bit for a WSP SelectWIR (SWIR2 bit) to the cores, and one SIB for selection of the daisy chained cores, Core 2.1 through Core 2.n. The TAM2 logic is shown below in Figure C.3.

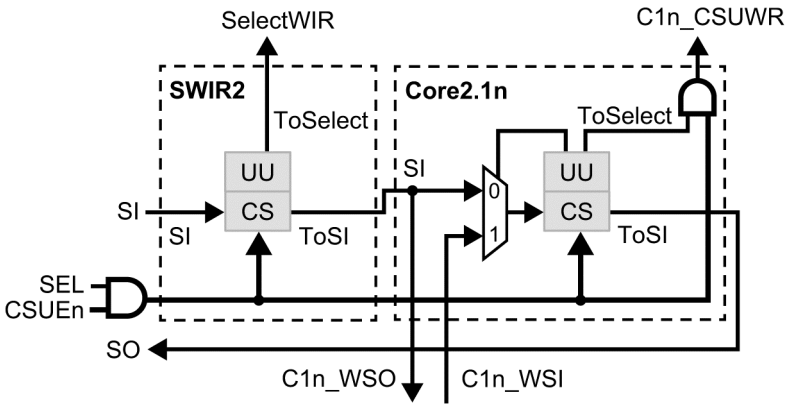


Figure C.3 —TAM2 logic

C.2.2 Steps to access Core1.1 together with Core2.1 through Core2.n

Below is an example of the steps used to access Core1.1 together with Core2.1 through Core2.n from the TAP controller.

1. **Reset:** Reset the TAP controller and IEEE 1500 TAM network by asserting TRSTN to logic 0, and then deasserting to logic 1
   — This will reset the TAP, TAM1, TAM2, and all the IEEE 1500 cores
   — The TAMs and cores will be unselected
   — The TAP controller DR will be either BYPASS, or IDCODE if it is implemented

Copyright © 2022 IEEE. All rights reserved.
Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

2. **Select TAM network**: Update the TAP controller IR with the instruction used to select the IEEE 1500 TAM network

   — The ToCSUEn outputs of the TAP controller do not operate yet, as the TAP FSM is moving through the IR states

   — On the falling edge of TCK in the UpdateIR state, the ToSelect to TAM1 is asserted, selecting TAM1

3. **Select TAM1 as DR:** Moving the TAP controller from UpdateIR to the SelectDR state, will select the five bits of TAM1 as the DR selected between TDI and TDO

4. **Enable access to TAM1 core and TAM2:** Shift and update the TAM1 bits with 5'b11100, this will enable Core1.1 WIR (SWIR1=1) and TAM2 register bits to be inserted into the scan path

5. **Insert Core1.1 WIR and TAM2 into scan path:** Move the TAP controller from UpdateDR back to the SelectDR state, the DR scan order selected between TDI and TDO is now:

   - TDI→SWIR1→SWIR2→Core2.1-n SIB→TAM2 SIB→Core1.1 WIR→ Core1.1 SIB→Core1.2 SIB→Core1.3 SIB→TDO

6. **Load Core1.1 instruction and load TAM2 register bits:** Shift and update the TAM1, Core1.1 and TAM2 scan path with:{4'b0111, Core1.1 WIR, 3'b100}

   - This will update an instruction in the WIR of Core1.1 and select the associated WDR in Core1.1 for the next shift of the network, as SWIR1=0 is updated here
   - Core2.1 through Core2.n will be selected and their WIRs are enabled to be inserted into the scan path for the next shift of the network, since SWIR2=1 is updated here
   - TAM2 and Core1.1 remain selected, while Core1.2 and Core1.3 remain deselected for the next shift of the network

7. **Insert Core1.1 WDR and Core2.1-n WIRs into the scan path:** Move the TAP controller from UpdateDR back to the SelectDR state, the DR scan order selected between TDI and TDO is now:

   - TDI→SWIR→SWIR2→Core2.1-n WIR→Core2.1-n SIB→TAM2 SIB→ Core1.1 WDR→Core1.1 SIB→Core1.2 SIB→Core1.3 SIB→TDO

Note that in step 7 above, Core1.1 is shifting its WDR while Cores 2.1 though n are shifting their WIRs. This does not follow the recommendation of 15.2.2(b). This was done because of the sequence of accessing hierarchy, as Core1.1 can load an instruction before Core2.1 through Core2.n are selected through TAM2. The recommendation can be followed by keeping SWIR=1 in step 6. This keeps the Core1.1 WIR selected for both the shift in step 6 and the next shift in step 7. So, there is an unnecessary shift of the Core1.1 WIR in step 6, however then the Core1.1 and Core2.1 through Core2.n WIRs would be shifted together, reloading a Core1.1 instruction in step 7.

Copyright © 2022 IEEE. All rights reserved.

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.

IEEE SA
STANDARDS ASSOCIATION

RAISING THE WORLD'S STANDARDS

Connect with us on:

Twitter: twitter.com/ieeesa
Facebook: facebook.com/ieeesa
LinkedIn: linkedin.com/groups/1791118
Beyond Standards blog: beyondstandards.ieee.org
YouTube: youtube.com/ieeesa

standards.ieee.org
Phone: +1 732 981 0060

IEEE

Authorized licensed use limited to: Asian Institute of Technology. Downloaded on October 14,2025 at 18:26:26 UTC from IEEE Xplore. Restrictions apply.